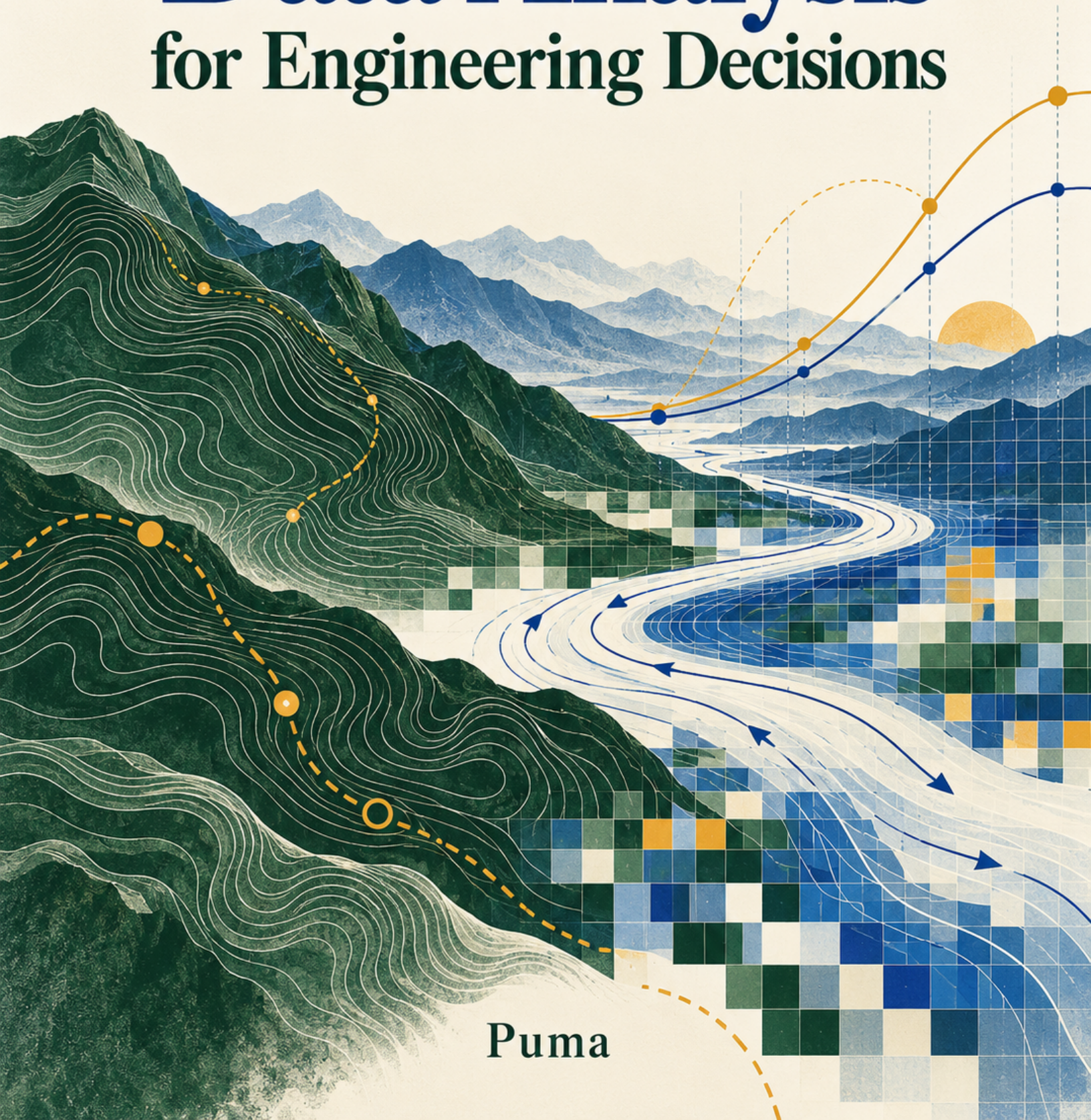


Practical Data Analysis for Engineering Decisions



Puma

Practical Data Analysis for Engineering Decisions

Puma

Expanded edition

A practical path from engineering traces
to mathematical analysis

Scheduler traces · Agent evaluation · Research data
Vectors and matrices · Images · GIS
Finance · Optimization · Uncertainty

2 October 2026

18 chapters, 16 original figures, and a reproducible companion

Contents

Start here	4
A route through the expanded edition	5
1 Begin with a decision	6
2 Measure what happened	9
3 Make a trustworthy dataset	12
4 See the shape of behavior	16
5 Compare policies fairly	19
6 Explain a losing scheduler run	22
7 Find out what a change causes	29
8 Analyze a system over time	32
9 Evaluate agents and their judges	35
10 Analyze training data and model results	40
11 Collect and interpret research data	43
12 Turn findings into an engineering decision	46
13 Practice with the companion	48
14 Reason with vectors and matrices	53
15 Measure images as data	60
16 Analyze places and spatial relationships	66
17 Analyze cash flows returns and risk	75
18 Optimize under uncertainty	86
Mathematical reference and practice	92
Sources and evidence	95

The editable manuscript and reproducible companion accompany this book. All scheduler measurements come from a pinned synthetic simulation; other generated examples are labeled explicitly.

Start here

This book is for someone who can build software but is beginning to learn data analysis. The aim is practical: inspect what went into a system, what it did, and what came out; find the pattern that matters; then decide what to change and how to check it.

Our running example is a scheduler whose elaborate policy sometimes loses to a simple one. We investigate possible measurement errors, unfair comparisons, missing scheduling actions, inaccurate predictions, and a score that rewards the wrong thing.

The same methods apply to evaluating agents and their judges, understanding training results, and collecting research data. Chrema and Nearwork appear as possible applications based on the needs described for them. Their private implementations were not inspected. Those examples are proposals, not claims about their current code or data.

What is real in this book

The scheduler case uses actual reports and event traces from a Python simulator. These are measured simulator results, not measurements of a production service or real GPU. Simulation exposes mechanisms and enables controlled comparisons, but can omit effects that matter in reality.

Other datasets are labeled synthetic. They isolate concepts and make calculations reproducible. A convincing chart made from invented data remains an illustration; it is never evidence that a real project works.

A route through the book

Read Chapters 1 through 6 in order, from a question to a complete investigation. Chapters 7 and 8 cover experiments and behavior over time. Read Chapters 9 through 11 for your current project. Chapter 12 turns findings into a decision; Chapter 13 provides labs, solutions, a glossary, and further reading.

The companion includes data, executable analysis, and the numbers behind the figures. Run it unchanged, then alter one question or chart. A small analysis you understand is a better starting point than a complicated notebook you cannot explain.

Four sentences to keep separate

An observation states what was measured. An interpretation proposes a reason. An experiment tests a change under stated conditions. A decision says what you will do given the evidence, costs, and remaining uncertainty.

For example: “This run took longer” is an observation. “CPU placement may have caused the delay” is an interpretation. “We changed only the placement rule and reran matched workloads” describes an experiment. “Keep the simple policy while testing a revised objective” is a decision. Many bad analyses become plausible by silently moving from the first sentence to the last.

A route through the expanded edition

The first thirteen chapters develop the central habit of this book: begin with a decision, build trustworthy measurements, compare fairly, and turn evidence into a bounded next action. The additional chapters extend that practice to mathematical representations and new kinds of data. You can use them when your project needs them without postponing your first useful analysis.

Chapter 14 begins with a table of run measurements and introduces vectors, projections, least squares, covariance, PCA, and clustering. Chapter 15 turns pixels into a physical measurement and shows how filtering, segmentation, geometry, and calibration change the answer. Chapter 16 adds location: coordinate systems, spatial relationships, surfaces, and the sampling decisions behind a map. Chapter 17 follows cash flows and returns through time, with explicit attention to compounding, uncertainty, and honest backtesting. Chapter 18 chooses a worker count under a model, then asks how numerical error and uncertain inputs affect the choice.

The calculations use small worked numbers before general notation. The extra companion examples are synthetic and identified as such. They are meant to expose the arithmetic and assumptions, while the original scheduler investigation remains tied to its pinned reports and traces. Each project ends with a decision or a validation question so that the mathematics stays connected to engineering work.

If mathematical notation is unfamiliar, use the reference at the end when a symbol appears. Read a formula as a sequence of operations on quantities with names and units. Then run the example, check its known answer, and change one assumption. Understanding which changes ought to affect the result is a strong defense against code that runs successfully and answers the wrong question.

1 Begin with a decision

The first useful question

“Analyze the data” is too broad to guide an investigation. It does not tell you which rows matter, what comparison is fair, or what a useful answer would change. Begin with a decision you could actually make: should we replace the scheduler policy, adjust a scoring term, add a faster execution path, or leave the current design alone?

A workable first question is: “On workloads we expect to run, does the new policy reduce completion time enough to justify its extra planning cost, while keeping failures and long waits acceptable?” This already contains a population, an alternative, an outcome, a cost, and constraints. Each part can become a measurement.

Avoid trying to answer every version of “better” at once. A policy may finish the whole workload sooner while making the typical task slower. It may improve average speed but create rare unacceptable waits. It may reduce simulated completion time while taking much more real time to make a decision. These are different outcomes. Data will not choose your priorities for you.

A complete investigation in miniature

Consider five tasks run under policies A and B. The following numbers are synthetic task latencies in seconds, matched by task identifier:

Task	Policy A	Policy B	B minus A
t1	4	3	-1
t2	5	4	-1
t3	6	5	-1
t4	7	6	-1
t5	18	22	4

Both policies have mean latency 8 seconds: the sums are both 40 and there are five tasks. B has a better median, 5 instead of 6 seconds. It wins on four of five tasks, but its slowest task takes 4 seconds longer. If a dashboard shows only the mean, you will see no difference. If it shows only the win rate, B looks excellent. If the last task is the urgent one, that recommendation may be wrong.

Suppose the product requirement says that every task in this tiny test must finish within 20 seconds. A passes; B fails. That does not prove A is universally better. It answers a narrower question under an explicit constraint. Suppose instead that the requirement values typical

responsiveness and permits the observed tail. The same measurements could justify a different choice. This is why the decision comes first.

Now ask whether these were fair trials. Did both policies see the same inputs? Were the machine and model already warm in one run but cold in the other? Were failed tasks included? Did the policy that ran second benefit from a cache? These questions can matter more than choosing a sophisticated statistical test.

Finally, inspect `t5`. Its trace may show a long queue, a slow tool call, a retry, or an unfavorable resource assignment. A summary tells you that an important difference exists. A trace helps you construct a testable explanation. You then change one relevant mechanism and rerun. That is a complete analysis loop, even with a spreadsheet and five rows.

Translate an intention into a measurement

A metric is a rule for turning observations into a number. It needs more than a name. “Latency” might mean submission to first response, ready to execution start, execution time alone, or submission to an accepted final result. All could be useful; mixing them invalidates the comparison.

Write a metric contract before implementation. For each important metric, state the unit of observation, start and end events, physical unit, eligible population, treatment of failures, aggregation rule, and desired direction. For example: “For every submitted workflow, measure seconds from accepted submission to accepted final output; count incomplete workflows separately at the observation cutoff; report median, a tail percentile, and completion fraction.”

There are two kinds of units here. A physical unit is seconds, bytes, dollars, or a fraction. An observational unit is a task, attempt, workflow, user, machine, or independent experimental run. “100 observations” is almost meaningless until you know what was observed. One run can produce a million log messages without becoming a million independent tests.

Choose one primary outcome for the decision. Add guardrails that can veto an apparently good result, such as correctness, failure rate, memory limits, or maximum waiting time. Keep diagnostic metrics, such as cache hits or model loads, because they help explain changes. A diagnostic metric does not automatically deserve to be optimized.

Write the analysis plan in ordinary language

A useful plan fits in a few paragraphs. State what decision is pending, what alternatives will be compared, what inputs will be held constant, and what evidence would change your mind. Define the smallest improvement worth acting on. A 0.1 percent speedup may be irrelevant if the new policy doubles maintenance cost. A 5 millisecond improvement may matter greatly on a critical interaction path.

Then state what you will not know after the test. A short simulation can compare mechanisms under its own assumptions. It cannot establish device throughput, production stability, or user satisfaction. A labeled agent test can measure behavior on those tasks. It cannot establish that unseen tasks have the same distribution.

This does not weaken the analysis. It makes the result usable. An engineer needs to know whether a number supports changing a line of code, running a wider experiment, or shipping a new default.

Baselines are part of the argument

A baseline is the alternative you would use without the proposed change. First-in-first-out is a natural scheduler baseline because it is easy to understand. It may not be the strongest practical alternative. A simple policy that avoids unnecessary model reloads could perform better without the complexity of a planner.

Compare against a plausible simple competitor as well as the current implementation. If a new system barely beats a weak baseline and loses to a stronger simple one, the result is a design lesson. It suggests that the useful insight may be much smaller than the architecture built around it.

Keep baseline capabilities aligned. If one policy gets fairness protection, priority boosts, retry handling, or a larger resource budget and another does not, the comparison is between bundles of features. That can be a legitimate product comparison, but it cannot identify the effect of just the scheduling algorithm.

What to produce

Your first deliverable should be a small evidence package: a question, a metric contract, a table with all eligible cases, one or two informative plots, a statement of what is observed, and a next test or decision. A notebook with many charts is not automatically a better package. The reader should be able to follow the chain from source events to conclusion.

Exercise: choose one system you are building. Complete this sentence: “I need to decide whether to ___ rather than ___, for ___, because ___ is the outcome that matters, subject to ___.” If you cannot finish it, the next useful work is clarifying the decision rather than collecting more data.

2 Measure what happened

Build an account of work

A system is easier to analyze when its records preserve the life of a piece of work. For a scheduler, a task may be submitted, become ready, wait, load a model, execute, drain output, and have its result accepted. It may also fail, retry, be cancelled, or produce a stale result that is correctly rejected. Those are different events with different meanings.

Start with identifiers. A workflow identifier ties together dependent tasks. A task identifier refers to logical work. An attempt identifier distinguishes retries. A run identifier identifies the policy and environment used for an experiment. A trace identifier can connect events across components. Do not overload one field to mean all of them.

An accepted result is especially important. A worker can finish computation after a lease expires or another attempt succeeds. Counting every worker completion as delivered work inflates throughput. For systems with retries, record both executed work and accepted useful output. Extra execution can increase cost without increasing the user's benefit.

OpenTelemetry's trace model provides a useful vocabulary: a trace connects an operation's path, spans represent operations with start and end times, and attributes attach context. You do not have to adopt a particular telemetry stack to benefit from those distinctions. Our book's event schema is deliberately small and domain-specific. [OpenTelemetry traces](#)

An event schema you can reason about

A practical event record contains an event identifier, schema version, run identifier, timestamp, event type, relevant entity identifiers, and a small set of event-specific fields. A phase-start event might include task, attempt, host, device, model, and phase. A policy-decision event might include the eligible candidates, their scores, the chosen action, and the reason other actions were excluded.

Record the facts available at decision time. If the scheduler predicted an 8 second duration but observed 13 seconds later, keep both fields with clear names. Never overwrite the original prediction with the observed duration. Doing so destroys the evidence needed to evaluate prediction quality or detect leakage.

For a policy decision, the complete candidate universe can be large. You may keep a compact structured record of candidate identifiers and score components, plus a separate artifact for full detail. What matters is that a future investigation can distinguish “the good action was unavailable,” “it was feasible but not generated,” and “it was scored lower.” These lead to different fixes.

Keep schema changes explicit. If version 1 measures completion at worker finish and version 2 measures accepted delivery, a single continuous latency chart hides a change in meaning. Preserve the old definition, add the new definition, and annotate the transition. A versioned metric can be more trustworthy than a superficially uninterrupted time series.

Time has more than one meaning

Wall-clock time identifies when an event happened in the outside world. A monotonic clock measures elapsed duration without moving backward when the system clock changes. Simulated time is the clock inside a model. They are not interchangeable.

Use a monotonic clock for local elapsed durations. Distributed clocks may disagree, so a negative duration formed by subtracting timestamps from different machines is a data-quality warning rather than evidence of time travel. Preserve clock source and synchronization assumptions when you need cross-host timing.

The scheduler case has a particularly important distinction: simulated task durations and measured planner execution times. The simulator can advance its model clock while spending real CPU time computing a decision. Unless the simulator explicitly charges planner time to the modeled schedule, adding the planner's milliseconds to simulated makespan is not a valid correction. Report both and explain how a deployment would account for planning overhead.

Choose and label units at the boundary. Store seconds or milliseconds consistently. Bytes, decimal gigabytes, and binary gibibytes are different units. A number such as 8.5 in a memory field is incomplete evidence until its unit and meaning are known: physical allocation, reservation, peak usage, or model size.

Derive durations from events

For a simple task with one accepted attempt, define submission latency as accepted-result time minus submission time. Define ready waiting time as dispatch time minus ready time. Define compute time as compute-end minus compute-start. Define setup time from actual load intervals, not from a guess based on whether a model name changed.

These components may overlap. A GPU can load one model while a CPU computes another task. A task's output drain may overlap another task's computation. Summing all task durations does not give wall-clock makespan when work runs concurrently. A timeline makes the overlap visible.

Similarly, waiting time is not necessarily submission-to-start. A task whose dependencies are unfinished is not ready to run. Keep dependency-blocked time separate from ready-queue time if the question concerns scheduler responsiveness. Otherwise you can blame a scheduling policy for work it was not yet allowed to schedule.

For retries, decide whether the user-facing latency starts at the original submission and ends at the accepted result. Usually that is the useful quantity. Attempt-level duration is still valuable for diagnosing execution failures. It belongs in a separate table, with a many-to-one relationship to the logical task.

Counters, gauges, and distributions

A counter accumulates events, such as accepted results or failures. A gauge describes a current level, such as queue length or allocated memory. A distribution retains variation across observations, such as latency. These types answer different questions.

A high total completion count may simply mean that a system ran longer. Divide by an appropriate observation duration to obtain a rate. A high average resource usage can coexist with severe short bursts that cause failures. Preserve peaks and a time series where burst behavior matters. A low mean latency can coexist with a long tail. Preserve the distribution.

Do not average percentiles from separate groups and call the result the combined percentile. The percentile of a union of observations depends on the full distributions and group sizes. Histograms can support aggregation when their bucket definitions align; precomputed quantiles generally cannot be combined that way. [Prometheus histograms and summaries](#)

Measure absence as carefully as presence

An event stream naturally contains what occurred. It may omit what never finished. If you calculate latency only from completed tasks, a policy can look faster by leaving hard tasks unfinished. Always reconcile submitted, completed, failed, cancelled, and still-pending work at a stated cutoff.

At cutoff time, an incomplete task has a known minimum age but an unknown eventual completion time. Record its state and age. Do not replace its latency with zero, delete it silently, or pretend the cutoff is its completion. Later chapters discuss censored observations and decision-oriented alternatives such as completion probability by a deadline.

Logging itself can fail. Include counts that reconcile producer and collector views where possible. A missing phase-end event might mean the process crashed or the telemetry was lost. Those are different explanations with the same visible symptom. Keep a quality flag until you can distinguish them.

Instrument economically

More telemetry is not always better. Fine-grained traces can consume storage, slow the system, and retain private information. Start with fields tied to a decision or a known debugging need. Prefer structured identifiers and compact result summaries to raw user content. Record sampling rules because a sample enriched for errors cannot estimate the ordinary error rate without accounting for that design.

For a first implementation, add a run manifest and a small set of lifecycle events. Then write a validator that checks whether every observed accepted result can be traced back to a submission and a compatible attempt. A measurement system that can explain its own gaps is more useful than one that merely produces many records.

Exercise: draw the lifecycle of one task in your system. Mark which transitions are observed, inferred, or missing. Circle every duration whose endpoints are on different clocks. Those circles identify where you need a measurement decision before a performance conclusion.

3 Make a trustworthy dataset

Keep raw evidence separate

Treat the raw input as an immutable record. Copy it into a versioned analysis snapshot, record a cryptographic hash, and do transformations elsewhere. A hash does not prove that data is correct; it proves which bytes your analysis used. That is enough to prevent an important class of confusion when reports are regenerated during development.

Keep a manifest with the source, collection or run time, software revision, configuration, random seed, relevant environment, and interpretation notes. If some of these are unavailable, say so. An honest missing field is better than a guessed revision identifier.

A simple directory layout has raw inputs, derived tables, analysis code, figures, and a short README. Raw inputs should not change when you rerun analysis. Derived outputs may be rebuilt. The separation lets you test whether a changed result came from different data, different transformation logic, or different plotting choices.

Decide what one row means

A table becomes much easier to use when each column holds one variable and each row is one observation of a defined kind. A task table, an attempt table, and an event table are different observational levels. The tidy-data principle formalizes that separation. [Wickham, Tidy Data](#)

For the scheduler, use a run table with one row per policy, workload, seed, and configuration combination. Use a task table with one row per logical task within a run. Use a phase table with one row per observed phase interval. A decision table can hold one row per planning decision, and a candidate table one row per candidate within that decision.

Do not force everything into one giant flat table. If a task has three phases and five candidate scores, joining both onto the task without care creates fifteen rows. A later sum can count its latency fifteen times. This is a data-model error that a beautiful chart will not reveal.

Choose keys explicitly. A task identifier may repeat across runs, so the key is usually run identifier plus task identifier, not task identifier alone. An attempt adds another key. Before a join, state whether it should be one-to-one, many-to-one, or many-to-many.

Validate before calculating

Validation checks whether the data satisfies expectations that must be true before interpretation. Begin with key uniqueness, allowed values, required fields, units, nonnegative durations, and event ordering. Then add domain checks: accepted work must have a submission; phase ends need matching starts; completion counts must agree across tables; and resources must stay within modeled capacity when the model promises that constraint.

A failed check is useful information. Do not delete the row simply to make the code continue. Put suspect records in a quarantine output with the reason, and decide whether they prevent

answering the question. For a performance comparison, missing all the slowest results is a serious problem. A missing optional model label may only prevent one cohort breakdown.

Different checks have different severity. A duplicate run key should usually stop a policy comparison. A missing optional diagnostic field may trigger a warning. Document the choice so another person knows what the analysis accepted.

In pandas, use explicit join validation when possible. For example, joining a task table to run metadata can use `validate="many_to_one"`. An outer merge with an indicator can reveal unmatched keys before you choose the intended analysis join. Be aware that pandas can match null join keys to each other, unlike usual SQL null behavior; reject or handle missing identifiers explicitly. [pandas merge documentation](#)

A small transformation you can audit

Suppose each event has `run_id`, `task_id`, `event`, and `time_s`. Extract arrival and accepted-result events into separate tables. Validate one arrival and one accepted result per logical task for the simple one-attempt case. Left-join completion onto arrival, preserving tasks that have not completed. Derive latency only where completion exists. Add `status` and `age_at_cutoff` for the rest.

The key choice is the left join from submissions. Starting with completions would hide unfinished work. An inner join is convenient because it yields clean durations; convenience is not a justification for changing the population.

A compact SQL pattern is:

```
SELECT
    s.run_id,
    s.task_id,
    s.submitted_s,
    c.accepted_s,
    CASE WHEN c.accepted_s IS NOT NULL
         THEN c.accepted_s - s.submitted_s END AS latency_s
FROM submissions AS s
LEFT JOIN accepted_results AS c
  ON s.run_id = c.run_id AND s.task_id = c.task_id;
```

This query assumes the tables have already passed key-uniqueness checks. SQL does not automatically protect you from duplicate matching rows. If `accepted_results` contains duplicate accepted outcomes, the join replicates submissions and corrupts aggregates.

Missing is not zero

A missing duration can mean not yet complete, not applicable, not instrumented, redacted, or corrupted. These are different states. Keep a reason field when the distinction changes interpretation. The value zero means the observed quantity was zero. It should not become a universal placeholder.

Consider research records with an optional quality rating. If difficult cases are less likely to receive a rating, the average among rated cases overstates overall quality. More rows will not

remove that selection problem. First inspect missingness by relevant groups and process stages.

Imputation replaces missing values with estimates. It can be useful in a carefully designed predictive pipeline, but it does not recover the true unobserved measurement. A beginner-safe first analysis reports missingness and uses an explicit complete-case subset only where its limitations are acceptable. For a decision-critical outcome, try to recover the data or measure bounds rather than silently filling it.

Work at the right aggregation level

A common first calculation is group-by followed by an average. Ask what each row contributes to the answer. Averaging one number per workflow weights workflows equally. Averaging all task latencies weights workflows with more tasks more heavily. Neither is universally correct. The decision determines which population should receive equal weight.

The same issue appears in agent evaluation. A task with ten repeated trials can contribute ten times as much as a task with one trial if you pool all rows. If your question is performance on a typical task, first summarize within task or use equal planned trial counts. If your question is the success rate of a random production attempt, the production attempt distribution may be the appropriate weight.

Always show the number of runs, tasks, and attempts separately. It prevents a large event count from being mistaken for broad coverage.

When the data stops fitting in memory

You do not need a distributed system just because a dataset is larger than a spreadsheet. First estimate its size, select only needed columns, filter the relevant time range, and avoid making unnecessary copies. A columnar file format can make repeated analytical reads more efficient because queries can read selected columns. A database can filter and aggregate near the stored data before returning a small result.

The important distinction is between the logical analysis and its execution. A group-by mean is a logical operation. It might run in pandas, SQL, or a larger processing system. Learn the operation and its assumptions first; then choose the smallest tool that handles the volume and update pattern reliably.

pandas documents strategies such as efficient dtypes, loading fewer columns, and chunking. Chunking works naturally when each chunk can be reduced to a small summary that can be combined correctly. More complex global operations need different treatment. [pandas scaling guidance](#)

For example, the overall mean of a nonmissing numeric field can be computed from its total sum and total count across chunks. Do not average the chunk means unless all chunks have the same nonmissing count. The final chunk is often smaller, and missingness can change counts within every chunk. Keep missing and invalid counts alongside the valid sum.

A global percentile cannot generally be recovered by averaging chunk percentiles. For exact quantiles, use an appropriate database, sorting procedure, or retained data. Approximate

streaming quantiles can be useful at scale, but their approximation method and error properties become part of the metric definition.

Large joins deserve care. Filter and deduplicate keys before joining, inspect expected cardinality, and materialize a small diagnostic sample before launching an expensive computation. A many-to-many mistake can turn a manageable dataset into an enormous intermediate result. Faster hardware does not repair the wrong relationship between tables.

Keep an economical development loop. Validate the analysis on a small representative extract and known edge cases, then run the same logic over the full eligible population. A random sample can miss rare failures, so also inspect important error strata and boundary cases. Label sample-based exploratory results separately from full-population calculations. Do not treat a dataset that fits in memory as necessarily representative, or one that needs a cluster as necessarily informative.

Reproducibility is a chain

A trustworthy result can be followed from an input hash to transformation code, a derived table, a calculation, and a figure. Every figure should identify its data population, units, and exclusions. Keep the code that generated it. Manual edits to a plot can be useful for publication polish, but they should not change the data or hide inconvenient points.

Use relative paths, fixed seeds for synthetic examples and resampling, and a clear command that reproduces the outputs. Record installed library versions. A seed makes a particular random computation repeatable under an implementation; it does not make the experiment valid or guarantee identical results across all versions and hardware.

Exercise: make three tests for an existing dataset: one key test, one arithmetic reconciliation, and one population test. The population test should answer, “Which expected observations are absent?” It is often the most informative of the three.

4 See the shape of behavior

Explore before compressing

Exploratory data analysis means looking for structure, exceptions, and problems before committing to one summary or model. Graphics are central because a mean and a standard deviation can conceal very different shapes. The NIST engineering-statistics handbook treats plots and assumption checks as part of understanding the data rather than decoration after a calculation. [NIST exploratory data analysis](#)

Start with counts and ranges. How many eligible records are present? What are the minimum and maximum values? Which categories dominate? Are there impossible values or a surprising mass at exactly zero? Are values repeated because of rounding, simulation defaults, or duplicate records? A simple sorted table is often the fastest first plot.

Then look at a distribution. Latency is often right-skewed: many ordinary cases and a smaller number of much slower ones. The mean is sensitive to those slow cases, which may be exactly what matters for total resource cost. The median describes the middle case, which may better describe typical responsiveness. Neither is the “correct” statistic for every decision.

Read an empirical cumulative distribution

An empirical cumulative distribution function, or ECDF, answers: what fraction of observations are at or below this value? Sort the observations. At each observed value, plot the cumulative fraction. No bins are required.

For latency, pick a time on the horizontal axis and read upward to find the fraction completed within it. Pick a fraction on the vertical axis and read across to find an approximate percentile. A curve farther left generally represents smaller values, but curves can cross. Crossing curves tell you that the winner depends on which part of the distribution matters.

In the five-task example, B is faster for most tasks but has the slower maximum. An ECDF preserves that tradeoff. A single bar labeled “average latency” erases it.

At small sample sizes, an ECDF has large steps. That is honest. A smoothed density curve can make a dozen observations look like a richly known population. Show points or steps when data is sparse. A percentile near the extreme of a small sample is essentially a statement about one or two observations, not a stable description of a production tail.

Histograms and box plots have jobs

A histogram groups values into bins and shows how often each range occurs. It helps reveal multiple modes, skew, and outliers. The choice of bin width changes the picture, so inspect more than one reasonable choice. Keep bins consistent when comparing policies.

A box plot compresses a distribution into a median, quartiles, and whiskers under a stated convention. It is useful for comparing many groups. It can hide multimodal behavior and

sample size, so add points or counts when groups are small. A box plot of three runs does not provide rich evidence just because the software draws a box.

Logarithmic axes are useful when values span orders of magnitude. They emphasize ratios rather than absolute differences. Label them clearly and remember that zero cannot be plotted on a log axis. Do not quietly drop zeros or negative differences to make a log plot work.

Choose the visual encoding from the question

Use a timeline when the question is what happened first, what overlapped, and where time was spent. Use an ECDF when the question is how a distribution changes. Use a paired dot or line plot when the question is how each matched workload changed. Use a scatter plot when the question is whether two quantities move together. Use a heatmap when two categorical or ordered dimensions organize many comparable cells.

A stacked bar can show how a fixed accounting total is divided among phases. A Pareto plot can show quality and cost together. A calibration plot can compare predicted confidence with observed frequency. Each visual has a reason to exist; a chart gallery without a question is a distraction.

For bars representing magnitudes, start the quantitative axis at zero unless there is a compelling, clearly communicated reason. For small differences around a reference, a dot plot with an explicit scale is often better than a truncated bar. Do not use 3D perspective to display ordinary 2D quantities. It makes value comparisons harder without adding evidence.

Compare cohorts before telling a story

A cohort is a group defined by something relevant: CPU versus GPU tasks, cold versus warm starts, short versus long inputs, task family, model size, customer class, or collection source. Break down an aggregate when those groups plausibly behave differently.

Here is a synthetic example of a composition reversal. Policy A succeeds on 90 of 100 easy tasks and 1 of 10 hard tasks. Policy B succeeds on 19 of 20 easy tasks and 20 of 100 hard tasks. B is better within both groups: 95 versus 90 percent on easy tasks and 20 versus 10 percent on hard tasks. Yet its overall rate is 39 of 120, about 32.5 percent, below A's 91 of 110, about 82.7 percent. B received many more hard tasks.

This is a form of Simpson's paradox. The aggregate uses different group weights. The practical lesson is to inspect workload composition before attributing an aggregate change to the policy. You can compare within groups and, when justified, calculate both policies under the same explicitly chosen group weights.

Do not create dozens of cohorts after seeing the data and treat the best-looking result as a confirmed finding. Exploration is good for finding hypotheses. Confirmation needs fresh evidence or a procedure that accounts for the search.

Outliers are questions

A very slow task may be a data error, a legitimate rare event, an operational failure, or the most valuable clue in the dataset. Inspect it before removal. A timeout recorded as a completed duration is different from a true completion at that duration. A stale worker result is different from an accepted result. A memory-unit mistake can look like a dramatic resource spike.

If you exclude a point, record the reason and show how the conclusion changes with and without it. “It looked too large” is not a sufficient rule. Predefined invalid-data criteria are safer than criteria chosen to improve the result.

Pay attention to points that disagree with your preferred explanation. If model reloads supposedly explain slowness, find the slow runs with few reloads and fast runs with many reloads. These cases reveal whether the explanation is incomplete.

Write a caption that makes a claim checkable

A good caption tells the reader what was plotted, which observations are included, the unit, and the bounded takeaway. For example: “Task submission-to-accepted-result latency in one simulated workload, 12 completed tasks per policy. The proposed policy has a lower middle latency but a later upper tail. This is a descriptive within-workload comparison.”

A caption should not say that a chart proves a cause it cannot identify. If a scatter plot shows high queue length alongside high latency, it establishes an association in the plotted records. It does not tell you whether queue length caused the delay, a slow service caused the queue, or both responded to a traffic burst.

Exercise: take one chart you have made or seen. Write the question it answers in one sentence. Then name one important question it cannot answer. If those two sentences sound almost identical, the chart may be encouraging an overclaim.

5 Compare policies fairly

Preserve the pair

If two policies run the same workload, the natural observation is their difference on that workload. Define difference as new minus baseline, so negative values favor the new policy for a duration metric. Calculate the difference before averaging. This preserves the pairing and makes good and bad cases visible.

Why does pairing help? A difficult workload may make both policies slow, while an easy workload makes both fast. Comparing their difference within the same workload removes much of that shared difficulty from the comparison. It does not remove a systematic bias such as always running one policy with a warm cache.

The scheduler follow-up contains three held-out seeds for each of four scenarios, three policies, and two service regimes. That is 72 report rows. It is not 72 independent pieces of evidence that the proposed policy is better. A comparison within one scenario and one regime has three matched workload pairs per baseline. The multiple policy rows are alternatives applied to the same workload, not extra workloads.

Absolute and relative effects answer different questions

If a baseline takes 50 seconds and a proposal takes 45 seconds, the difference is -5 seconds, the proposal-to-baseline ratio is 0.9, and the reduction relative to baseline is 10 percent. The baseline is 11.1 percent slower than the proposal because that statement uses 45 as the denominator. State the reference explicitly.

Averaging per-workload percentage changes gives each workload equal weight in percentage terms. Dividing mean proposed time by mean baseline time is a different calculation. A tiny 1 second workload and a 100 second workload can influence those summaries very differently. Choose the summary that matches the workload mix and decision; show the per-pair values so the reader can inspect the choice.

An equal-weight mean across scenario families answers a hypothetical question in which those families are equally likely. It does not estimate production performance unless production has that mix. If you know the real mix, report a weighted estimate and its source. If you do not, report the scenarios separately.

Variation is not uncertainty

Variation describes how observations differ. Uncertainty describes how much you do not know about a quantity you are trying to estimate. The spread of task latencies is variation. The uncertainty in a policy's mean improvement on future workloads is a different object.

A confidence interval is produced by a procedure with a stated long-run coverage property under its assumptions. A 95 percent confidence procedure would cover the target in 95 percent of repetitions in the relevant idealized setting. It is not a promise that the particular interval contains the truth, and it does not include every source of uncertainty. A narrow

interval from a biased sample can be precisely wrong. For paired observations, conventional mean-difference intervals are based on the distribution of within-pair differences. [NIST intervals for paired differences](#)

With only three held-out seeds, display all differences and be cautious about generalization. A numerical interval can still be calculated, but strong distributional assumptions or extremely limited resampling support may dominate its interpretation. This book does not use a tiny-sample interval to turn the scheduler follow-up into a deployment claim.

Bootstrap a quantity you can name

The bootstrap constructs repeated resamples from the observed data and recalculates a statistic. For paired runs, resample entire pairs, or equivalently their differences. Do not resample the new and baseline columns independently; that breaks the relationship you intended to use.

The companion includes 30 independent synthetic blocks. Their measured mean difference is about -1.240 seconds. A seeded 20,000-resample percentile bootstrap gives an illustrative 95 percent interval of approximately -2.508 to 0.059 seconds. These are generated teaching data. The interval crosses zero, so this particular method and sample do not clearly establish the sign of the population mean difference. The point estimate still describes the sample.

Here is the core calculation, assuming differences is a one-dimensional array containing one value per independent paired block:

```
rng = np.random.default_rng(82)
indices = rng.integers(0, len(differences),
                      size=(20000, len(differences)))
resampled_means = differences[indices].mean(axis=1)
low, high = np.quantile(resampled_means, [0.025, 0.975])
```

This simple percentile example teaches the mechanics. It is not a universal best method. Bootstrap variants differ, and small, highly skewed, dependent, or degenerate data can cause difficulties. SciPy documents several methods and explicitly supports paired resampling. Check a chosen method's assumptions rather than treating its output as automatic validation. [SciPy bootstrap](#)

If observations share a user, workflow, document, machine session, or collection batch, that group may be the resampling unit. Randomly resampling individual rows from a strongly correlated group makes the sample look more informative than it is. Time series may require blocks that preserve temporal dependence. If you cannot identify a defensible independent unit, pause before attaching an inferential interval.

A p value does not make the decision

A p value describes how incompatible the observed data are with a specified statistical model, often a no-effect model. It does not measure the probability that the hypothesis is true, the size of the benefit, or the importance of the result. The American Statistical Association warns against reducing scientific conclusions to whether a threshold is crossed. [ASA statement on p values](#)

For engineering, report the effect in useful units, uncertainty, cost, and guardrails. A tiny effect can be statistically detectable in a huge dataset but not worth maintaining. A potentially large effect can be uncertain in a small dataset and deserve a better experiment. “Not statistically significant” does not establish equivalence. If your decision requires proving that two alternatives differ by less than a tolerable amount, formulate an equivalence or noninferiority question with a justified margin and suitable design.

Searching changes what a result means

Suppose you try 40 parameter settings, inspect 15 metrics, and publish the best improvement. Even if none is genuinely better, some may look good through chance or idiosyncratic fit to the test cases. The issue is the selection process, not just one final comparison.

Separate exploration from confirmation. Use development workloads to find candidates. Freeze a promising choice and its primary outcome. Evaluate on a fresh set that was not used for tuning. Once you inspect that set and change the design in response, it becomes part of development history. You need new evidence for the next strong confirmation claim.

For a predeclared family of tests, methods such as Bonferroni can bound the chance of one or more false positives by allocating the error budget across comparisons. They trade sensitivity for control and do not repair biased data or an undefined search process. [NIST multiple comparisons](#)

Make the comparison auditable

A comparison table should state the policies, workload population, independent unit, number of pairs, common controls, metric definition, and direction of improvement. Show failures and missing pairs. Do not quietly compare different subsets after one policy fails to finish.

The scheduler's quantile convention also deserves a note. Its report uses nearest rank: sort values and select rank ceiling(p times n). For 12 completed tasks, its p95 selects the maximum. Its p50 selects the sixth value, whereas the conventional median of an even sample averages the sixth and seventh. The companion reproduces the report's convention rather than silently changing it. That small distinction is a useful example of measurement precision: two libraries can disagree without either being broken.

Exercise: choose a metric and write three different summaries you could report: mean difference, mean percentage change, and ratio of means. Explain which would be most relevant to your decision and why. Then identify the independent unit you would resample if you wanted an uncertainty interval.

6 Explain a losing scheduler run

State the loss precisely

The pinned diagnostic workload is randomized seed 7 in the supplied simulator. FIFO finishes the complete batch at 45.694653 simulated seconds. The proposed policy finishes at 56.642415 seconds, both before and after a candidate-generation repair. The difference is 10.947762 seconds, about 24.0 percent of FIFO's time. All 12 tasks complete in these diagnostic runs.

These are observed simulator results. They do not establish real-device throughput or universal policy quality. They are strong enough to ask a useful local question: what happened in this run, and which controlled change reduces the loss?

The evidence is preserved in the snapshot's loss diagnosis report and five corresponding traces. The companion independently reconstructs task durations from events, checks completion counts, and reconciles phase totals with the report. This is an important first step: before explaining a result, make sure you can reproduce its measurement.

Start with the whole shape

The timeline shows load, compute, and drain intervals on CPU and GPU. Numbers inside compute intervals identify tasks. Read each device row horizontally. Blank space means no displayed task phase occupies that device in the trace. It is not automatically proof that useful work could have run there.

A slower finish is visible in the final CPU interval

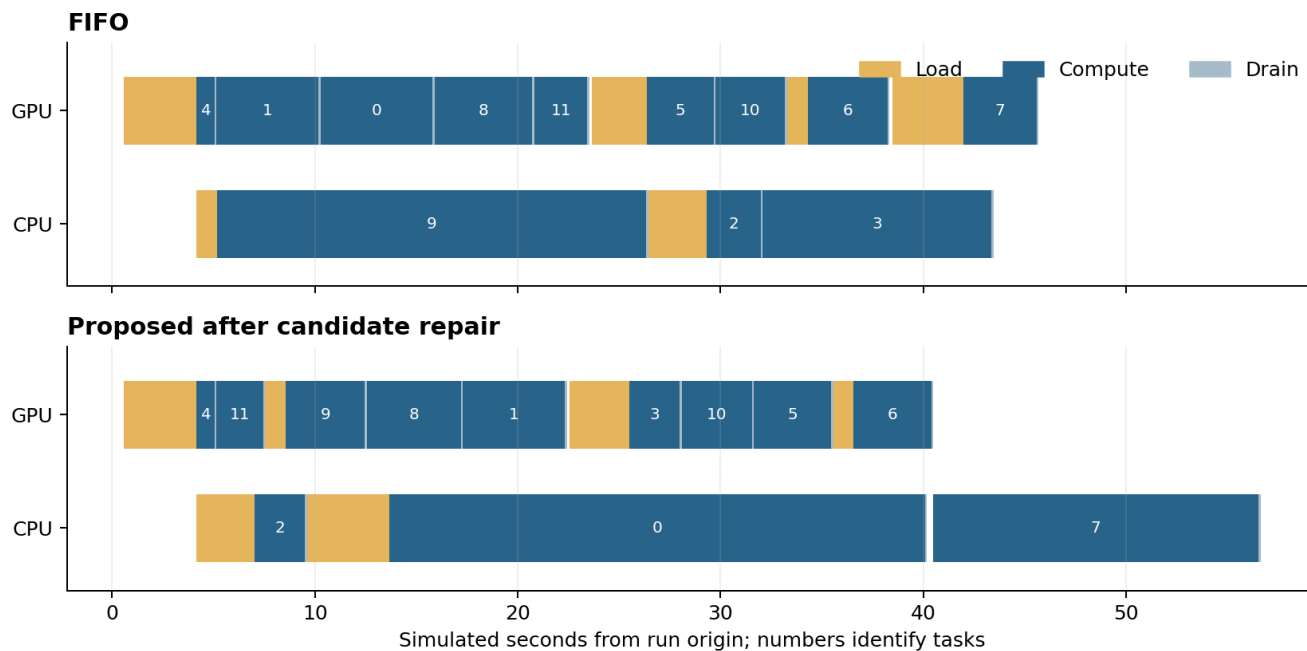


Figure 1. Actual simulator trace intervals for FIFO and the repaired proposed policy on seed 7. The final proposed task runs on the CPU while the GPU row is empty. The chart locates a question; feasibility and score records are needed to explain the choice.

The latency ECDF adds a second perspective. The proposed policy improves some earlier task latencies while making the upper tail worse. A lower middle latency can coexist with a later batch finish. The forced-final-GPU diagnostic is included as a targeted intervention, not as another deployable policy.

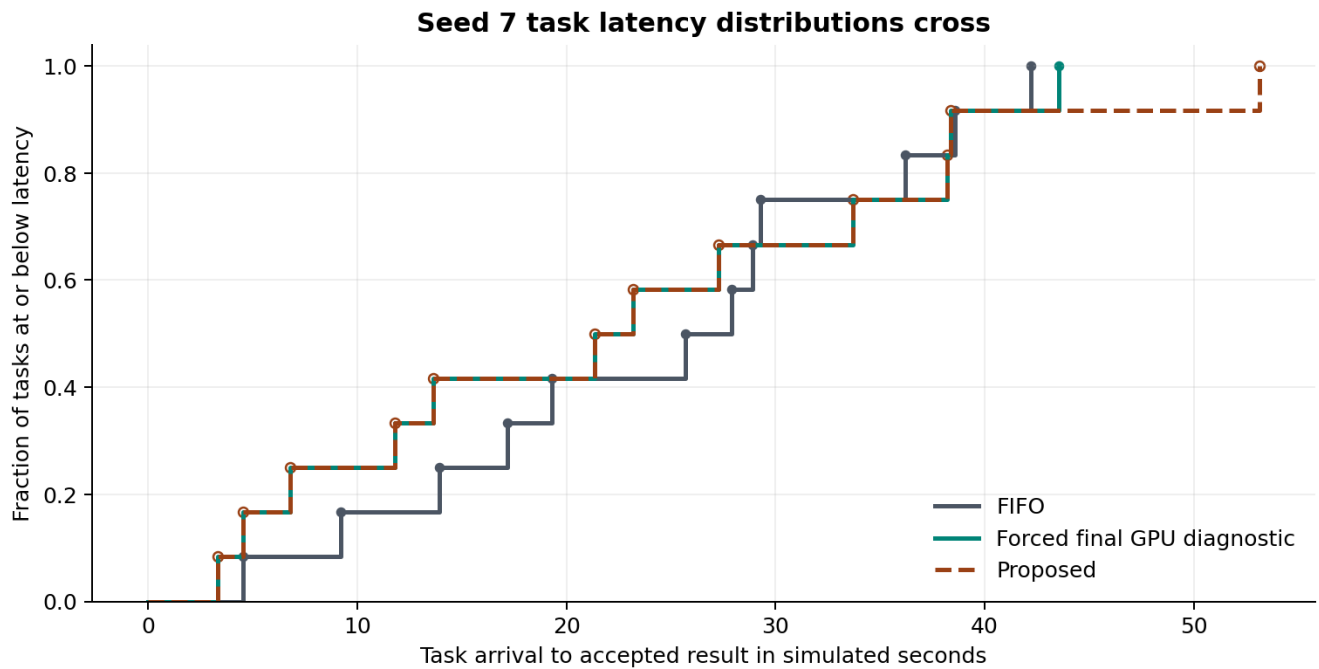


Figure 2. Twelve arrival-to-accepted-result task latencies per trace. Curves are descriptive, and these tasks share one workload. They are not 12 independent randomized policy experiments. With this sample size, the report's nearest-rank p95 is its largest task latency.

Account for resource time

For a device with one execution slot, total observed device time can be partitioned into compute time, reserved noncompute time, and unreserved time under this simulator's accounting. The categories sum to the observation duration. Reserved noncompute time includes phases such as loading and draining. Unreserved time is the remaining interval, not a direct measure of wasted opportunity.

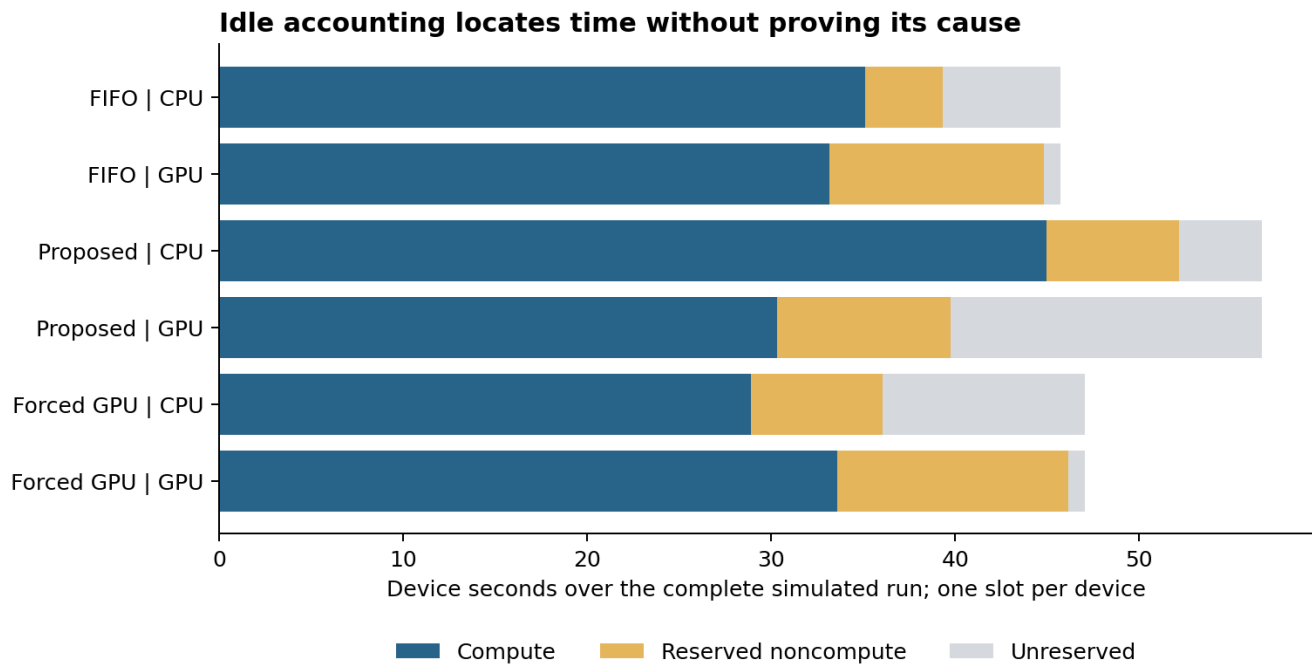


Figure 3. The companion verifies the accounting identity for CPU and GPU in each plotted run. The proposed GPU has substantial unreserved time, while the final CPU interval extends the run. Absolute time and the denominator matter: a longer run can have lower utilization even when it performs similar useful work.

This chart prevents an easy mistake. Higher CPU utilization is not necessarily better. A policy can keep a slow CPU busy for longer instead of using a faster GPU and finish later. Utilization is a diagnostic metric. The user-facing objective is useful completion under the required constraints.

Find the decision that matters

Under the proposed policy, the final task, `random7`, becomes ready at simulated time 40.495955 seconds. It needs the vision model. That model is warm on the CPU; other models occupy the GPU. The policy starts the CPU and completes at 56.642415 seconds.

There were two separate questions. First, did the planner even consider a cold GPU replacement? Second, once it considered that action, did its scoring rule prefer it?

The original candidate helper stopped looking for replacement alternatives when any execution mode already fit. A feasible warm CPU could therefore hide a faster cold GPU option. The repair changed candidate coverage and added a known-cost regression fixture. That is an implementation defect with a concrete test.

However, the complete seed-7 schedule did not improve after the repair. The unchanged 56.642415 second result is useful negative evidence. Fixing one real bug does not guarantee fixing the observed performance loss. Keep the before and after results instead of assuming the repair worked because its unit test passes.

Inspect the score in its own units

After the repair, the decision trace contains these alternatives at the final decision:

Candidate at time 40.495955	Shaped score
Start the warm CPU	0.712417
Wait with no immediate action	0.624792
Evict replicas to prepare the GPU path	0.597209

These scores are not seconds, probabilities, or measured throughput. They combine the planner's modeled progress and penalties. The score rewards partial progress inside a finite horizon and charges setup or eviction effects. Under the configured weights, the earlier-finishing GPU continuation does not receive enough advantage to overcome its penalties.

This is objective alignment in concrete form. A surrogate is a quantity used in place of the true goal because it is convenient to compute or optimize. A good surrogate should rank important choices similarly to the goal. Here, a higher shaped score selects an action that finishes the batch later in the tested state. The ranking mismatch remains after the candidate coverage fix.

Do not infer that every partial-progress reward is wrong. It can be useful when a horizon ends before any task completes. The question is whether its scale and interaction with setup penalties produce the choices you actually want across relevant states.

Use a targeted intervention

The diagnosis forces the final GPU path once, while otherwise keeping the relevant simulator state and physical ledger constraints. That run finishes at 47.038627 seconds, a reduction of 9.603788 seconds relative to the proposed default.

This intervention provides stronger mechanistic evidence than a correlation between GPU idle time and slow completion. It shows that changing that particular decision, under the simulated conditions, improves the outcome. It is not an exact oracle and not a complete general scheduling policy. It still finishes 1.343974 seconds later than FIFO because earlier decisions and readiness times differ.

That remainder matters. It prevents the story from becoming “we found the one cause.” The data supports a substantial local contribution from the final placement choice. It does not isolate all earlier differences or prove the best possible schedule.

Test the attractive alternative explanation

A plausible explanation was that fairness protection caused the loss. The diagnostic disables protection in the proposed policy. The result is 56.665930 seconds, essentially the same slow batch finish. This does not support describing the seed-7 loss as the necessary cost of fairness.

The narrow conclusion is that removing this mechanism did not remove this loss in this tested configuration. It does not prove fairness has zero effect on every task, every schedule, or every workload. Changes to protection can alter earlier dispatch decisions and tradeoffs. A sound conclusion is precise about what was varied and what outcome remained.

Widen the comparison without overselling it

The follow-up uses previously unexamined seeds 59, 83, and 101, with the same protected-turn rule and boost inputs available across policies. The baseline ranking rules still differ by design. Matching service obligations and available information makes this a better policy comparison; it does not make every dispatch decision identical.

Matched service scenario	FIFO mean seconds	Residency mean seconds	Proposed mean seconds
Randomized	47.352	40.343	47.084
Priority stress	54.166	54.166	54.166
Contention	40.904	40.904	34.755
Lookahead	14.638	14.638	12.821

The proposed policy is close to FIFO on the randomized mean and slower than the simple residency baseline. It improves the means in the contention and lookahead scenarios. There are only three seeds per scenario. Show every pair and keep the conclusion descriptive.

Only three held-out seeds per scenario; show every pair

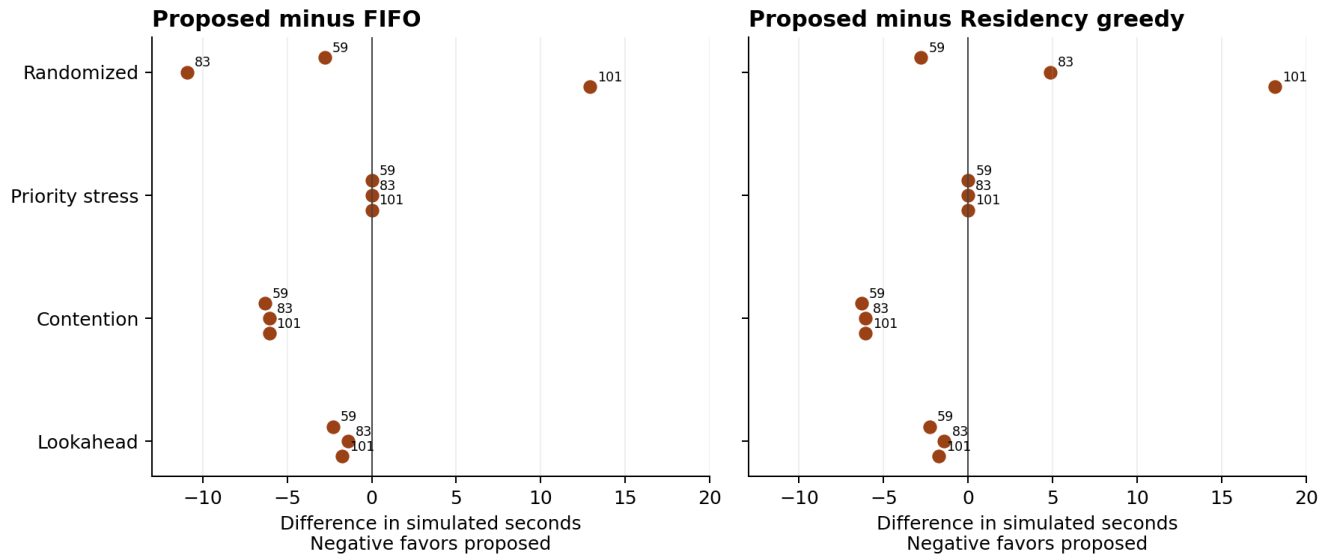


Figure 4. Every held-out seed is shown. Negative differences favor the proposed policy. Different scenario families answer different questions, so the book does not combine them into an unsupported universal win rate.

A single-parameter follow-up increases `delay_weight` from 0.25 to 1.0. The randomized proposed mean drops from 47.084 to 43.412 seconds but remains above residency's 40.343 seconds. Contention changes slightly from 34.755 to 34.780; the other displayed scenario means stay the same. This makes objective design a worthwhile next experiment. It does not validate a newly tuned default, especially after the same seeds have now informed the investigation.

End with the next engineering test

The evidence supports keeping the candidate-coverage regression test, improving instrumentation for score components, and comparing completion-oriented objective profiles against stronger simple baselines. Use fresh arrival and model-popularity seeds, fixed resource limits, matched service obligations, and a stated primary objective. Report tail behavior, unresolved work, reloads, and planner cost alongside completion.

Only after that should a shadow evaluation on real runtime traces test the simulator's assumptions. The useful result of this investigation is not an unconditional policy recommendation. It is a narrowed problem: one repaired coverage defect, one demonstrated local score mismatch, a partly explained remaining aggregate loss, and a specific next experiment.

7 Find out what a change causes

Observation and intervention

Observational data records what happened under the process that produced it. An experiment changes something deliberately and compares outcomes under a design. Both are useful. They answer different questions.

Suppose GPU tasks finish faster than CPU tasks. Perhaps the GPU is faster for those tasks. Perhaps the scheduler sends easy tasks to the GPU and difficult ones to the CPU. Perhaps GPU tasks are more likely to arrive when the queue is short. A raw comparison mixes these explanations.

A causal question asks what would happen if the same relevant population received one intervention rather than another. You cannot usually observe both outcomes for the exact same real-world event. A design tries to make the observed comparison stand in for that missing counterfactual. Hernán and Robins' open textbook develops this framework and the assumptions needed to use observational comparisons causally. [Causal Inference What If](#)

Draw the process before adjusting for it

Use a simple causal sketch in words. Task difficulty influences device choice and completion time. Queue state influences device choice and completion time. Policy influences device choice, queue state, and completion time. Those relationships tell you why comparing device groups may be confounded.

A confounder is a pre-intervention common cause that can distort the treatment-outcome comparison. A mediator is part of the pathway through which an intervention works. If a policy changes cache hits and cache hits change latency, cache hits may be a mediator. Adjusting away that change can remove part of the effect you wanted to measure.

Not every available variable should be added to a regression. Conditioning on consequences of the policy or selection process can introduce bias. For a beginner, a clear controlled comparison is often safer than a complicated observational adjustment whose assumptions are hard to defend.

A practical controlled experiment

Write the protocol before running the comparison. Name the treatment, baseline, eligible workloads, assignment unit, primary outcome, guardrails, run budget, stopping rule, and exclusion criteria. Record the code and configuration versions. Decide how failed runs count.

Randomize assignment or run order when the environment can drift. If all baseline runs occur in the morning and all new-policy runs occur after a driver change or under different machine load, the policy comparison is confounded with time. Alternating or randomizing order within a block reduces that risk.

Block on major known sources of variation. For example, compare both policies on the same workload and machine class, then repeat across seeds or workload batches. A block is a group within which the alternatives are made comparable. NIST's experimental-design guidance distinguishes comparative designs, screening designs, and response-surface designs because those questions call for different experiments. [NIST design selection](#)

A seed is not a complete fairness guarantee

Simulation often uses common random inputs so alternatives face the same arrivals and task characteristics. This can make paired comparisons more precise. However, if each policy consumes a single random-number stream in a different order, later draws may no longer refer to equivalent events. “Same seed” then does not guarantee the same realized disturbances.

Separate workload generation from policy execution when possible. Save the generated workload itself. For execution noise, key random draws to stable entities such as task, attempt, phase, and scenario when that matches the model. Document when the alternative legitimately changes which events occur.

The book's companion reads saved results and traces; it does not assume a seed proves identical all-purpose stochastic conditions. The snapshot's configuration and the simulator's explicit limitations remain part of the interpretation.

Interference changes the unit of assignment

Two tasks sharing one scheduler can affect each other's outcomes. Assigning policy A to some tasks and policy B to others in the same queue may not create two independent systems. One group's long tasks or model loads can delay the other. This is interference.

For a scheduler, consider randomizing whole isolated workload runs, clusters, or time windows rather than individual tasks, depending on the real architecture. Time-window experiments need washout or carryover analysis if queues, caches, or resident models persist across windows. Isolation improves interpretability but may reduce realism. State that tradeoff.

Agents can also interfere. Two agents may share a rate limit, modify the same repository, read one another's outputs, or compete for a tool. Evaluation environments must distinguish intended collaboration from accidental leakage between trials.

Change one thing when diagnosing

An ablation removes or changes one component to learn what it contributes. The scheduler's protection-off diagnostic asks a narrower question than a full policy replacement. Its delay-weight change is another ablation. These are useful because they avoid bundling a new objective, a wider search, a different predictor, and a bigger memory budget into one opaque improvement.

One-factor-at-a-time tests can miss interactions. Increasing search depth might help only after the score is improved. Changing the score might help only when the candidate generator

includes the relevant option. After a local diagnosis, a small factorial design can test combinations deliberately.

For two binary factors, run all four combinations: old and new objective crossed with narrow and wide search. Compare the effect of the objective at each search setting. If the effect changes substantially, there is an interaction worth understanding. Do not average it away and claim a universal component effect.

Stop for a reason you chose in advance

Watching results and stopping the moment a desired threshold is crossed changes the statistical procedure. A fixed-horizon analysis should use its planned sample size or stopping condition. A sequential design can permit repeated looks, but it must account for them.

Operational guardrails are different. If an experiment causes unacceptable failures or resource use, stop it to protect the system. Record why it stopped and include the failure in the outcome. Safety-driven stopping does not justify reporting only the earlier favorable performance window.

There is no universal minimum number of runs. Plan around the smallest worthwhile effect, expected variation, dependence, and cost of the wrong decision. Use a pilot to estimate variation, then design the confirming test. If resources only permit a small sample, give a descriptive result and a bounded decision rather than an artificially confident claim.

Transfer from simulation to deployment

Simulation lets you manipulate states that are difficult or costly to reproduce. It is excellent for finding counterexamples and testing an accounting or scheduling mechanism. Its external validity depends on what it models: setup latency, contention, memory behavior, device heterogeneity, failure patterns, and scheduling overhead.

A staged path is reasonable: unit fixtures for correctness; synthetic workloads for mechanism coverage; replay or shadow analysis on observed workloads; isolated real-device benchmarks; then a limited deployment with guardrails. At each stage, ask what new uncertainty is being reduced. Passing one stage does not silently grant the evidence of the next.

Exercise: write a four-run factorial design for two changes you are considering. State the assignment unit and one plausible source of interference. Then explain which result would make you keep only one of the changes.

8 Analyze a system over time

Keep order when order matters

A table of durations loses the sequence that produced them. For a queue, sequence is part of the system: arrivals create backlog, backlog affects waiting, model residency changes future setup costs, and failures can trigger retries. Preserve event time and analyze the sequence alongside the distribution.

Start with a timeline of arrivals, completions, queue length, active work, and resource usage. Use a common time axis. A burst of arrivals followed by a growing queue tells a different story from a stable arrival rate followed by collapsing completion throughput.

Distinguish rates from counts. Ten completions in one second and ten in one minute are different workloads. When resampling events into windows, state the window width and boundary convention. A five-minute average can hide a ten-second overload that violates a latency requirement.

Queue length connects demand and delay

A queue accumulates when arrivals exceed departures over an interval. Even when average capacity exceeds average demand, bursts and variable service times can create waits. High utilization leaves less spare capacity to absorb variation, so optimizing utilization alone can damage responsiveness.

Little's law relates long-run average number in a stable system, throughput, and average time in that same system: L equals λ times W . If a stable service completes 2 jobs per second and jobs spend an average of 5 seconds in the chosen boundary, the average number inside is 10. The boundary must match: queue-only counts go with queue-only waiting time; in-system counts include service. Finite windows, changing backlog, and inconsistent populations can break a naive application. [Columbia notes on Little's law](#)

Use the relationship as a consistency check and a way to think, not as a promise that doubling hardware will halve latency. Bottlenecks, parallelism limits, and workload composition can change when capacity changes.

Measure a queue by area

A time-weighted average queue length is the area under the queue-length curve divided by elapsed time. If the queue has length 2 for 9 seconds and length 20 for 1 second, the average is 3.8. Averaging just two recorded states gives 11, which is wrong because the states lasted different durations.

For an event-based trace, reconstruct queue length after each arrival or dispatch and integrate until the next change. Make the event ordering convention explicit when several events share a timestamp. For a sampled gauge, account for the sampling interval and missing samples.

This idea generalizes to memory residency and resource utilization. Event counts do not equal time exposure. A state that appears in one log line may persist much longer than a state that produces a hundred lines.

Separate stable patterns from change

Time-series behavior can include a long-term trend, recurring daily or weekly structure, abrupt shifts, and irregular variation. Compare like periods before declaring an anomaly. Monday morning traffic may differ from Saturday night even when nothing is broken.

For a performance predictor, plot residuals over time: observed duration minus predicted duration, or a clearly defined ratio. Break them down by model, device, size range, and warm or cold state. A persistent positive residual in one cohort suggests underprediction there. A global average near zero can hide offsetting cohort errors.

Do not calculate prediction errors using predictions regenerated after the observation. Preserve the prediction made at the time. Otherwise the model can appear calibrated because it has already learned from the outcome you are testing.

Drift and anomalies are hypotheses

Drift means that a relevant distribution or relationship changes. Inputs may become longer, a model may load more slowly, a new task family may arrive, or the relationship between features and duration may shift. An anomaly is an observation or interval that is unusual relative to a chosen reference. The reference is part of the claim.

A detector that fires frequently can create more work than it saves. Evaluate its false alarms, missed important events, detection delay, and operational response. An alert should connect to an action or investigation. “Some number changed” is rarely enough.

Control charts are one established way to monitor a process against a reference under stated assumptions. Their limits are not interchangeable with specification limits or user requirements. A stable process can consistently fail a requirement, and a useful process can show a genuine change without becoming unacceptable. [NIST process monitoring](#)

Validate on the future direction

When forecasting or predicting time-dependent outcomes, a random train-test split can allow future information to influence a model evaluated on the past. Use splits that respect time. A rolling-origin evaluation repeatedly trains using data available before a cutoff and tests on a later period. The forecast horizon and retraining schedule should resemble the intended use. [Forecasting Principles and Practice on time-series cross-validation](#)

Temporal splitting does not automatically eliminate leakage. A feature computed from a full dataset, a label added later, or duplicated entities across periods may still reveal future information. Audit when every feature becomes available.

Keep unfinished work visible

Suppose one scheduler completes 90 short tasks and leaves 10 long tasks unresolved at cutoff. Another completes all 100. Comparing mean latency only among completed tasks can reward the first policy for leaving work behind.

Useful summaries include completion fraction by a fixed deadline, unresolved count and age, failure count, and a curve of the fraction completed over elapsed time. If you use survival-analysis methods for right-censored observations, their assumptions about censoring need attention. A policy-dependent timeout or selective abandonment can be informative censoring rather than a harmless observation cutoff.

For a beginner, a clear joint report of completion and latency is often safer than a sophisticated estimator used without understanding the missing outcomes. Do not rank an unfinished run as “faster” merely because its last observed completion occurred early. The snapshot distinguishes a completed-batch metric from the last accepted completion for this reason.

A small operational view

A useful first dashboard can be small: request or task volume, completion or error rate, latency distribution, and saturation or backlog, each with a meaningful time window. Google SRE's monitoring guidance emphasizes latency, traffic, errors, and saturation, while distinguishing user-visible symptoms from internal causes. [Google SRE monitoring](#)

Use the dashboard to notice and localize a change. Use a saved trace and controlled analysis to explain it. Dashboards are optimized for repeated observation; investigations need richer context and a record of competing explanations.

Exercise: construct a queue that has the same average arrival rate in two runs but different burst patterns. Predict which should have larger waits, then simulate or calculate the sequence. Keep the service rule fixed. Explain why the average arrival rate alone was insufficient.

9 Evaluate agents and their judges

An agent has an outcome and a path

For an agent, a final answer can look correct while the process was unsafe, wasteful, or incomplete. A coding agent might claim success without passing tests. A research agent might produce fluent prose with unsupported citations. An agent that operates tools might reach the right final state after taking an unauthorized intermediate action.

Define outcome checks and process checks separately. Outcome checks ask whether the requested result exists and satisfies its requirements. Process checks ask whether the agent respected constraints, used tools correctly, preserved data, and stayed within budgets. A trace helps explain a failure, but a long trace is not evidence of a good outcome.

Anthropic's engineering guidance distinguishes tasks, repeated trials, grading logic, transcripts, and final environment outcomes. That vocabulary is useful because it prevents one agent response from being mistaken for the whole evaluation. It also emphasizes that different grader types have different strengths. [Demystifying evals for AI agents](#)

For a Chrema-style system that observes agents and evaluates other agents, this suggests a concrete starting schema: task, attempt, agent version, environment version, trace reference, final-state evidence, individual checks, judge version, and review status. This is a proposed design for the stated use case, not a description of Chrema's current implementation.

Write a rubric that can fail

“Good answer” is too vague for a reliable label. Break the task into observable criteria. For a research answer, criteria might include whether each central factual claim has a supporting source, whether cited pages were actually retrieved, whether the response answers the question, and whether uncertain facts are labeled appropriately.

A rubric should include positive, negative, and borderline examples. Specify what counts as failure. If a missing critical source is unacceptable, do not allow strong writing style to average it away. Some requirements should be gates rather than weighted scores.

Use deterministic checks where they are appropriate: schema validity, exact state, unit-test results, file existence, constraints, or known invariants. Use model-assisted or human review for aspects that require interpretation. Keep the component results so you can tell whether an aggregate changed because of factuality, usefulness, formatting, or a different mix of tasks.

Build an evaluation set around intended use

Collect ordinary cases, difficult cases, known regressions, and important rare failures. These sets serve different purposes. A production-weighted set estimates expected behavior under a stated workload distribution. An adversarial set probes whether a failure is possible. A regression suite protects previously fixed behaviors. Do not pool them into one average without explaining the weights.

Keep task families and difficulty indicators. Where possible, compare variants on the same tasks and planned number of trials. Preserve a development set for prompt and rubric iteration and a held-out set for confirmation. If evaluation tasks are repeatedly exposed to the agent or used to tune its prompts, they no longer provide a clean estimate of performance on unseen tasks.

Also version the environment. Tool availability, repository contents, permission settings, network behavior, and service responses can change results. A prompt alone is not a reproducible agent test case.

Repeated trials reveal reliability

A single success shows that an agent can solve a task once under the tested conditions. It does not show that it will do so reliably. Run repeated trials when stochastic behavior matters, keeping the task and environment controlled.

Report trial-level success and task-level summaries. “At least one success in several attempts” measures a different capability from “every attempt succeeds.” If independent attempts each have success probability p , the chance of at least one success in k attempts is $1 - (1 - p)^k$. Real retries may be dependent, so do not use that formula blindly.

A system that succeeds after five attempts may be useful when a verifier can cheaply select the correct result. It may be unacceptable when the first wrong action causes irreversible damage. Reliability must be evaluated in the actual interaction pattern, including budgets and verification costs.

A judge is another measurement system

A model judge can reduce review cost, but its output is an imperfect measurement. It may prefer an answer's position, length, or stylistic resemblance to its own outputs. The MT-Bench and Chatbot Arena study documented position, verbosity, and self-enhancement biases in the judges it examined. Its results do not establish that every modern judge is accurate on your domain. [Zheng and colleagues on LLM judges](#)

Validate the judge on a relevant, independently labeled sample. Blind reviewers to the agent variant when possible. Randomize answer order in pairwise comparisons and test whether reversing order changes the result. Keep the judge prompt, model, decoding settings, and rubric version. Inspect disagreements rather than only the agreement rate.

Human labels also need a process. Use clear instructions, multiple reviewers for an overlap sample, and a way to resolve consequential disagreements. A reference label is an adjudicated standard for the test, not infallible truth. Record ambiguity rather than forcing every case into a confident binary label.

Read a confusion matrix

The companion contains 1,080 synthetic trial records across 120 invented tasks, three illustrative agent variants, and three trials per variant. The fields named `human_pass` and `judge_pass` are generated labels. No real humans or deployed agents produced this dataset. It exists to make the following arithmetic reproducible.

Using the generated reference label as the standard, the synthetic judge has 762 true positives, 182 true negatives, 85 false positives, and 51 false negatives. Its overall agreement is 944 divided by 1,080, about 87.4 percent.

That single percentage hides an important asymmetry. Among the 267 reference failures, the judge accepts 85, about 31.8 percent. Among the 847 judge-approved cases, 85, about 10.0 percent, are reference failures. These are different denominators and answer different questions. The first is a false-positive rate; the second is the fraction of approved outputs that are wrong under the reference.

If a judge is a release gate, false approvals may be especially costly. If it is a triage tool that sends uncertain cases to a reviewer, coverage and review cost also matter. Choose the operating threshold for the consequence, then validate it on data not used to choose the threshold.

Calibration is a separate question

A judge may emit a score that looks like a probability. Calibration asks whether cases assigned a given probability succeed at roughly that frequency. A score of 0.9 should not be interpreted as a 90 percent chance of passing merely because it lies between zero and one.

Group predictions into a few bins, plot mean predicted probability against observed reference success, and show how many observations occupy each bin. The calibration chart in the companion has a very sparse middle because the synthetic judge tends to be either confident or doubtful. Its middle bin contains only four trials. A connecting line does not make that region well measured; the counts are essential to interpreting the apparent shape.

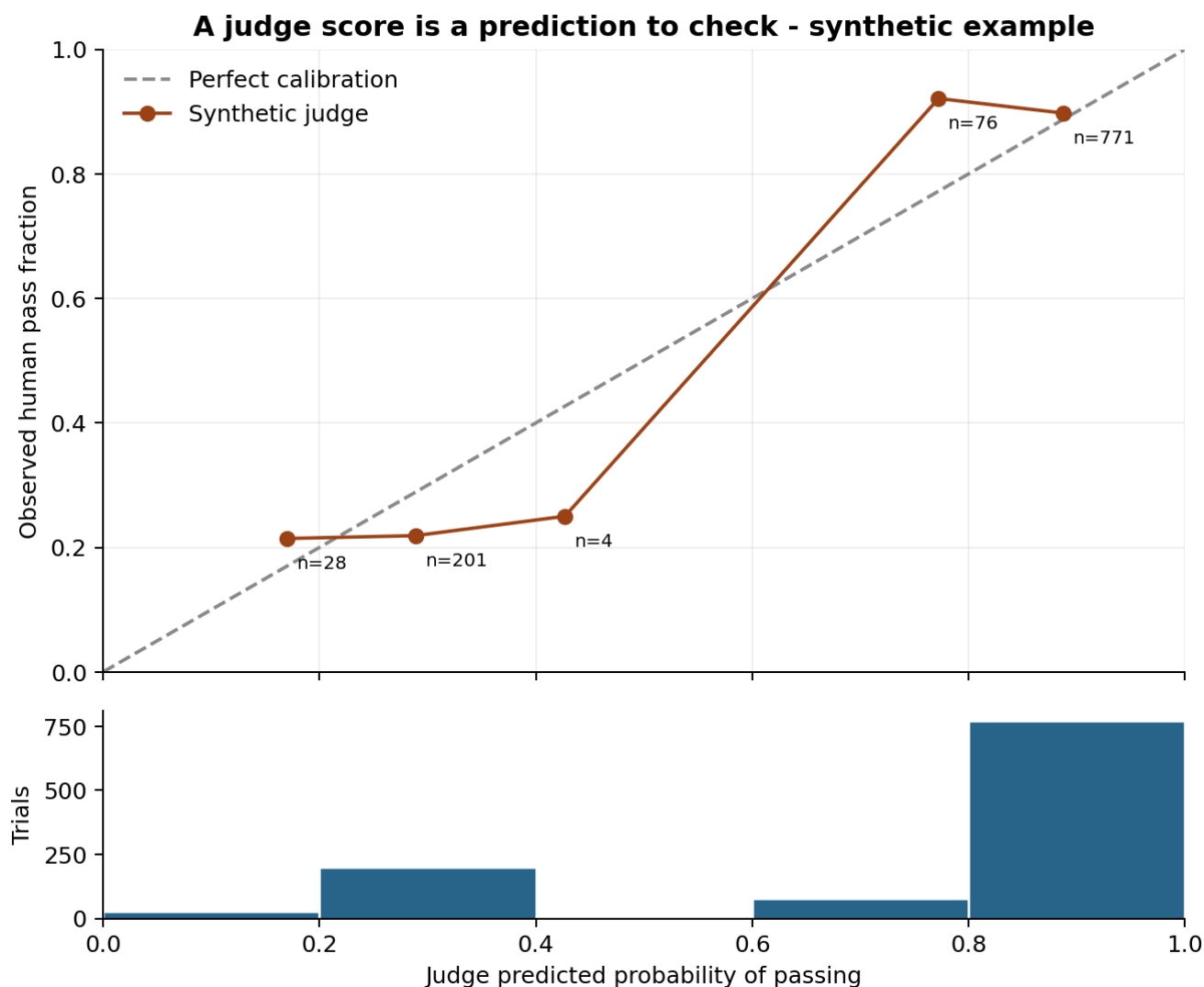


Figure 5. Generated judge probabilities compared with generated reference labels. Bin counts and the score distribution are shown. This is an illustration of calibration analysis, not evidence about Chrema or any real model judge.

Calibration and discrimination are different. A model can rank likely successes above failures while its probabilities are too extreme. A well-calibrated low-information predictor can emit the base rate for every case and be poor at ranking. Evaluate both according to the intended decision. [scikit-learn probability calibration](#)

Cost belongs in the same evaluation

Measure total cost of attempts, tools, retries, verification, and human review where relevant. “Cost per successful outcome” can be calculated as total evaluation cost divided by the number of verified successes for a defined workload. It includes failed attempts in the numerator. Calculating average cost only among successful attempts hides wasted work.

That ratio is descriptive for the observed evaluation process. It is not automatically the expected cost of retrying until success. Such a claim needs a retry policy, dependence assumptions, stopping rules, and an account of tasks that may never succeed.

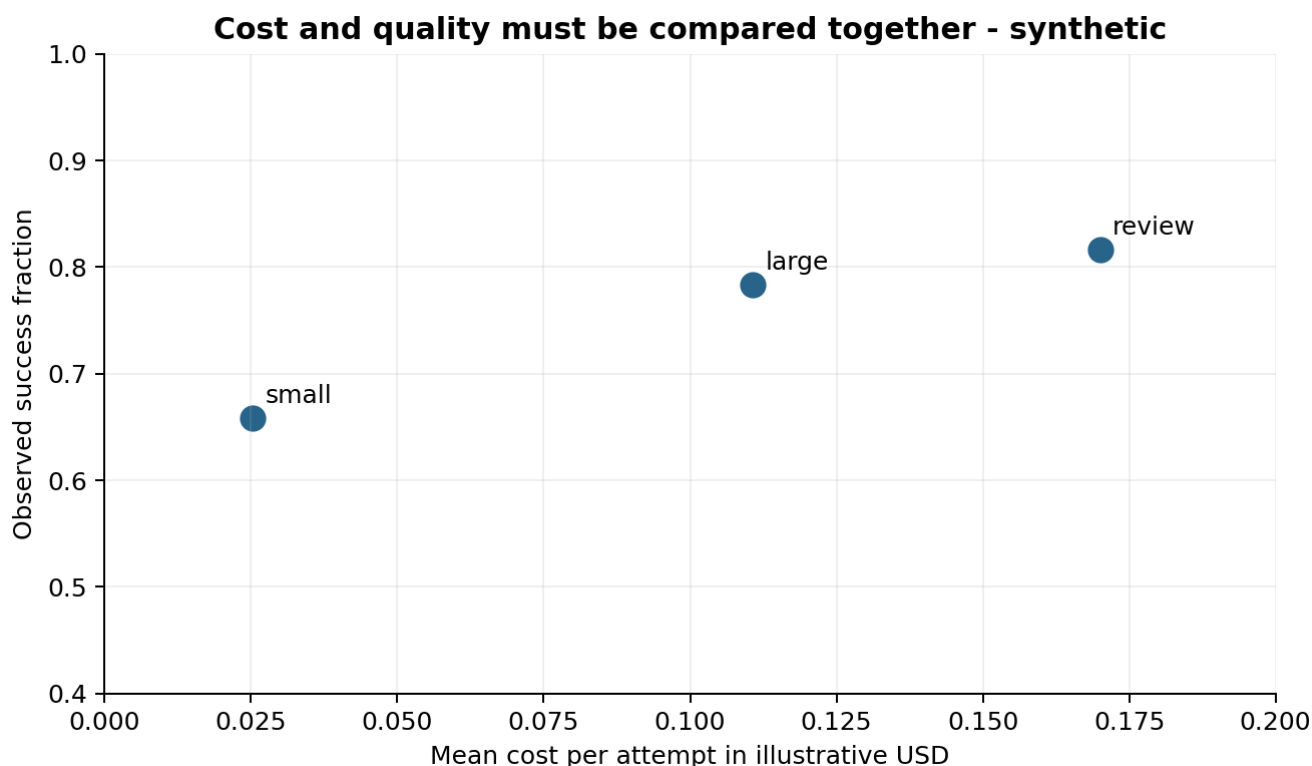


Figure 6. Synthetic mean cost and reference success for three invented variants. More expensive variants are not automatically preferable. The right choice depends on quality requirements, failure consequences, latency, and the production task mix.

A Pareto-dominated variant is worse or equal on every chosen objective and strictly worse on at least one. Removing clearly dominated choices can simplify a decision. The remaining frontier still requires priorities; a plot cannot decide how much one additional success is worth.

A practical first evaluation for Chrema

Begin with a small set of representative tasks and explicit outcome checks. Run two versions on the same tasks, with repeated trials where affordable. Review an overlap sample independently. Compare results by task family, inspect the largest disagreements, and measure the judge's false approvals on known failures.

Create a regression set from important real failures after they occur, but keep it separate from a fresh evaluation set. Track changes to the rubric as carefully as changes to the agent. If the judge changes and the score rises, you have not yet shown that the agent improved.

Exercise: design one deterministic check, one model-assisted check, and one human-review question for an agent task you care about. Then describe a case that could pass all three while still failing the user's real goal. Use that counterexample to improve the evaluation.

10 Analyze training data and model results

Start with the deployment question

A model's test score is meaningful only relative to the cases it is supposed to handle. Is the model predicting durations for familiar task families on new days, recognizing new documents, evaluating new users, or generalizing to unseen organizations? Those are different forms of generalization and require different splits.

Define the target, prediction time, available inputs, and loss associated with errors. A duration predictor for scheduling might need a useful upper bound as much as an accurate mean. A classifier used to block a dangerous action has different error costs from one used to suggest a low-stakes tag.

Keep a simple baseline. For regression, a constant or group-average predictor may be informative. For classification, a majority-class or frequency-based predictor exposes whether a high accuracy is mostly a base-rate effect. A complex model should earn its complexity on the decision that matters.

Split before learning from the data

Training data fits parameters. Validation data guides model or hyperparameter choices. A test set evaluates a frozen choice. Repeatedly selecting a model based on test performance converts the test set into another validation set.

Preprocessing can leak information too. Fit normalization, feature selection, imputation, vocabulary construction, and other learned transformations using only the training portion within each evaluation fold. Then apply the fitted transformation to the held-out portion. Pipeline tools help enforce this separation. [scikit-learn common pitfalls](#)

A random row split may be inappropriate when related rows share a document, person, device, source repository, or collection session. Keep related groups together if deployment requires generalization to new groups. Use time-respecting evaluation when the future is the target. The scikit-learn cross-validation guide distinguishes ordinary, grouped, stratified, and time-series split strategies for these reasons. [scikit-learn cross-validation](#)

Leakage can be semantic

A dataset can have no exact duplicate rows and still leak answers. A paraphrase of a training example may appear in the test set. A feature may be recorded only after the outcome. A document's filename may encode its label. A research dataset may include future information unavailable when the prediction would be made.

Audit the information path. For every feature, ask when and how it was created and whether the real model would know it at prediction time. For every split, ask which entities or source materials overlap. The research on leakage in machine-learning-based science shows that

these problems can undermine apparently strong results across domains. [Kapoor and Narayanan on leakage](#)

For training corpora, provenance and duplicate-family identifiers are useful. Store source, version, license or allowed-use information, collection time, transformation history, and known links between derived examples. Do not assume that a public URL implies unrestricted reuse.

Read learning curves carefully

A training-loss curve shows how an optimization process behaves on its training objective. It is not itself a measure of real task quality. A falling training loss with worsening validation loss may indicate overfitting under the chosen setup. A flat curve could reflect an optimization problem, insufficient model capacity, a difficult target, or an uninformative metric.

Plot against examples or tokens processed and against compute or elapsed time when comparing efficiency. Equal numbers of steps may represent different amounts of data or work. Record batch size, learning-rate schedule, data mixture, checkpoint, and evaluation procedure.

A data-scaling learning curve varies the amount of training data while holding the comparison as fair as possible. It helps ask whether more data appears useful. It does not guarantee that an extrapolated trend will continue or that more of a biased source improves the intended population.

Choose metrics that expose the error

For a duration predictor, mean absolute error expresses typical error in seconds. Squared-error metrics emphasize large misses. Relative errors emphasize scale but become unstable near zero. Plot observed against predicted values and residuals against input size, task family, and time. A single average can hide systematic underprediction for long or cold tasks.

For classification, a confusion matrix exposes false positives and false negatives. Precision asks how many predicted positives are truly positive under the reference; recall asks how many reference positives were found. Accuracy can be uninformative when one class dominates. Precision-recall and ROC views answer threshold-dependent questions and should be read with class prevalence and deployment costs in mind. [scikit-learn model-evaluation metrics](#)

Do not compare scores calculated on different test populations as if they were head-to-head. Preserve the per-example predictions so you can make paired comparisons and inspect disagreement cases.

Check probability and interval claims

If a model emits probabilities, evaluate calibration as in the agent chapter. If it emits an upper duration bound intended to cover 90 percent of outcomes, measure the fraction of observed outcomes below the bound. Also inspect bound width and coverage by relevant cohort. An extremely wide bound can have excellent coverage while being operationally useless.

Coverage in a small sample has uncertainty. The scheduler's handful of predictor observations per run cannot establish a stable guarantee. Training and evaluating the bound on the same observations can also make coverage optimistic. Use time-ordered or held-out predictions appropriate to deployment.

A confidence interval for an average is different from a prediction interval for a future individual outcome. Scheduling a single long task often requires understanding the latter. Confusing them can yield a very narrow but inappropriate resource or time promise.

Use error analysis to choose the next data

Read a sample of failures. Categorize them by a useful mechanism rather than only severity: missing input evidence, label ambiguity, unfamiliar domain, truncation, retrieval failure, tool failure, or a predictable model limitation. Validate the categories on multiple cases before treating them as facts.

Then choose an intervention tied to the mechanism. More random data may help a broad coverage problem. Better labels may help ambiguous supervision. A new feature may help an unobserved state. A product constraint or fallback may be more effective than another training run.

Keep an evaluation report with intended use, test populations, important slices, limitations, and operational behavior. Model Cards provide a structured precedent for this kind of reporting; the goal is to make performance and scope visible to people deciding whether to use the model. [Model Cards for Model Reporting](#)

Exercise: identify one feature in a current or hypothetical training dataset that might only be available after the outcome. Explain how it could improve a test score while making deployment worse. Then design a split and a data audit that would reveal the problem.

11 Collect and interpret research data

A collection is not automatically a sample

A research tool can gather thousands of records quickly. That does not mean the records represent the population you care about. Search rankings, accessible sources, language, geography, publication incentives, and duplicate syndication can shape what is collected.

For a Nearwork-style research workflow, begin with the question and the unit. Is one row a source document, a claim, a person, an organization, a location, or an observation at a place and time? One document can contain many claims, and several documents can repeat the same underlying report. Counting every extracted sentence as independent evidence creates false confidence.

This chapter proposes a research-data workflow for the stated need. It does not assume Nearwork currently has a particular database, extraction model, map, or user interface.

Separate sources from claims

Keep a source table with a stable identifier, URL or artifact reference, publisher or author when relevant, publication and retrieval dates, content hash or version, access conditions, and collection method. Keep a claim table with the extracted statement, source identifier, location in the source, extraction version, review state, and scope.

A claim should retain context. “The system improved by 20 percent” is incomplete without the baseline, metric, population, conditions, and uncertainty. Store those fields when the research question depends on them. If the source does not provide them, mark the gap rather than supplying a plausible interpretation.

Track whether a field was directly observed, extracted by a model, inferred by a rule, or entered by a reviewer. A downstream analyst should not have to guess which facts have been checked.

Provenance describes how an artifact came to exist. The W3C PROV model distinguishes entities, activities, and agents involved in producing or transforming information. A lightweight application can use those ideas without implementing the entire standard: identify the input, the transformation, the actor or software version, and the output. [W3C PROV primer](#)

Deduplicate at the level of evidence

Exact hashes find identical bytes. Normalized text can find copies with formatting changes. Near-duplicate detection can find paraphrases or syndicated versions. Entity resolution tries to determine whether different names refer to the same real-world entity. These are related but different tasks.

Keep duplicate groups rather than blindly deleting everything that resembles something else. Several copies can be useful for availability or provenance, while still counting as one

underlying evidence source for a claim. A correction may look nearly identical to the original but change the one sentence that matters.

Review false merges and false splits. Merging two distinct organizations can corrupt every later statistic about them. Treat ambiguous entity matches as uncertainty, not as a resolved identifier merely because a similarity score is high.

Missingness is a property of the process

The companion includes 100 synthetic research records. Some difficult records remain unverified, so their verification time is missing. The missingness was generated intentionally to demonstrate a bias: the completed cases make the collection process look easier than it really is.

Count missing values by source, task type, collection stage, and review status. Ask what mechanism makes a value absent. Missing completely at random means the absence is unrelated to the data of interest. Missing at random is a technical assumption that missingness can depend on observed information but, after conditioning on that information, does not depend on the missing values themselves. Missing not at random allows dependence on unobserved values even after conditioning on observed information. The labels are assumptions about a process, not properties you can reliably infer from a null-count table alone. [Rubin on inference with missing data](#)

In the generated example, 84 records have observed times and 16 are unresolved. The observed mean is 5.5 minutes. Because this is a constructed example, we also know the complete-data mean implied by its generating rule: 7.5 minutes. The easy records dominate the observed subset. In real research you usually do not know the missing values, which is precisely why documenting the collection process matters.

For a practical first pass, report which records are missing and why, compare the observed groups, and perform a sensitivity analysis. If all unresolved records took much longer than observed ones, would the conclusion change? If it would, the missing records are decision-relevant and deserve follow-up.

Do not confuse a field that was not collected with a negative answer. “No evidence found” is different from “evidence that the statement is false.” This distinction is particularly important when a research agent searches only a limited set of sources.

Model extraction needs evaluation too

If an agent extracts facts from documents, evaluate the extraction stage separately from the final analysis. Sample source passages and compare extracted fields to a reference review. Measure errors that matter: omitted claims, unsupported additions, wrong units, wrong dates, mixed entities, and lost qualifications.

A model can return valid JSON containing incorrect content. Schema validation catches structural failure; source-grounded review catches semantic failure. Keep both checks. Confidence scores from the extractor need calibration before they can support automatic acceptance.

Stratify the review sample when failures are likely to differ across source types or languages. If you oversample difficult cases to find problems, do not report the resulting raw error rate as the overall production rate without accounting for the sampling design.

Spatial data adds another measurement layer

If a research question genuinely involves location, record what a coordinate represents: an exact observation, an address geocode, an approximate city center, or a region. Store the coordinate reference system and precision. A point displayed on a map can look exact even when the source is vague.

Latitude and longitude are angular coordinates, not distances in meters. Distance and area calculations require a method appropriate to the coordinate system, geography, and desired accuracy. GeoPandas distinguishes assigning a coordinate reference system from transforming coordinates to another system; confusing those operations can place data incorrectly. [GeoPandas projections](#)

A map of raw counts often reflects where more people or records exist. If the question concerns rates, use a relevant denominator such as population, exposure, or collection effort. Show uncertainty for small counts and avoid precise-looking rates based on tiny denominators.

Spatially nearby observations may be correlated. A random split that puts neighboring or repeated sites in both training and test sets can overstate generalization to new areas. Consider geographic holdouts when that matches the intended use. Also treat precise personal locations as sensitive information: collect and expose only what the research purpose requires.

Write a dataset description before reuse

A reusable dataset needs a short description of why it was collected, what it contains, how it was collected and transformed, what is missing, who or what is underrepresented, and what uses are inappropriate. Datasheets for Datasets offers a systematic model for documenting those questions. [Gebu and colleagues on datasheets](#)

The description should travel with exported data. A CSV without its population and collection history can be easy to misuse. A provenance field is not just paperwork; it is what lets an analyst distinguish ten independent studies from ten articles describing one study.

Exercise: design source and claim tables for a question you might research. Include one field that preserves uncertainty, one that identifies duplicate evidence, and one that records review status. Then write a query that counts independent source groups rather than raw extracted claims.

12 Turn findings into an engineering decision

Decide what the evidence permits

An investigation should end with an action, a deliberate choice to wait, or a specific missing piece of evidence. “More research is needed” is rarely enough. Name the uncertainty that could change the decision and the cheapest credible way to reduce it.

For the scheduler, the evidence permits keeping a repaired candidate-coverage test and investigating the score. It does not justify calling the proposed policy generally superior. A stronger simple baseline wins the held-out randomized mean, the seed count is small, and real-device behavior has not been measured.

A useful decision can still be made under uncertainty. Keep the simpler default while testing a promising mechanism. Ship a narrow improvement only where its preconditions and guardrails are clear. Choose a fallback for failure cases. Uncertainty changes the scope and reversibility of the decision rather than forcing all work to stop.

Distinguish constraints from preferences

A hard constraint defines an unacceptable outcome: exceeding a memory limit, violating a required permission, corrupting a result, or leaving critical work unbounded. A preference trades one desirable property against another: lower latency, lower cost, fewer reloads, or simpler implementation.

Do not blend a hard constraint into a weighted average where enough speed can compensate for violating it. Evaluate feasibility first, then compare tradeoffs among feasible choices. For soft objectives, state the weights or present the frontier without pretending there is one universal winner.

The scheduler's shaped score illustrates why units and scale matter. A setup penalty and a progress reward may be mathematically combinable but poorly aligned with the user-facing objective. Sensitivity analysis should show whether modest changes to weights alter the choice. If a recommendation flips under plausible weights, that is a decision-relevant fact.

Include the cost of complexity

A policy's total cost includes computation, observability, testing, debugging, maintenance, and operational risk. A small benchmark win can disappear when planning overhead, failed attempts, or human review are included. Conversely, a modest speed improvement on a heavily used critical path may be worth substantial engineering effort.

Use the costs relevant to the decision, in their own units where possible. Report simulated time, real planning time, monetary cost, and developer burden separately before combining them. If you convert them into one utility score, make the conversion assumptions visible.

Prefer reversible experiments when the evidence is early. A shadow mode or feature flag can reduce the cost of learning. Define rollback conditions before rollout, and preserve enough version and trace information to diagnose a regression.

Write a decision memo that can be challenged

A compact memo can have five parts: decision, evidence, interpretation, limitations, and next test. Put the strongest result first, including an inconvenient one. Link the data and reproducible calculation. State what would change the recommendation.

Here is a worked example based on the simulator investigation:

“Keep the current simple policy as the general default while testing revised completion-oriented scoring. On seed 7, repairing candidate coverage did not change the proposed policy's 56.642 second batch finish, compared with FIFO's 45.695 seconds. Forcing one final GPU continuation reduced the proposed run to 47.039 seconds, demonstrating a large local contribution from that placement decision under the simulator. It did not remove the full gap. On three fresh seeds per scenario with matched service controls, the proposed randomized mean was 47.084 seconds, versus residency's 40.343. The next test will compare fixed objective profiles and stronger simple baselines on fresh workloads, while tracking completion, tail waits, unfinished work, and planning cost. Real-device validation remains necessary before deployment claims.”

This memo is useful because another engineer can disagree with the decision while checking the same facts. It does not bury the caveats in a footnote or call a local intervention a general policy.

Keep the analysis alive only while it has a purpose

Save the data snapshot, code, report, and decision. Add important counterexamples to regression tests. Track the follow-up experiment with a clear stopping condition. Do not turn every completed investigation into a permanent dashboard or an endless stream of alerts.

A mature analytical practice reduces repeated uncertainty. It gives future you a way to know what was tried, why a choice was made, and what would justify revisiting it. The value is better decisions, not the number of plots produced.

13 Practice with the companion

Run a complete analysis first

The companion uses Python with NumPy, pandas, and matplotlib. The verified run used Python 3.12.14, NumPy 2.3.5, pandas 2.2.3, and matplotlib 3.10.8. These versions describe the environment that produced the book's outputs; they are not a requirement to upgrade an existing project blindly.

From the book directory, run:

```
python companion/analyze.py
python companion/check_outputs.py
```

The program verifies hashes in the snapshot manifest, reconstructs task and phase tables, checks arithmetic against the simulator reports, writes derived CSV files, and regenerates six figures. It also creates the labeled synthetic agent, paired-run, and research examples. It never runs or changes the scheduler itself.

If a hash check fails, stop. Do not edit the expected hash merely to make the test pass. You may have mixed versions, changed an input, or downloaded an incomplete file. Restore the intended snapshot or intentionally create and document a new analysis version.

The parser is intentionally strict for these five complete diagnostic traces. They contain one ready event and one accepted result per task, and all phases close. A production parser needs additional states for retries, cancellations, open intervals, and partial traces. A test fixture is not a universal ingestion library.

Lab 1 Reproduce a metric from events

Open the generated scheduler_tasks.csv. Each row is one task within one diagnostic run. Inspect the fields before calculating anything: run_id, task_id, arrival_s, ready_s, first_phase_s, accepted_s, latency_s, ready_wait_s, and first_device.

Load it and check its key:

```
from pathlib import Path
import pandas as pd

root = Path('.')
tasks = pd.read_csv(root / 'companion/derived/scheduler_tasks.csv')
assert not tasks.duplicated(['run_id', 'task_id']).any()
assert len(tasks) == 60

one = tasks[tasks.run_id == 'after_fix_proposed'].copy()
assert len(one) == 12
assert (one.accepted_s >= one.arrival_s).all()
print(one.accepted_s.max())
```

Expected result: approximately 56.642415 seconds. Trace timestamps are rounded to six decimal places, so use tolerances rather than exact floating-point equality when comparing them to full-precision reports.

Now sort `latency_s` and calculate the sixth value and the average of the sixth and seventh. The first reproduces the report's nearest-rank p50; the second is the conventional median for 12 values. Explain why they differ without calling either a software bug.

Next, compare `first_phase_s` minus `arrival_s` with `ready_wait_s`. They answer different questions because dependency time can occur before a task is ready. Write the endpoint definitions beside each result.

Lab 2 Preserve a policy pair

Load `holdout_runs.csv` and filter to `matched_service`. Check uniqueness on regime, scenario, seed, and policy. Pivot policy into columns so each row is one scenario-seed combination:

```
runs = pd.read_csv(root / 'companion/derived/holdout_runs.csv')
matched = runs[runs.regime == 'matched_service']
wide = matched.pivot(index=['scenario', 'seed'],
                      columns='policy', values='makespan_s')
assert len(wide) == 12
assert not wide.isna().any().any()
wide['difference_s'] = wide['proposed'] - wide['residency']
print(wide.loc['randomized'])
```

Show all three randomized differences before calculating a mean. At least one seed favors the proposed policy and others favor residency. The mean therefore hides important variation. Describe the result as three tested workloads, not a reliable estimate of all future randomized workloads.

Then compute the mean of per-seed ratios and the ratio of mean times. Explain why they need not be identical. State which weighting scheme each represents.

Lab 3 Inspect one decision

Open `scheduler_candidates.csv` and select the repaired proposed run at time approximately 40.495955 seconds. The `actions_json` field preserves the candidate's structured action list. Identify the warm-CPU candidate, the no-action candidate, and the eviction candidate.

Write down their scores and score components. Do not convert a score into seconds. Use the forced-final-GPU trace to compare the resulting completion time, while preserving the fact that this was a single diagnostic intervention.

Expected interpretation: candidate coverage was repaired, but the scoring rule still preferred the warm CPU. The forced GPU continuation improved this one run substantially without reaching FIFO's finish time. A good answer names both the supported mechanism and the unexplained remainder.

Lab 4 Audit a judge

Load `synthetic_agent_trials.csv`. Count tasks, variants, trials, and rows separately. Rebuild the confusion matrix between `human_pass` and `judge_pass`. These are generated reference and judge labels, not observations from actual human reviewers.

Calculate overall agreement, the false-positive rate among reference failures, and the false-discovery fraction among judge approvals. Expected values are about 87.4 percent, 31.8 percent, and 10.0 percent respectively. Explain which quantity matters if a false approval can trigger an unsafe action.

Break the results down by easy and hard task families. Then compare variants' observed success and total cost per verified success. Do not estimate uncertainty by treating all repeated rows as independent tasks. A task-level resampling or hierarchical analysis would be a more appropriate starting point if inference were needed.

Lab 5 Make missingness visible

Load `synthetic_research_records.csv`. Count nonmissing `verification_minutes` and missing values by `hard_to_verify` and `source`. Compute the observed mean. Expected result: 84 observed records, 16 missing records, and an observed mean of 5.5 minutes.

Read the generation rule in `analyze.py`. In this constructed population, 75 easy records would take 4 minutes and 25 hard records would take 18 minutes, for a complete-data mean of 7.5. The gap comes from which records are missing, not random arithmetic noise.

In a real research workflow, the generating rule would be unknown. Write a short note explaining what you could observe, what you would have to assume, and which additional records you would try to verify first.

Lab 6 Write the next experiment

Choose one scheduler hypothesis: a completion-oriented score, a stronger baseline, or a predictor improvement. Write a one-page protocol with a fixed primary outcome, guardrails, eligible workloads, fresh workload generation rules, pairing, run budget, failure handling, and stopping condition.

Include at least one case designed to disprove the hypothesis. For example, a stronger delay penalty could improve batch finish while harming a background-work requirement. Define how you would detect that regression. Do not choose a new default based only on the same seed 7 case that suggested the change.

Common mistakes and their repairs

If a join multiplies rows, check its key cardinality before adjusting the aggregate. If a percentile differs from the report, check the definition and population before blaming the library. If a policy looks faster after excluding timeouts, put the timeouts back into the population accounting. If a chart has a dramatic trend, inspect the denominator and collection process. If an agent judge agrees 90 percent of the time, inspect what happens in the other 10 percent and whether the label distribution makes agreement easy.

If a parameter change wins one famous failing case, add that case to development tests and evaluate a frozen choice on fresh workloads. If a simulator looks excellent, test the assumptions that would be most costly to get wrong in deployment. If the conclusion cannot be reproduced from saved inputs, repair the evidence chain before making it more persuasive.

A compact working checklist

Before collecting: name the decision, primary outcome, population, observational unit, and guardrails.

Before analyzing: preserve raw inputs, record provenance, validate keys and units, reconcile counts, and inspect missingness.

Before comparing: align controls, preserve pairs, identify independent units, include failures, and choose meaningful effect units.

Before explaining: inspect traces, consider alternative mechanisms, separate correlation from intervention, and seek counterexamples.

Before deciding: show uncertainty and limits, include cost, choose a reversible next step when evidence is early, and record what would change your mind.

A short glossary

Baseline: the alternative used as a reference.

Cohort: a group defined by a relevant shared characteristic.

Confounding: distortion of a comparison by a factor affecting both assignment and outcome.

Counterfactual: the outcome under an alternative intervention that was not observed for the same event.

ECDF: a plot of the fraction of observations at or below each value.

Effect size: the magnitude of a difference in useful units or a specified standardized scale.

Estimand: the precise quantity an analysis is trying to estimate, such as mean paired completion-time difference for a defined workload population.

Guardrail: a constraint or secondary outcome that prevents an unacceptable tradeoff.

Leakage: use of information in development or evaluation that would not be legitimately available in the intended prediction setting.

Observational unit: the kind of thing represented by one observation, such as a task, attempt, or independent run.

Provenance: the origin and transformation history of information.

Right censoring: knowing that an outcome time exceeds an observation cutoff without knowing its eventual value.

Surrogate: a proxy quantity optimized or measured in place of a harder or more expensive goal.

Where to read next

For hands-on Python data manipulation, read the author's online edition of Wes McKinney's Python for Data Analysis. Work through indexing, grouping, joining, missing data, and time-series chapters as the need arises. [Python for Data Analysis](#)

For plots and graphical choices, use Claus Wilke's Fundamentals of Data Visualization. Recreate one relevant chart with your own data rather than reading it only as a gallery. [Fundamentals of Data Visualization](#)

For statistical foundations, OpenIntro Statistics is a gentler starting point than a machine-learning text. For engineering examples and reference procedures, keep the NIST handbook nearby. [OpenIntro Statistics](#) and [NIST engineering statistics](#)

When you are ready to model data, An Introduction to Statistical Learning with Applications in Python provides an accessible progression with practical labs. Use the authors' legally available edition. [An Introduction to Statistical Learning](#)

For time-dependent problems, use Forecasting Principles and Practice. For causal questions, begin with the first, less model-heavy part of Causal Inference What If. These are deeper follow-on texts, not prerequisites for your first useful engineering investigation. [Forecasting Principles and Practice](#) and [Causal Inference What If](#)

The best next exercise is still a small real question from your own system. Preserve the evidence, make one fair comparison, and write down what the result allows you to do.

14 Reason with vectors and matrices

Start with a collection of runs

Suppose you have hundreds of scheduler traces. You can already calculate waiting time, setup time, execution time, and final completion time. You now want to answer a different question: which runs fail in similar ways, and which measurements move together?

A vector is an ordered collection of numbers. One run might become a vector containing its mean task wait, total GPU setup time, and peak queue length. A matrix stacks those vectors into rows. This representation lets you compare runs, fit a simple prediction model, and compress a large table into a few informative directions.

The order matters. The vector [wait, setup, queue] is not interchangeable with [queue, wait, setup]. Store column names and units alongside the array. Keep run identifiers outside the numerical feature matrix so you can return to the underlying traces. A row is still a measurement of a particular run under particular conditions, even after its identifier disappears from a plot.

This chapter uses small invented arrays so every calculation can be checked by hand. It does not claim that the pinned scheduler traces have the clusters or principal directions shown here. The project is to learn a representation you could apply to those traces after defining comparable features.

Read matrix operations as data operations

Let X have n rows and p columns. Its shape is n by p : n observations of p features. A feature column is a vector of length n . A row is a vector of length p . The transpose, written X^T , exchanges rows and columns.

The dot product multiplies corresponding entries and adds the products. For $x = [3, 1]$ and $w = [2, 4]$, $x \text{ dot } w = 3 \times 2 + 1 \times 4 = 10$. A weighted score is a dot product. This is useful only when the weights convert the features into a meaningful shared objective. Adding seconds to bytes with arbitrary weights can produce a number without producing a useful decision.

Matrix multiplication applies many dot products at once. If X is n by p and a weight vector w has p entries, $X @ w$ produces n scores. The inner dimensions must agree. In NumPy, $@$ is matrix multiplication; $*$ multiplies matching entries element by element. Confusing these operations can yield the right shape and the wrong analysis.

A matrix can also represent a transformation. A two by two rotation matrix changes coordinate axes without changing Euclidean lengths. A diagonal matrix can scale one axis much more than another. These transformations affect how a cloud of observations looks and how distances behave. Preprocessing is therefore part of the model, not a neutral cleanup step.

A projection explains one direction

Take $x = [3, 1]$ and the direction $u = [1, 1]$. We want the point on the line through u that is closest to x . Every point on that line has the form $a u$, for some scalar a . The squared error is the sum of the squared entries of $x - a u$.

Expanding gives $x \cdot x - 2a(x \cdot u) + a^2(u \cdot u)$. Its derivative with respect to a is $-2(x \cdot u) + 2a(u \cdot u)$. Setting the derivative to zero gives:

$$a = (x \cdot u) / (u \cdot u)$$

$$\text{projection} = a u$$

Here $a = 4 / 2 = 2$, so the projection is $[2, 2]$. The residual is $[1, -1]$. Its dot product with u is zero, which means the residual is perpendicular to the direction. The original vector has been separated into an explained component along u and an unexplained component across it.

This geometry is the foundation of least squares. A model represents the directions it can explain. Fitting finds the closest point inside that space, under a chosen error measure. Perpendicular residuals are a geometric property of the fitted model; they do not imply that the model has captured the mechanism generating the data. [MIT on projections and least squares](#)

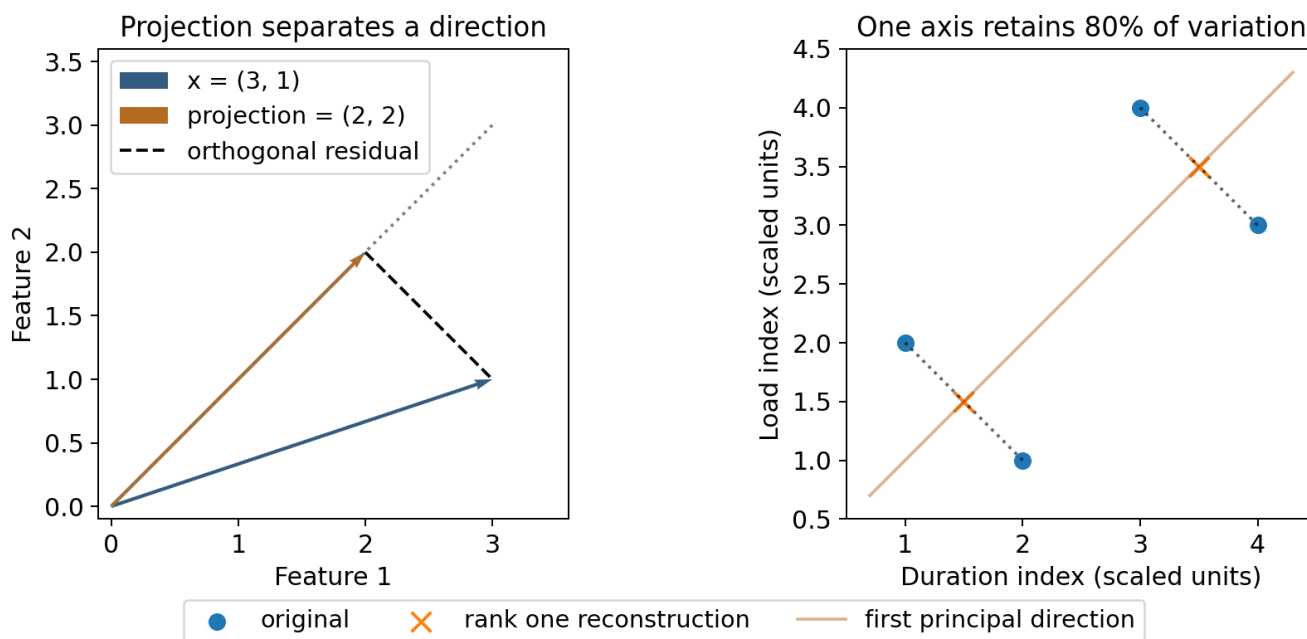


Figure 7. Left: the vector $[3, 1]$ projected onto the line through $[1, 1]$. Right: four synthetic observations compressed onto their first principal direction. Dotted segments show information lost during reconstruction. Both panels use equal axis scaling so geometric angles and distances remain meaningful.

A tiny fitted model

Imagine three synthetic measurements of duration at workload levels 0, 1, and 2: the durations are 1, 2, and 2 seconds. Fit $\text{duration} = \text{intercept} + \text{slope} \times \text{workload}$. The feature matrix contains a column of ones for the intercept and a column of workload values.

```
import numpy as np
A = np.array([[1., 0.], [1., 1.], [1., 2.]])
y = np.array([1., 2., 2.])
beta = np.linalg.lstsq(A, y, rcond=None)[0]
predicted = A @ beta
residual = y - predicted
```

The fitted intercept is $7/6$ and the slope is $1/2$. Predictions are $7/6$, $5/3$, and $13/6$. Residuals are $-1/6$, $1/3$, and $-1/6$. Their sum is zero, and their dot product with workload is zero. Those are the two perpendicularity conditions, one for each feature column.

The normal equations write this condition as $A^T A \beta = A^T y$. They help explain the geometry. For computation, use a least-squares solver rather than explicitly calculating an inverse of $A^T A$. Nearly redundant columns can make fitted coefficients unstable; forming $A^T A$ worsens conditioning. Predictions may remain similar while individual coefficients swing. Report that instability instead of assigning causal meaning to each coefficient.

This three-point fit is a calculation lesson, not a useful performance predictor. A real model needs held-out runs, residual checks, plausible feature definitions, and observations across the workload range where it will be used. Fitting an intercept and slope does not establish that changing workload alone causes the fitted change.

Distance depends on your units and purpose

Suppose each observation contains execution time in seconds and memory in MiB. Let $A = [1, 100]$, $B = [2, 100]$, and $C = [1, 110]$. Raw Euclidean distance calls B closer to A : the distances are 1 and 10. Convert time from seconds to milliseconds and they become 1,000 and 10. The nearest neighbor changes even though the physical observations did not.

Choose scales before interpreting distance. If a one-second change and a 20-MiB change are the meaningful reference differences, divide the two columns by 1 and 20. The resulting distances from A are 1 for B and 0.5 for C . The scales encode a substantive choice about similarity.

Standardization instead subtracts each training-set mean and divides by its standard deviation. This produces unitless features and can prevent a large numerical scale from dominating. It gives high weight to features with small variation, including possibly uninformative noise. Constant columns need explicit handling because their standard deviation is zero. Fit the transformation on the training data and reuse it on validation and future data.

Euclidean distance adds squared coordinate differences before taking a square root. Manhattan distance adds absolute differences, reducing the influence of one unusually large coordinate difference. Cosine similarity compares directions through the normalized dot product. It can treat two very different magnitudes as identical: $[1, 2]$ and $[10, 20]$ have the same direction. The zero vector has no defined direction. Pick the measure for the question, and record the treatment of zeros and missing features.

Mahalanobis distance also accounts for covariance: a change along a direction with large usual variation counts less than a similarly sized change along a stable direction. Its squared

form is $\mathbf{d}^T \mathbf{S}^{-1} \mathbf{d}$, where $\mathbf{d}$ is the difference and $\mathbf{S}$ is a covariance estimate. Estimating $\mathbf{S}$ reliably becomes difficult when features approach or exceed independent observations. Redundant features can make it singular. Regularization or dimensionality reduction introduces assumptions that belong in the analysis record; an inverse operation cannot create the missing information.

Covariance records how deviations align

Use four synthetic observations of two comparable, already scaled indices:

Observation	Duration index	Load index
A	1	2
B	2	1
C	3	4
D	4	3

Both column means are 2.5. Subtract the means to obtain a centered matrix $\mathbf{C}$. A sample covariance multiplies paired deviations and divides their sum by $n - 1$. The covariance matrix for this example is:

$$\mathbf{S} = \mathbf{C}^T \mathbf{C} / (n - 1)$$

$$\mathbf{S} = \begin{bmatrix} 5/3 & 1 \\ 1 & 5/3 \end{bmatrix}$$

The diagonal entries are the sample variances. The off-diagonal value 1 says that deviations tend to have the same sign. Correlation divides covariance by the product of the two standard deviations. Here the correlation is $1 / (5/3) = 0.6$. It is unitless, whereas covariance carries the product of the feature units.

Covariance describes joint variation; a shared workload driver could produce it without either measured feature causing the other. If runs come from different workload families, inspect within-family relationships before interpreting the pooled covariance. In NumPy, observations in rows require `rowvar=False` when calling `np.cov`. That orientation option matters. [NIST on covariance matrices](#), [NumPy covariance](#)

PCA finds directions with large variation

Principal component analysis asks which unit-length direction captures the most variation of the centered observations. Projecting $\mathbf{C}$ onto a unit vector $\mathbf{v}$ gives scores $\mathbf{C} \mathbf{v}$. Their sample variance is $\mathbf{v}^T \mathbf{S} \mathbf{v}$. The maximizing direction is an eigenvector of $\mathbf{S}$: applying $\mathbf{S}$ to that direction changes its length but not its direction.

For our matrix, the first direction is $[1, 1] / \text{square root of } 2$. Its variance is $8/3$. The perpendicular direction $[1, -1] / \text{square root of } 2$ has variance $2/3$. The total variance is $10/3$, so the first direction retains 80 percent and the second retains 20 percent.

Keep only the first component and reconstruct each point. A and B both become $[1.5, 1.5]$; C and D both become $[3.5, 3.5]$. The total squared reconstruction error is 2. This calculation makes the tradeoff concrete: a compact representation merges observations that differed in the discarded direction.

A singular value decomposition computes this efficiently. For a centered real matrix C , write $C = U \text{diag}(s) V^T$. The columns of V supply feature directions; U multiplied by s supplies observation scores. Squaring each singular value and dividing by $n - 1$ gives the corresponding sample variance. NumPy returns V^T as its third result, named Vt or Vh . [MIT on SVD](#), [NumPy SVD](#)

```
X = np.array([[1., 2.], [2., 1.], [3., 4.], [4., 3.]])
mean = X.mean(axis=0)
C = X - mean
U, singular, Vt = np.linalg.svd(C, full_matrices=False)
scores = C @ Vt.T
variance = singular**2 / (len(X) - 1)
reconstructed = scores[:, :1] @ Vt[:, :1] + mean
```

The sign of a principal direction is arbitrary. An implementation may return the negative direction and negative scores with exactly the same reconstruction. Equal or nearly equal principal variances can also allow unstable directions within a stable subspace. Compare reconstructions or subspaces before treating a flipped plot as a changed phenomenon.

PCA preserves variance under the preprocessing and squared-error metric you chose. It does not preserve every rare failure or every feature useful for prediction. A rare safety violation may occupy a low-variance direction. Choose the number of components using reconstruction diagnostics and the downstream task, then validate on held-out data. A two-dimensional scatter plot is a view of the data, not the full dataset.

Clusters are hypotheses about similarity

K-means chooses K centers and assigns points to reduce the sum of squared distances to their assigned centers. For the four points above, two natural centers are $[1.5, 1.5]$ and $[3.5, 3.5]$, with total within-cluster squared distance 2. The result summarizes the geometry you supplied.

You can see one assignment-and-update step without a clustering library. Start the centers at A and C. Assign each observation to the nearer center, then replace each center by the mean of its assigned observations. In this fixture, A and B share one center and C and D share the other. The updated centers are $[1.5, 1.5]$ and $[3.5, 3.5]$, and another assignment step leaves the groups unchanged.

```

centers = X[[0, 2]].copy()
squared = ((X[:, None, :] - centers[None, :, :])**2).sum(2)
labels = squared.argmin(axis=1)
centers = np.array([X[labels == k].mean(0) for k in range(2)])
within_error = ((X - centers[labels])**2).sum()
assert np.isclose(within_error, 2.0)

```

This code exposes the core operation for a case where both groups are nonempty. A general implementation must handle empty groups, convergence, initialization, and scale.

Hierarchical clustering builds a sequence of merges under a chosen rule for distance between groups. Density-based methods identify dense connected regions and can leave some observations unassigned. Neither approach removes the need to justify features and neighborhood definitions.

The optimization can depend on initialization. Its preference for compact groups around means can fit some datasets poorly, especially elongated groups, unequal densities, or isolated outliers. Other clustering methods make other assumptions. Changing the distance, feature scale, feature selection, or number of clusters may change the answer. [scikit-learn clustering guide](#)

For an engineering investigation, inspect representative traces and unusual members of each group. A cluster containing high setup time may suggest repeated model loading; it does not establish that loading caused the final delay. Compare the group's composition by workload, resource, and policy. Check whether groups recur in fresh runs and whether recognizing them changes a practical decision.

Do not evaluate a cluster solely by how separated it looks on a PCA plot. Projection can hide overlap or create an impression of separation. Check stability in the full chosen feature space and explain why the groups matter operationally.

More dimensions can make closeness less useful

Consider an explicit model: two independent points each have d independent standard normal coordinates. Their coordinate differences are independent normal variables with variance 2. Each squared difference has mean 2 and variance 8. Adding over d coordinates gives expected squared distance $2d$ and variance $8d$. The standard deviation divided by the mean is therefore square root of $(2/d)$.

Under this model, relative variation in squared distances decreases as dimensions increase. Our simulation of 5,000 pairs gives coefficients of variation about 1.00, 0.45, 0.14, and 0.044 for 2, 10, 100, and 1,000 dimensions. Distances grow while their relative differences shrink.

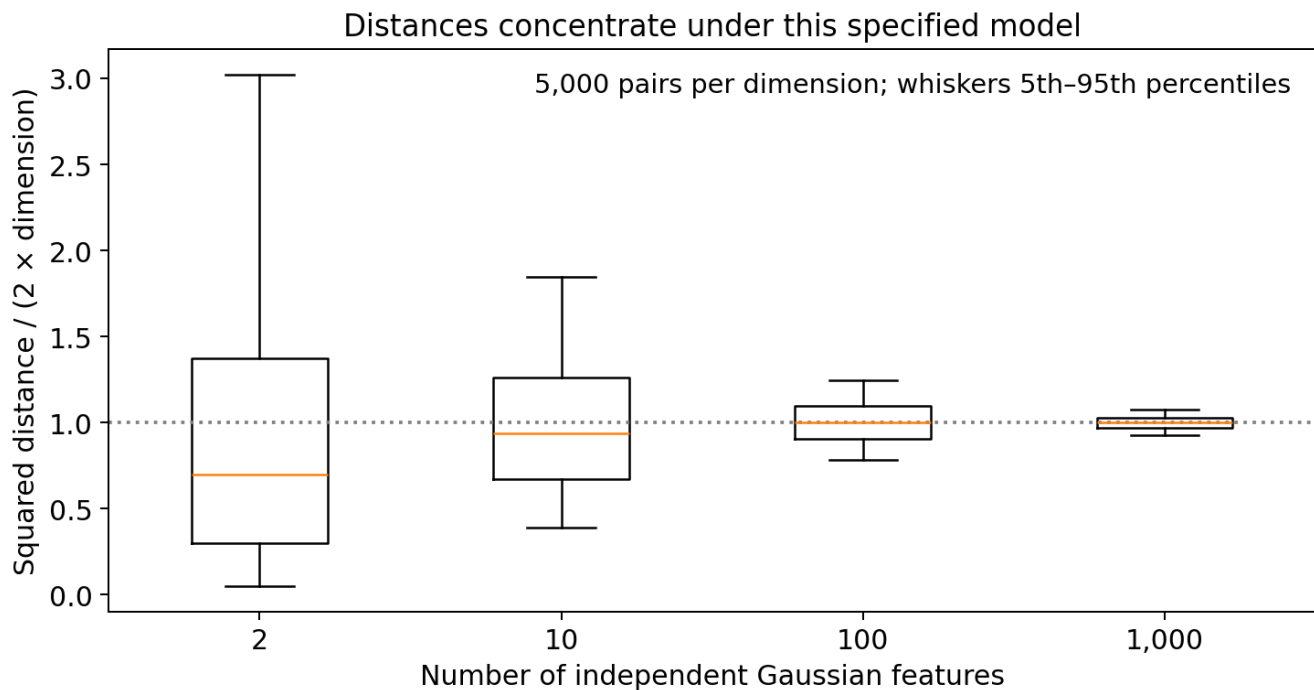


Figure 8. Squared distances divided by their theoretical mean for independent Gaussian features. Boxes show the middle half of simulated pairs and whiskers the fifth through ninety-fifth percentiles. The model and normalization are essential: this is not a universal claim about every embedding or high-dimensional dataset.

Adding irrelevant features can bury meaningful differences. More features also require more observations to estimate flexible relationships. Start with features connected to plausible mechanisms, compare against a simpler representation, and test whether the additional dimensions improve retrieval, prediction, or diagnosis on fresh cases.

Practice and a stopping rule

Run `python companion/math_examples.py` and `python companion/check_math_examples.py` from the expanded book root. The first writes the arrays, six figures, and numerical results; the second checks identities and known answers. Then alter one feature's units in the distance example and verify that explicit physical scaling restores the original comparison.

Exercise: explain why retaining 80 percent of variance does not guarantee retaining 80 percent of useful diagnostic information. A useful answer names a decision and a low-variance feature that could control it. For example, one infrequent permission violation may matter more than ordinary variations in execution time.

For your own traces, stop when the representation supports a clear next action: identifying a reproducible failure family, finding comparable runs, or producing a simpler predictive baseline. A more elaborate embedding is worthwhile when it improves that action under a fair evaluation.

15 Measure images as data

Turn a picture into a defined measurement

Suppose a camera records manufactured parts and you want to estimate the area of a bright component. The decision might be whether the component is within a size tolerance or whether a process is drifting. Before choosing an image algorithm, define the physical quantity, the acceptable uncertainty, and the acquisition conditions.

An image is an array, but its entries acquire meaning from the camera and processing history. A value might represent a sensor count, a display brightness, a categorical class, or a calibrated physical quantity. A visually bright pixel is not automatically a linear measurement of illumination. Exposure, lens distortion, compression, color processing, and saturation can affect the result.

The worked project uses an invented 64 by 64 grayscale image. A known square occupies rows 18 through 41 and columns 20 through 43, inclusive. It has 576 pixels. At an assumed spacing of 0.2 mm in both directions, its true synthetic area is $576 \times 0.2 \times 0.2 = 23.04 \text{ mm}^2$. We blur the boundary, add noise, insert one bright speck outside the object, and make one dark hole inside it. Because we generated the scene, we can distinguish the known object from what an algorithm measures.

Keep values and coordinates explicit

A grayscale array has two axes, usually row then column. A color array commonly adds a final channel axis. Image coordinates often start at the top left, with row increasing downward. A geometric Cartesian plot usually has vertical coordinates increasing upward. Always state the coordinate convention before comparing centroids, rotations, or trajectories.

Integers and floating-point values need deliberate conversion. An unsigned eight-bit image stores values from 0 to 255; subtraction can wrap if performed in that type. Convert to a suitable floating type before arithmetic. Dividing by 255 makes a unitless 0-to-1 representation; it does not perform sensor calibration or reverse a nonlinear color encoding. Image libraries also have dtype and range conventions that may differ from a general numerical array. [scikit-image image data types](#)

Preserve the original image, acquisition identifier, timestamp, exposure settings, pixel calibration, and every transformation. If you resize an image, update spacing. If you crop it, retain the offset needed to map coordinates back to the original. A missing pixel, a saturated pixel, and a genuine zero measurement are different states.

A histogram cannot locate an object

A histogram counts pixels in intensity intervals. It can reveal saturation, limited contrast, or a rough separation between dark and bright regions. It cannot tell you where the bright pixels are. Randomly shuffling all pixels leaves the histogram unchanged while destroying the shape.

A threshold classifies pixels using a rule such as intensity greater than or equal to 0.5. It is a simple segmentation method: it divides the image into foreground and background. In this project, thresholds 0.3, 0.5, and 0.7 produce different boundaries because blurring creates intermediate intensities.

Threshold	Largest component pixels	Area in mm ²	IoU with known square
0.30	617	24.68	0.930
0.50	574	22.96	0.997
0.70	485	19.40	0.842

These are measured outputs of the generated fixture. The largest component excludes the isolated bright speck. Threshold 0.5 is close to the known area in this one image, but selecting it after inspecting the answer is tuning. It needs validation on separate images with different object sizes, brightness, noise, and acquisition conditions before becoming an operational rule.

A threshold sweep gives a sensitivity analysis, not a confidence interval. A confidence or coverage interpretation requires a defensible probability model or sampling design. The spread among three settings does not acquire such a meaning merely because it looks like an error bar.

Convolution makes local context explicit

A filter combines neighboring pixel values. A three by three mean filter replaces a pixel with the average of its neighborhood. If the neighborhood contains one value of 8 and eight zeros, the result is 8/9. A kernel is the small array of weights used in that calculation.

For two-dimensional convolution, the output at row r and column c is a sum of kernel values multiplied by image values at reversed offsets:

$$\text{output}[r, c] = \text{sum over } i, j \text{ of } \text{kernel}[i, j] \times \text{image}[r-i, c-j]$$

Cross-correlation uses the opposite offset convention without reversing the kernel. The two operations agree for a symmetric kernel, such as the smoothing kernel used here. They differ for asymmetric kernels, including some directional filters. Library names are not a substitute for checking the convention.

Our blur kernel is the outer product of $[1, 2, 1]$ with itself, divided by 16. The weights sum to one, so a constant image remains constant away from complications introduced by boundaries. The center gets the greatest weight; corners get the least. This suppresses some fine-scale variation while also spreading an abrupt boundary across nearby pixels.

At the edge of the image, the neighborhood extends beyond available data. Zero padding, reflection, repetition of the nearest value, and periodic wrapping give different results. The

companion's two-dimensional convolution uses NumPy reflect padding, which excludes the edge value from the reflected extension. This corresponds to SciPy's mirror mode; SciPy's reflect mode repeats the edge value. Its frequency illustration deliberately uses periodic wrapping so its sinusoidal calculation is exact. Boundary behavior belongs in the method definition. [SciPy convolution documentation](#)

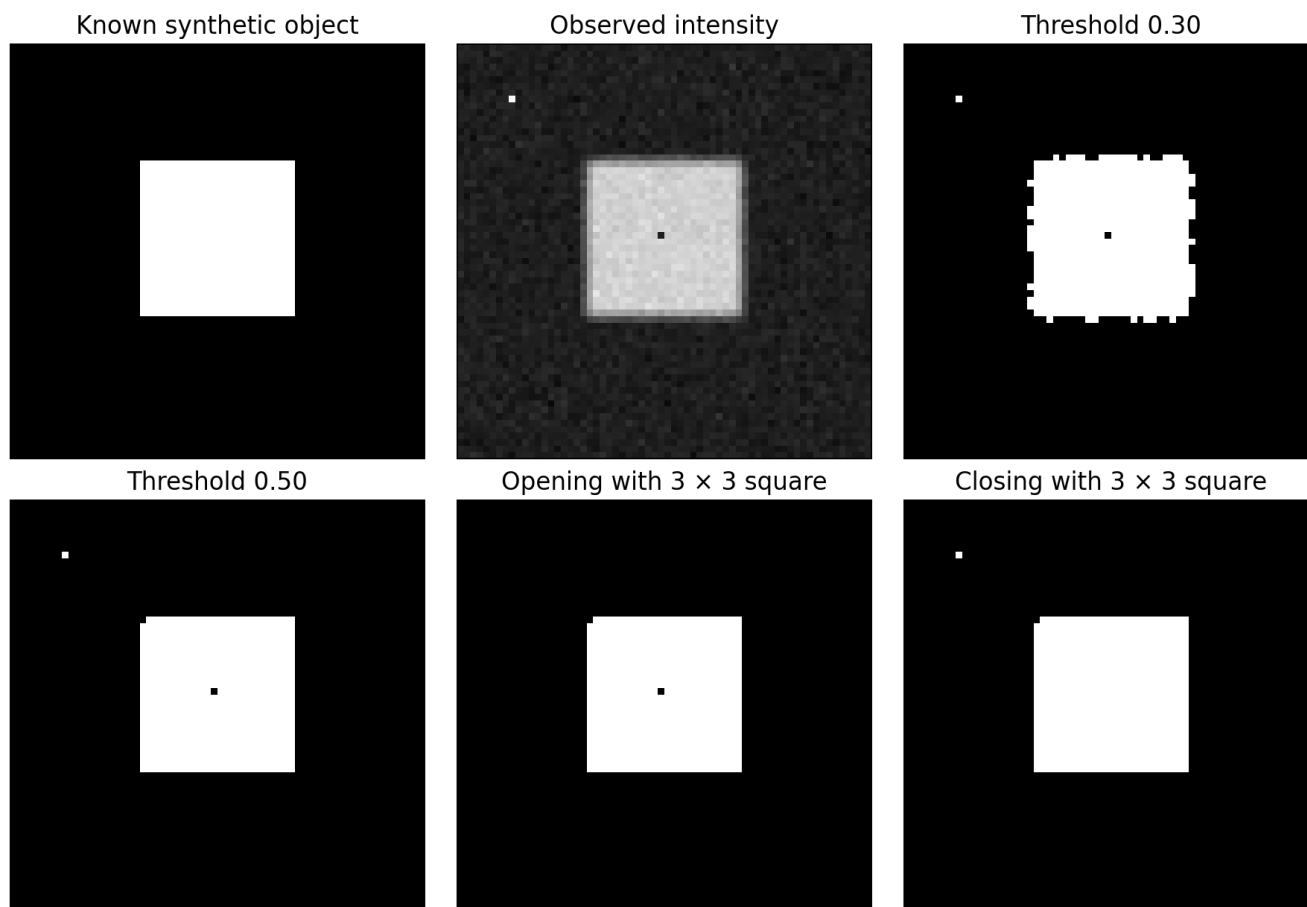


Figure 9. The known square, simulated observed image, two thresholds, and two morphological transformations. Each panel uses the same pixel grid. Changes that make a mask appear cleaner can also change the measured area or topology.

Frequency describes how quickly a pattern changes

A spatial frequency says how often a pattern repeats per unit of distance. Four cycles across a 64-pixel row is $4/64 = 0.0625$ cycles per pixel. Twenty cycles is 0.3125 cycles per pixel. The companion adds a sinusoid of amplitude 1 at the first frequency to a sinusoid of amplitude 0.3 at the second.

The discrete Fourier transform expresses the sampled signal as a combination of frequency components. For N samples, its k th coefficient combines each sample with a sinusoid completing k cycles across the record. The FFT is an efficient algorithm for computing this transform. The frequency units depend on sample spacing: using millimeters rather than pixels changes the physical frequency labels. [NumPy Fourier transform conventions](#)

Now smooth with weights $[1/4, 1/2, 1/4]$. For a sinusoid at frequency f cycles per sample, the amplitude multiplier is $1/2 + 1/2 \cos(2\pi f)$. At $4/64$, it is about 0.96194. At $20/64$, it is about 0.30866. The output amplitudes are therefore about 0.96194 and 0.09260. The higher-frequency component is attenuated more strongly.

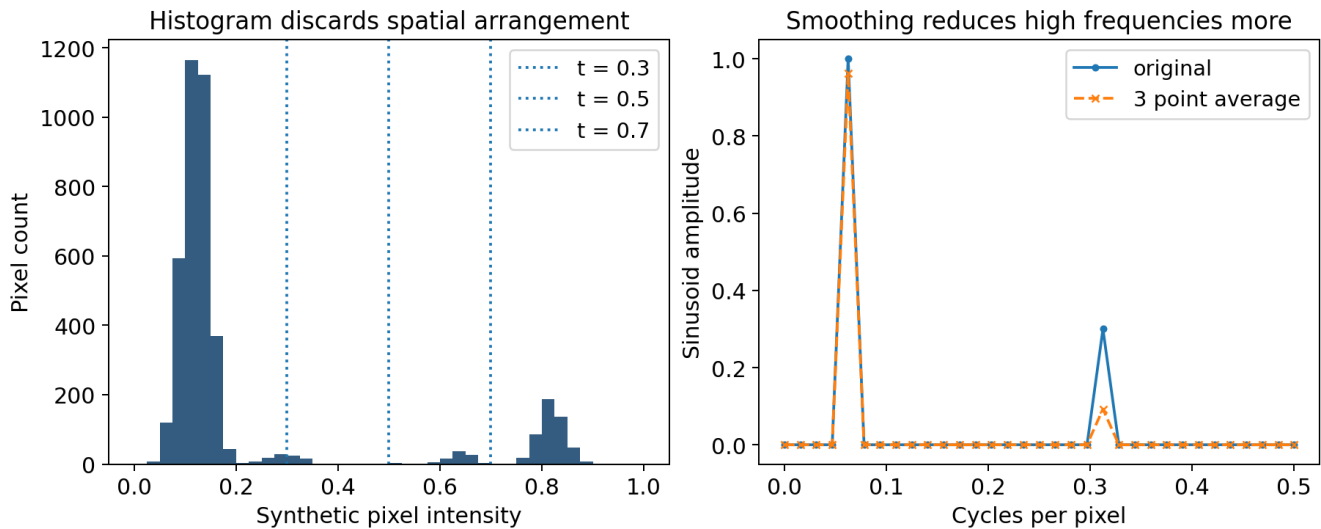


Figure 10. Left: the generated image histogram with three candidate thresholds. Right: exact-frequency sinusoids before and after periodic three-point smoothing. The frequency plot uses a single-sided amplitude convention, with special handling of the zero and Nyquist bins.

This does not mean that every high frequency is noise. Fine edges, small defects, and text contain high-frequency information. Smoothing can remove the feature you intended to measure. Likewise, an abrupt crop can introduce frequency leakage; a window reduces some leakage while changing amplitude and resolution properties.

Sampling imposes another limit. With one sample per pixel, 0.5 cycles per pixel is the Nyquist frequency. Frequencies above it can appear as lower frequencies after sampling. Resizing a finely striped pattern without appropriate filtering can produce new-looking bands. Once distinct physical patterns have produced the same samples, a later transform cannot recover which original pattern was present without additional assumptions.

Morphology acts on shape

For a binary mask, erosion keeps a foreground pixel only when the chosen neighborhood fits within the foreground. Dilation expands foreground wherever that neighborhood touches it. Their effects depend on the neighborhood, often called a footprint or structuring element.

Opening applies erosion followed by dilation. It can remove small isolated objects or narrow protrusions. Closing applies dilation followed by erosion. It can fill small holes or bridge narrow gaps. A three by three square footprint affects diagonal and axial neighbors; a cross-shaped footprint has different geometry. [scikit-image morphology](#)

In the fixture, the threshold-0.5 mask contains 575 foreground pixels, including the bright speck. Opening leaves 574. Closing leaves 576. These counts are not interchangeable with the largest-component area in the table. Closing can retain a speck while filling a hole; a plausible

total count can conceal two opposite errors. Look at the mask and its components, not just its total area.

The companion uses a background of false outside the image for binary operations. Objects touching the border deserve special attention because that convention can shrink or remove boundary structures. A morphology recipe should specify footprint, iteration count, operation order, padding, and whether border-touching objects are retained.

From connected pixels to geometry

Connected-component labeling assigns an identifier to each contiguous foreground region. Four-connectivity joins pixels through shared edges. Eight-connectivity also joins them through corners. Two diagonal pixels form two objects under the first definition and one under the second. Choose a definition that matches the physical measurement and keep it consistent.

For a component with N pixels and rectangular pixel spacing s_{row} and s_{col} , area is $N \times s_{\text{row}} \times s_{\text{col}}$. The centroid is the mean of the component's pixel coordinates. If coordinates refer to pixel centers, state that convention before converting to physical coordinates. A bounding box summarizes extreme row and column positions; its area includes background between those extremes.

For the threshold-0.5 largest component, the centroid is approximately row 29.521, column 31.521. The known square's pixel-center centroid is row 29.5, column 31.5. This small difference is an output of the specific simulated noise and missing pixels, not a universal localization accuracy.

Perimeter is more delicate than area. Counting exposed pixel edges gives a grid-dependent estimate that changes with orientation and resolution. A contour-based estimate makes different interpolation assumptions. Circularity, often written $4\pi \times \text{area} / \text{perimeter}^2$, inherits that perimeter error. An apparently precise shape score can be dominated by pixel geometry at small sizes. Measurement libraries expose spacing and shape properties, but the estimator still needs validation against objects with known dimensions. [scikit-image region measurements](#)

Carry calibration uncertainty into the result

For square pixels of side s , area $A = N s^2$. If N is treated as fixed and the standard uncertainty of s is u_s , a first-order approximation gives $u_A \approx 2 N s u_s$. For $N = 576$, $s = 0.2$ mm, and $u_s = 0.002$ mm, the calibration contribution is 0.4608 mm². A one-percent scale uncertainty becomes approximately two-percent area uncertainty.

This contribution is shared across every pixel. It does not shrink by treating 576 pixels as 576 independent calibration observations. If segmentation also makes N uncertain, its contribution and any dependence with scale must be considered. Chapter 18 develops the covariance form of this calculation. The current example isolates scale uncertainty so the arithmetic remains inspectable.

An uncertainty budget might include scale calibration, optical distortion, repeat imaging, threshold choice, and the discrepancy between a physical edge and the image-derived edge. Some terms are random under repeated acquisition; others represent uncertainty about a common correction or model. Repeating the same flawed processing on the same stored image does not measure acquisition variability.

Validate the measurement you will use

Intersection over union, or IoU, divides the pixels present in both reference and predicted masks by the pixels present in either. It measures overlap. Two masks with the same area can have poor overlap, and a very small object can have a large relative error from a few boundary pixels. Report physical area error, localization error, missed-object rate, or false-object rate when those match the decision better.

Split evaluation data by the independent acquisition unit. Patches from one image or repeated images of the same part can share lighting, texture, and defects. Putting related patches in both training and test sets can make a method appear more general than it is. Include conditions near the operational tolerance, and inspect failures by size, contrast, location, and camera session.

Exercise: use the fixture to find a case where total foreground count looks close to 576 but the mask is wrong. The closing result is a useful starting point. Then decide whether a false speck, a small internal hole, or a shifted boundary is most costly for your proposed application. That consequence should influence the measurement and evaluation, not just the visual appeal of the processed image.

16 Analyze places and spatial relationships

Start with a location decision

Suppose a research team has collected reports about service access in a city. Each report names a place, an observation date, a waiting time, and the source from which the claim came. The team can investigate only three places next week. Should it revisit an apparent cluster of long waits, visit an area with few reports, or verify an unusually severe individual report?

A map is useful here because location might explain which reports share a service catchment, which are close enough to revisit together, and where evidence is absent. A map can also mislead: duplicated reports can look like independent corroboration, a district centroid can look like an exact address, and a densely populated area can look unusually troubled simply because more people report from it.

The project in this chapter is to build an auditable spatial summary before choosing a follow-up. We will connect a table to geometry, join observations to areas, sample a raster, interpolate an unobserved value, and ask whether similar values are neighbors. Every coordinate and value in the companion is invented. These are proposed techniques for a Nearwork-style research workflow, not claims about Nearwork's present implementation or private data.

The output is a shortlist with reasons and limitations. It should preserve the evidence that supports each location, the spatial assumptions that change its ranking, and the gaps that additional research could resolve. The mathematical tools help identify those gaps; they do not decide which consequence matters most.

Give a location a meaning before calculating distance

A point is a pair of coordinates. A line is an ordered sequence of points. A polygon represents an area, possibly with holes and disconnected parts. These vector geometries describe objects. A raster divides a domain into cells and records a value for each cell. It describes a sampled field or a collection of area summaries. Neither representation is inherently more accurate; the geometry must match what was measured.

A coordinate reference system, or CRS, tells software what coordinates mean on Earth. Geographic coordinates commonly express longitude and latitude in degrees. Projected coordinates place locations on a plane, often in meters. Declaring a CRS interprets existing coordinates; transforming a CRS calculates new coordinates for the same locations. GeoPandas distinguishes these operations as `set_crs` and `to_crs`. Reassigning a meter-based CRS to degree values does not convert them. [GeoPandas projections](#)

The coordinate order is another part of the contract. GeoPandas geometries use x, y; geographic points are therefore longitude, latitude. A data source may publish latitude first. A pair of valid-looking numbers can be swapped without producing a software error. Check a

known landmark and the total extent before trusting the layer. Store the source CRS with the data rather than guessing it from a plausible map.

Consider two invented points one degree apart in longitude. On a sphere, the distance is about 111.195 km at the equator and 55.597 km at latitude 60 degrees. The difference comes from geometry, not a change in the coordinate unit. Longitude lines converge toward the poles. Treating degrees as uniform Cartesian distances would erase that fact.

For a sphere of radius R , convert angles to radians and calculate:

```
h = sin²((latitude2 - latitude1)/2)
  + cos(latitude1) cos(latitude2) sin²((longitude2 - longitude1)/2)
distance = 2 R arcsin(sqrt(h))
```

The companion uses $R = 6,371.0088$ km and clips h to $[0, 1]$ to protect against floating-point overshoot. This great-circle calculation is a teaching approximation. Production work that needs ellipsoidal accuracy should use a geodesic implementation and the correct datum. It also needs to distinguish straight-line distance from road travel time: a river crossing can dominate access even when two points are geographically close.

Use a projection suitable for the study extent and the quantity being measured. A local engineering distance problem and a country-wide area comparison may need different projections. All plane projections distort something; meter units alone do not guarantee adequate distance or area accuracy. Web display coordinates are not automatically an appropriate measurement system. Verify distortion against the required tolerance, especially across large extents or projection-zone boundaries. PROJ's geodesic routines provide a separate route for ellipsoidal distance calculations. [PROJ geodesic calculations](#)

Turn reports into a spatial join

Imagine a four-kilometer square divided into four two-by-two-kilometer zones. These are synthetic local Cartesian coordinates in kilometers; we do not attach a real-world EPSG code. Seven reports occur at $A = (0.5, 0.5)$, $B = (1.5, 0.5)$, $C = (2, 0.5)$, $D = (3.5, 0.5)$, $E = (0.5, 2.5)$, $F = (2.5, 2.5)$, and $G = (4, 1)$.

A spatial join attaches zone attributes to reports whose geometry satisfies a predicate. An attribute join instead matches identifiers such as `zone_id`. Production spatial predicates distinguish containment from intersection and boundary-inclusive relationships. A join can return several matches for one observation, particularly with overlapping polygons. Inspect the match cardinality rather than assuming that one input row remains one output row.

[GeoPandas spatial joins](#)

C lies exactly on the boundary between the two southern zones. G lies on the eastern outer boundary. A strict interior rule leaves both unmatched. A boundary-inclusive rule assigns C twice and includes G. To expose the arithmetic, the teaching code uses half-open rectangles: include the lower x and y bounds, exclude the upper bounds.

```

from gis_finance_examples import rectangle_join
points = [[.5,.5], [1.5,.5], [2.,.5], [3.5,.5],
          [.5,2.5], [2.5,2.5], [4.,1.]]
zones = [[0,0,2,2], [2,0,4,2], [0,2,2,4], [2,2,4,4]]
matches = rectangle_join(points, zones, boundary="half_open")
assert matches == [[0], [0], [1], [1], [2], [3], []]

```

The four counts are 2, 2, 1, and 1; G remains outside the defined domain. Half-open rectangles are a transparent partition for this grid, not a universal administrative-boundary rule. A real boundary case might need authoritative address assignment, an explicit tie rule, or an ambiguity flag. It should not silently disappear in an inner join.

Record the input count, distinct observation count, unmatched count, and observations with multiple matches. After aggregation, reconcile the total to the assignment rule. If a report intentionally belongs to two overlapping service catchments, its repeated membership may be legitimate; counting those memberships as two independent reports is not.

Before joining real polygons, inspect empty or invalid geometries, holes, duplicated features, boundary versions, and CRS agreement. A geometry repair can change an area, so preserve the original and document the repair. A spatial index speeds candidate lookup but does not replace the exact relationship test. For nearest joins, keep the measured distance, set a defensible search limit, and inspect ties. The nearest known facility is not necessarily an eligible, open, or reachable facility.

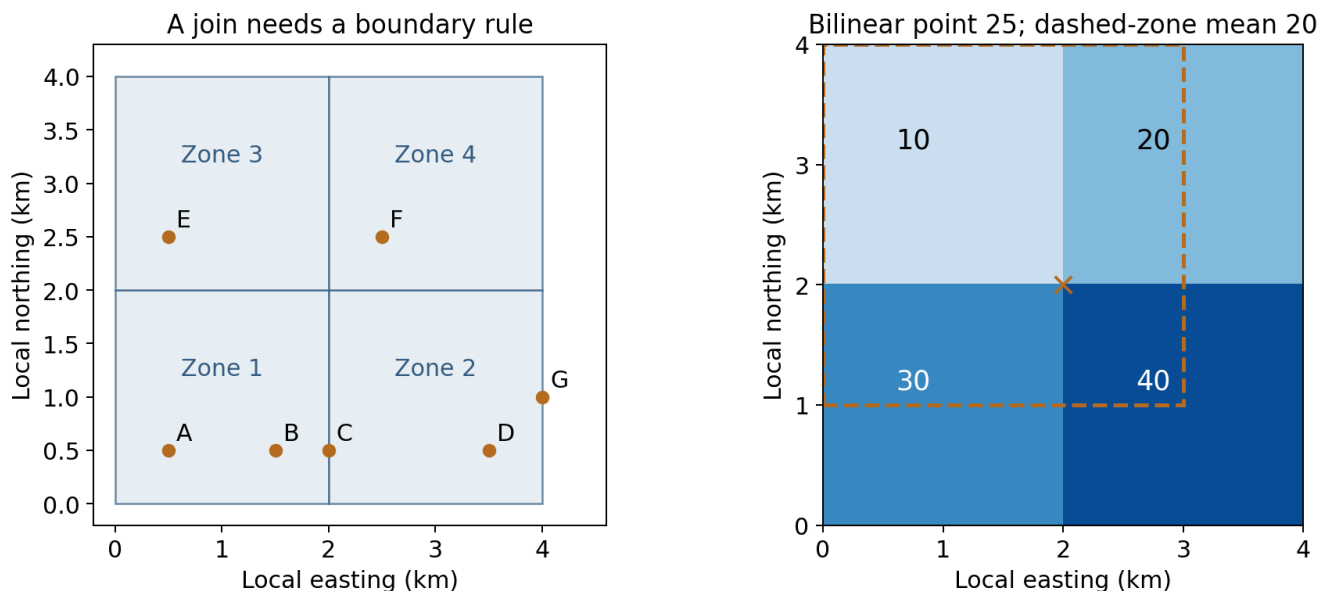


Figure 11. Left: seven invented reports and four zones; C and G expose boundary choices. Right: four raster cells with values 10, 20, 30, and 40. The orange cross has a bilinear estimate of 25, while the dashed polygon has an area-weighted mean of 20. Both panels use local kilometer coordinates and equal axis scales; they do not depict a real place.

Know whether a raster value belongs to a point or an area

Chapter 15 treated an image as an array whose pixels could be calibrated into physical coordinates. A georeferenced raster adds an explicit transformation between pixel coordinates and map coordinates. An affine transformation can translate, scale, rotate, and shear a grid.

In GDAL's convention, let c and r denote column and row coordinates measured from the upper-left pixel corner. With six coefficients g_0 through g_5 :

$$\begin{aligned}x &= g_0 + g_1 c + g_2 r \\y &= g_3 + g_4 c + g_5 r\end{aligned}$$

For a north-up raster without rotation, g_2 and g_4 are zero, g_1 is the pixel width, and g_5 is a negative pixel height. The center of array element $[r, c]$ uses $c + 0.5$ and $r + 0.5$ in this equation. Confusing corners with centers creates a half-cell displacement. [GDAL geotransform tutorial](#)

Our raster is $[[10, 20], [30, 40]]$. Each cell is two kilometers wide. The upper-left corner is (0, 4), so its transform is $[0, 2, 0, 4, 0, -2]$. The value 10 has center (1, 3); the value 40 has center (3, 1). To locate a world point in a rotated grid, solve the two-by-two linear system for c and r rather than assuming axis alignment. The companion rejects singular transforms.

At (2, 2), four cells meet. Our containing-cell convention selects row 1, column 1 and returns 40. Bilinear interpolation between centers gives each of the four values weight 1/4 and returns 25. Neither computation resolves what the raster actually measures. If cells contain land-cover category codes, averaging their numeric labels is meaningless. If cells hold a smooth field sampled at centers, bilinear interpolation can be sensible. If they contain total population, careless interpolation can invent or lose population.

An area query needs another operation. Let A_j be the area of overlap between the query polygon and cell j , and v_j the cell value. If v_j is a cell-average intensity, an area-weighted mean is:

$$\text{zone mean} = \text{sum}(A_j v_j) / \text{sum}(A_j)$$

For the rectangle from (0, 1) to (3, 4), overlap areas are 4, 2, 2, and 1 square kilometers. Its mean is $(4 \times 10 + 2 \times 20 + 2 \times 30 + 1 \times 40) / 9 = 20$. An unweighted mean of all touched cells would be 25. A cell-center inclusion rule can give yet another answer. Report which support and boundary convention you used.

NoData is not zero. If a missing cell is excluded, the denominator must also exclude its area, and the result describes only the observed part. The companion returns the observed area fraction alongside the mean. A zone with ten percent coverage should not look as certain as one with complete coverage. For extensive counts, allocating a fraction of a cell total by overlap area assumes uniform density within that cell. State that assumption and check whether it is credible.

When combining rasters, align CRS, resolution, pixel origin, bounds, time, units, and NoData masks. Resampling onto a smaller pixel size creates more array elements, not more independent measurements. Do not multiply a sample size by the number of interpolated cells.

Interpolate a missing value without inventing certainty

Suppose three sensors at (0, 0), (2, 0), and (0, 2) measure 10, 20, and 30 in a common unit. We want a value at (1, 0). Inverse-distance weighting, or IDW, gives closer observations larger weights:

```
raw_weight_i = 1 / distance_i^p
normalized_weight_i = raw_weight_i / sum(raw_weights)
prediction = sum(normalized_weight_i × observed_value_i)
```

The power p controls how quickly influence decreases with distance. Our example uses $p = 2$, all three observations, no smoothing, and a Euclidean local distance. This is one specified estimator, not a default that fits every surface. GDAL documents the same family of weighted interpolators and the role of the neighborhood and power. [GDAL gridding methods](#)

The distances are 1, 1, and square root of 5. Raw weights are 1, 1, and 1/5; normalized weights are 5/11, 5/11, and 1/11. The prediction is 180/11, or approximately 16.364. At a sensor location, the companion returns that sensor's value rather than dividing by zero. It rejects duplicate source coordinates because conflicting colocated measurements require an explicit aggregation or measurement model.

```
import numpy as np
from gis_finance_examples import idw
estimate, weights = idw([[0,0], [2,0], [0,2]],
                        [10,20,30], [1,0], power=2)
assert np.isclose(estimate, 180/11)
assert np.allclose(weights, [5/11, 5/11, 1/11])
```

A weighted average cannot exceed the input range when all weights are nonnegative. That is a useful test, but it is not an accuracy guarantee. IDW can smooth across an actual barrier, favor a dense cluster of redundant sensors, or extend a plausible-looking surface beyond the observed region. More decimal places do not repair those assumptions.

Now separate two uncertainties. If independent sensor errors each have variance 4, and weights a_i are fixed, the variance contributed by measurement error is $\sum(a_i^2 \times 4)$. Here it is 204/121, approximately 1.686, giving a standard deviation of about 1.298. In matrix form it is $a^T C a$, where C is the sensor-error covariance. Correlated errors add off-diagonal terms.

This calculation omits interpolation error: the true field at the query point need not equal the weighted combination of true sensor values. It also omits uncertain sensor locations, calibration bias, and uncertainty about the power and neighborhood. Label it measurement-only uncertainty, not a predictive confidence interval.

Leave-one-location-out validation illustrates the distinction. Hide each sensor, predict it using the other two, and compare the prediction with its measurement. The three predictions are 25, 16.667, and 13.333. Their root mean squared error is about 13.088, far larger than the measurement-only standard deviation. These are only three invented cases, but they show why a smooth interpolated map is not evidence of small predictive error.

Kriging takes another route: specify how covariance changes with separation, then choose weights using that covariance and a mean model. Under a known mean, a simple form solves $C a = c$, where C describes covariance among observed locations and c describes covariance

with the prediction location. Ordinary kriging adds an unknown constant mean and a sum-to-one constraint. The resulting uncertainty is conditional on the covariance model and its parameters. Learning those assumptions needs data; the word kriging is not a guarantee of calibration. [Esri on kriging assumptions](#)

For the follow-up decision, test a simple baseline, compare errors by distance to evidence, and hold out contiguous regions when the intended use is predicting new regions. A random split of neighboring points can test local interpolation while being advertised as geographic transfer. Structured cross-validation should reflect the deployment question. [Roberts and colleagues on structured cross-validation](#)

Express a neighborhood as a matrix

Location often makes observations dependent. Nearby reports may share weather, transport, service staff, or a common source document. Counting them as independent can make uncertainty too small. The same issue appeared in paired scheduler runs and repeated agent evaluations; here the relationship is partly spatial.

A spatial weights matrix W records which locations count as neighbors. For four locations along a chain, connect 1 to 2, 2 to 3, and 3 to 4. Start with symmetric binary connections and then divide each row by its number of neighbors:

```
W = [[0, 1, 0, 0],
      [1/2, 0, 1/2, 0],
      [0, 1/2, 0, 1/2],
      [0, 0, 1, 0]]
```

Each row sums to one, so $W x$ is the neighbor mean, often called a spatial lag. If $x = [1, 2, 3, 4]$, the lag is $[2, 2, 3, 3]$. Row standardization has made W asymmetric: location 1 gives all its weight to location 2, while location 2 gives only half its weight back. This is intentional, not a floating-point defect.

Define neighbors for the question. Shared polygon edges, shared corners, a distance threshold, the k nearest sites, and a travel-time network produce different graphs. A k -nearest rule can connect very distant places in sparse areas. A distance rule can leave islands with no neighbors. Our row-standardization function rejects islands so the analyst must decide whether to expand the neighborhood, analyze them separately, or use a justified zero-lag convention. Do not let a library default silently redefine the population.

The effect of dependence can be seen without a spatial model. If n measurements each have variance σ^2 and every distinct pair has correlation ρ , then:

```
variance of mean =  $\sigma^2 / n \times [1 + (n - 1) \rho]$ 
effective n under this model =  $n / [1 + (n - 1) \rho]$ 
```

This follows by expanding the variance of a sum into its n variance terms and $n(n-1)$ covariance terms. For $n = 9$ and $\rho = 1/4$, the effective n is 3. It is an illustrative equal-correlation model, not an estimator of effective sample size for every map. Real dependence changes with separation and with how observations were collected.

Compare Moran I with the right null

A global Moran statistic asks whether deviations from the overall mean tend to align across neighbor links. Write $z = x - \text{mean}(x)$ and $S_0 = \text{sum of all entries of } W$. Then:

$$I = (n / S_0) \times (z^T W z) / (z^T z)$$

The denominator measures overall variation. The numerator sums products of neighboring deviations, weighted by W . Matching signs contribute positively; opposite signs contribute negatively. Constant attributes make the denominator zero, so I is undefined. The companion raises an error rather than assigning zero.

For the four-location chain, $z = [-1.5, -0.5, 0.5, 1.5]$. Its squared length is 5. The weighted cross-product is 2. Because $S_0 = n = 4$, $I = 0.4$. This statistic depends on the chosen graph and transformation; it is not a universally bounded Pearson correlation. PySAL describes the statistic and its randomization reference distribution. [PySAL global Moran I](#)

The null used here is random labeling: hold the four locations, W , and the multiset of observed values fixed, and consider every assignment of those values to locations equally likely. This does not simulate new event locations or test a homogeneous point process. It also does not test whether one site's value causes its neighbor's value.

There are only $4! = 24$ assignments, so we can enumerate them. Their mean I is $-1/(4-1) = -1/3$. Two assignments have I at least 0.4, giving a right-tailed exact p-value of $2/24 = 1/12$, approximately 0.0833. A visibly ordered pattern can therefore coexist with limited evidence in a tiny dataset. The reference is slightly negative, not zero.

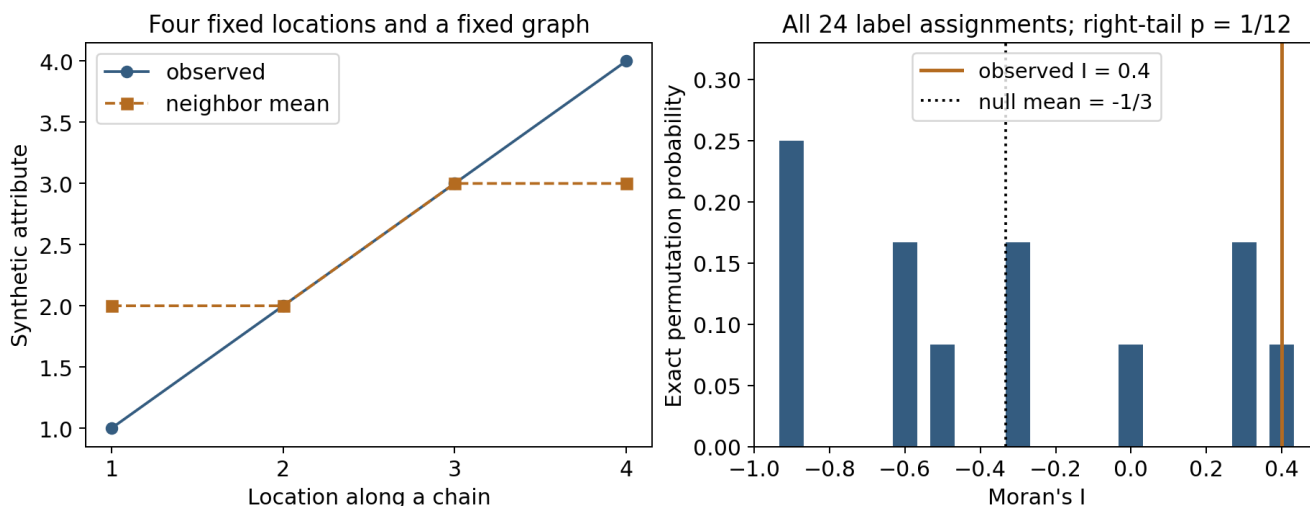


Figure 12. Left: the synthetic values and their row-standardized neighbor means. Right: the complete distribution over 24 label assignments. The orange line is the observed statistic and the dotted line is the exact null mean. The test was specified as right-tailed before examining the result.

For larger datasets, draw B random permutations and compute:

$$\text{Monte Carlo } p = [1 + \text{count}(\text{permuted } I \geq \text{observed } I)] / (B + 1)$$

The companion uses $B = 999$ and seed 1601. Its p-value is 0.069, while exhaustive enumeration gives 0.0833. That difference is Monte Carlo variability, not a different observed map. The smallest possible simulated p-value is $1/(B+1)$. Re-running until a preferred p-value appears would be another form of selection. A left-tailed or two-sided question needs a prespecified tail rule; do not choose the direction after viewing the statistic.

Random labeling requires exchangeability under the null. If district populations, measurement precision, or urban and rural baselines differ systematically, unrestricted shuffling of raw rates may be inappropriate. A positive I could reflect a broad trend, a shared covariate, or source duplication. Use the statistic to motivate a more specific model and collection plan, and validate that model's residual assumptions separately.

Keep local discovery and aggregation honest

A local statistic identifies contributions around particular locations. One convention uses $m2 = \sum(z_i^2)/n$ and $I_i = z_i \times (Wz)_i / m2$. In the chain, local values are [0.6, 0.2, 0.2, 0.6], whose average equals the global I because $S0 = n$. Other software conventions and inference schemes need to be checked explicitly.

A positive local value can mean high near high or low near low. The sign alone does not label a problematic area. Nor does quadrant membership establish significance. Local tests generally use a location-specific randomization design, and running many of them creates a multiple-testing problem. GeoDa's methodological discussion separates these questions.

[GeoDa local spatial autocorrelation](#)

For 100 valid tests of true null hypotheses, each using a 0.05 threshold, the expected number of false rejections is at most 5; it equals 5 when each test has exact 5 percent size. This does not require the tests to be independent. Bonferroni uses $0.05/100 = 0.0005$ to bound the family-wise error by 0.05. With only 999 random permutations, the minimum Monte Carlo p-value is 0.001, so that threshold cannot be reached. False-discovery procedures address a different error criterion and have dependence assumptions of their own. Report the tested family, correction, permutation count, and sensitivity to W . Avoid presenting an uncorrected cluster map as a confirmed set of findings.

Aggregation can change the pattern before any test is run. Put values [1, 1] in the northern row of a two-by-two grid and [9, 9] in the southern row. Group by rows and the area means are 1 and 9. Group by columns and both means are 5. The underlying four values are unchanged. This is an original illustration of the modifiable areal unit problem: the scale and arrangement of reporting units affect summaries. Repeat a consequential comparison under plausible alternative boundaries and resolutions.

A related mistake is inferring individual relationships from area averages. Consider two people in one synthetic zone with x values [1, 2] and outcomes [4, 3], and two in another with x values [3, 4] and outcomes [6, 5]. Within either zone, the outcome decreases by one when x increases by one. Across zone means, it increases by one. The positive relationship between area means cannot establish the individual relationship. The CDC's map-interpretation guidance warns about this ecological fallacy and unstable rates in small populations. [CDC on interpreting geographic patterns](#)

Make the map answer the research question

Show counts when the question concerns total workload, and rates when the question concerns frequency per eligible person, service opportunity, or time at risk. A zone with 20 reports out of 1,000 opportunities has a 2 percent rate; one with 10 out of 100 has a 10 percent rate. The first has more reports and the second has a higher rate. Neither fact alone identifies the best intervention.

A choropleth of a rate needs its denominator, observation period, boundary version, and missing-data treatment. Use a sequential color scale for ordered positive quantities and a diverging scale for deviations around a meaningful reference. Keep color limits comparable when showing change across maps. Distinguish no observations, missing data, and measured zero. Pair small-denominator rates with uncertainty or a denominator map. A blank area can be a collection gap, not an area without problems.

For a Nearwork-style system, preserve a location record separately from a source claim. Useful fields include original place text, candidate coordinates, geometry type, CRS, resolution or accuracy radius, geocoding method and date, source URL, observation time, extraction confidence, and whether the location is an event site or an organization's office. Do not place an uncertain region-level claim at a crisp centroid without indicating that uncertainty. Several documents repeating one report remain one underlying event unless independent evidence says otherwise.

A defensible shortlist might include one severe well-supported report, one uncertain boundary case, and one poorly observed region where another observation could change the conclusion. The reason for each choice should point back to a question the visit can answer. Sampling only the darkest areas of the map can reinforce the original collection bias.

Practice and a stopping rule

Run `python companion/gis_finance_examples.py` and `python companion/check_gis_finance_examples.py` from the expanded book root. For the snippets above, run Python with the companion directory on its import path. The checks exercise boundary cases, affine inverses, NoData, exact IDW values, and the complete four-location Moran null. They do not certify a production GIS pipeline.

First, change the rectangle boundary convention and reconcile every report. Second, replace one raster cell with NoData and explain both the new mean and the lost coverage. Third, change the chain to a complete graph: every other location becomes a neighbor. The companion checks that every label arrangement then has $I = -1/3$. A graph that removes the relevant notion of locality can remove the question you intended to ask.

Finally, write a one-paragraph collection decision. Name the locations, the uncertainty each visit can reduce, the information that might reverse your choice, and the privacy limits on publishing precise locations. Stop when the geometry, measurement support, and evidence are reliable enough for that decision. A more elaborate map is useful only if it improves the next observation or action.

17 Analyze cash flows returns and risk

Start with an engineering investment

An engineering team can spend \$10,000 now to improve a service. The proposal forecasts \$4,000 of net operating savings at the end of each of the next three years. A spreadsheet says the project makes \$2,000. Is that the right comparison? What changes if the savings arrive late, demand falls, or the project ties up money needed for another commitment?

This chapter starts with that decision. We will value cash flows at a common date, distinguish different kinds of returns, connect portfolio risk to Chapter 14's covariance matrix, and build a tiny backtest whose timing can be audited row by row. The aim is to understand what a financial result means and what evidence would support using it.

All monetary amounts, returns, portfolios, and scenarios here are synthetic educational examples. They are not current market estimates, an evaluation of a real investment, or personalized investment advice. The companion contains no live prices, trading connection, or recommended asset allocation.

The central discipline remains the same as in the scheduler chapters: define the alternative, preserve units and timing, test the accounting, and separate what was measured from what was assumed. A precise answer to the wrong cash-flow question can be more misleading than a rough answer to the right one.

Put every cash flow on a dated timeline

A cash flow is an amount entering or leaving the decision maker's account at a specified time. Use positive signs for inflows and negative signs for outflows. Record currency, date, nominal or inflation-adjusted units, and which alternative the flow belongs to. Profit, revenue, avoided cost, and available cash are different quantities.

For the improvement proposal, use incremental cash flows relative to keeping the existing system. Include implementation, ongoing maintenance, migration downtime, and any remaining resale or disposal value that the comparison actually changes. Avoided staff time becomes a cash saving only if it changes paid spending; otherwise it may represent capacity or another operational benefit worth reporting separately. Sunk spending that cannot change with the decision is not a new incremental outflow.

Our deliberately simplified timeline is:

Time in years	Incremental cash flow in USD	Meaning
0	-10,000	Pay for the improvement now
1	4,000	First year net savings
2	4,000	Second year net savings
3	4,000	Third year net savings

The undiscounted sum is \$2,000. To compare money at different dates, choose a discount rate whose time unit and interpretation match the flows. If one dollar now becomes $1+r$ dollars after one period, then one dollar received after t periods has present value $1/(1+r)^t$. Net present value adds those discounted flows, including any flow at time zero:

$$\text{NPV} = \sum \text{over } t \text{ of } \text{cashflow}_t / (1 + r)^t$$

Present-value analysis gives a common-date representation of dated payments. Choosing a rate and deciding whether the forecasts are credible are separate substantive tasks. MIT's finance material develops cash-flow valuation and compounding conventions; the calculations below are original teaching fixtures. [MIT present value relations](#)

At a stipulated annual discount rate of 5 percent, the three savings have present values approximately \$3,809.52, \$3,628.12, and \$3,455.35. Their sum minus the initial cost is \$892.99. At 10 percent, NPV is approximately -\$52.59. A positive NPV is conditional on the proposed alternative, cash flows, and chosen rate; it is not a guarantee that money will arrive.

```
import numpy as np
from gis_finance_examples import npv
cashflows = [-10000, 4000, 4000, 4000]
assert np.isclose(npv(cashflows, .05), 892.9921174819128)
assert np.isclose(npv(cashflows, .10), -52.59203606311212)
```

Notice that the companion places the first entry at $t = 0$. Spreadsheet and library NPV conventions can differ about whether the first input is discounted by one period. A one-period shift can change the decision. Test the function with an immediate payment before trusting a larger model. For irregular dates, supply explicit elapsed times and state the day-count convention; integer array indices do not know what a calendar year means.

Make rates and units agree

If a nominal annual rate of 12 percent is compounded monthly, the monthly rate is $0.12/12$ and the effective annual rate is $(1 + 0.12/12)^{12} - 1$, approximately 12.683 percent. If 12 percent is instead an effective annual rate, the equivalent monthly rate is $(1.12)^{(1/12)} - 1$, approximately 0.949 percent. These are different contracts expressed with superficially similar numbers.

Continuous compounding writes a growth factor as $\exp(k t)$, where k is a continuously compounded rate per unit time. The equivalent one-period effective rate is $\exp(k) - 1$. This is a change of representation, not additional economic growth. Store whether a quoted rate is periodic, effective annual, nominal with a compounding frequency, or continuously compounded.

Inflation introduces another unit choice. If nominal wealth grows by factor $1 + r_{\text{nominal}}$ while the price level grows by factor $1 + \text{inflation}$, real purchasing power grows by their ratio:

$$1 + r_{\text{real}} = (1 + r_{\text{nominal}}) / (1 + \text{inflation})$$

With a 5 percent nominal rate and 2 percent inflation, the real rate is about 2.941 percent. Subtracting the two percentages gives a useful small-rate approximation, not the exact answer. Discount nominal flows at a consistent nominal rate, or real flows at a consistent real rate. Mixing them can count inflation twice or not at all.

Do not add different currencies without an exchange-rate convention. Do not apply an annual rate once per monthly row. Do not interpret 5 as a 5 percent return when the code expects 0.05. A financial table benefits from the same schema discipline as an engineering trace: amount, currency, timestamp, unit, sign, and provenance are part of the measurement.

Test the assumption that controls the decision

Solve for the annual savings needed to break even at 5 percent. If the same savings amount S arrives in each of three years:

$$\begin{aligned} 0 &= -10000 + S \times [1/1.05 + 1/1.05^2 + 1/1.05^3] \\ S &= 10000 / [1/1.05 + 1/1.05^2 + 1/1.05^3] \end{aligned}$$

The break-even amount is approximately \$3,672.09 per year. The forecast is \$4,000, leaving only about \$327.91 per year of margin under these assumptions. That is a more actionable finding than quoting a positive NPV to two decimals: the team should verify whether net savings can reliably exceed the break-even level.

The companion evaluates three annual-savings scenarios. At \$3,000, NPV is $-\$1,830.26$; at \$4,000, it is \$892.99; at \$5,000, it is \$3,616.24. These are scenarios, not confidence limits and not equally likely outcomes unless you explicitly supply that probability model. The graph also varies the discount rate so the sensitivity is visible.

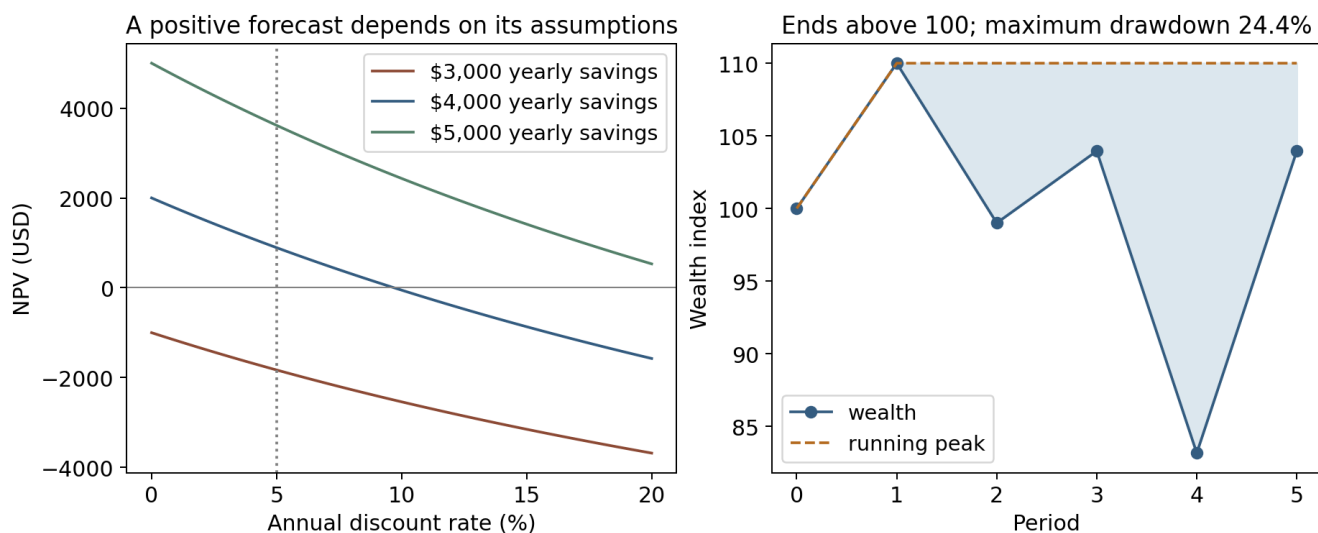


Figure 13. Left: NPV for a \$10,000 outlay followed by three equal year-end savings, under three synthetic savings scenarios. Right: an unrelated synthetic wealth path ends above its starting value but experiences a 24.4 percent peak-to-trough drawdown. Both panels show why one terminal number leaves important information out.

Uncertain cash flows can be treated as a vector C with covariance matrix Sigma_C . For a fixed discount rate, collect the discount factors into a vector d . Then $\text{NPV} = d^T C$, its expected value is $d^T E[C]$, and its variance is $d^T \text{Sigma}_C d$. The initial fixed cost contributes no variance. These are the same linear-combination identities used in Chapter 14.

A shared demand error may affect all three annual savings in the same direction. Treating those errors as independent can understate uncertainty. A Monte Carlo model should draw a coherent scenario, including shared drivers, and then calculate the entire cash-flow path. Report uncertainty about the model inputs separately from numerical Monte Carlo error. Simulating more draws reduces computational noise; it does not make the forecast more trustworthy.

The internal rate of return, or IRR, is a rate that makes NPV zero. It can be useful, but it is not always unique. The synthetic cash flows $[-100, 230, -132]$ have zero NPV at both 10 percent and 20 percent: multiply the NPV equation by $(1+r)^2$ and factor the resulting quadratic. A solver returning one root has not established that it is the only economically relevant answer. Nonconventional sign changes, incompatible project scale, and timing differences all argue for inspecting the NPV profile rather than ranking projects by one IRR.

Liquidity is another question. A project can have positive NPV and still require more cash early than the team can make available. Track the dated cumulative cash balance and commitments alongside valuation. A good decision record states the alternative, break-even assumptions, downside scenario, and practical funding constraint.

Distinguish an asset return from money added to an account

For a positive asset price P , the price-only simple return over one interval is $(P_t - P_{(t-1)})/P_{(t-1)}$. If an investor also receives a cash distribution D_t during that interval, a simple total-return calculation uses $(P_t - P_{(t-1)} + D_t)/P_{(t-1)}$, under the stated timing convention. Splits, distributions, delistings, and price adjustments must be handled consistently.

An adjusted-price series may already account for distributions. Adding dividends again would double-count them. Raw prices around a split can create a fictitious crash if the share count adjustment is omitted. Keep the provider's return definition and adjustment policy with the data. The important question is how the value of the held position changed, not whether one quoted number rose.

Deposits and withdrawals are external cash flows, not investment returns. If an account starts at \$100, earns 10 percent, receives a \$90 deposit, and then loses 10 percent, it ends at \$180. The two market subperiod returns compound to -1 percent. Ending wealth minus the \$190 of total contributions gives a \$10 loss, which answers a different question because more money was exposed during the losing interval.

A time-weighted return chains subperiod returns around external flows. A money-weighted return solves a dated cash-flow equation and reflects their timing. Neither measure is automatically more honest; choose the one that matches the question. Evaluating a strategy's return process and evaluating an investor's experienced outcome are related but distinct tasks.

Compound simple returns and add log returns

A simple return R_t multiplies wealth by $1 + R_t$. With no deposits, withdrawals, or omitted fees:

$$\begin{aligned}\text{wealth}_T &= \text{wealth}_0 \times \text{product over } t \text{ of } (1 + R_t) \\ \text{total return} &= \text{product over } t \text{ of } (1 + R_t) - 1\end{aligned}$$

A gain of 10 percent followed by a loss of 10 percent takes \$100 to \$110 to \$99. The arithmetic mean return is zero, but terminal wealth is one percent lower. Returns do not add across time because each acts on a different base.

For positive gross returns, define a log return $g_t = \log(1 + R_t)$. Products become sums:

$$\begin{aligned}\text{sum}(g_t) &= \log(\text{wealth}_T / \text{wealth}_0) \\ \text{total return} &= \exp(\text{sum}(g_t)) - 1\end{aligned}$$

Use $\log1p$ and $\expm1$ for stable numerical calculation near zero. A simple return of exactly -1 means total loss and has no finite log return. Our wealth function permits total loss; log-return analysis requires a stricter domain. Leveraged strategies may require additional accounting for liabilities, margin, and insolvency rather than blindly applying a positive-wealth formula.

For the five synthetic returns [10%, -10% , 5%, -20% , 25%], wealth is [100, 110, 99, 103.95, 83.16, 103.95], including the starting value. The arithmetic mean is 2 percent per period. The

geometric mean is $(1.0395)^{(1/5)} - 1$, approximately 0.778 percent per period. It is the constant rate that reproduces the same final growth across those five equal periods.

```
import numpy as np
from gis_finance_examples import wealth_path
returns = np.array([.10, -.10, .05, -.20, .25])
wealth = wealth_path(returns)
total = np.expml(np.loglp(returns).sum())
assert np.isclose(wealth[-1], 103.95)
assert np.isclose(total, .0395)
```

For small returns, expanding $\log(1+R)$ gives approximately $R - R^2/2$. Taking expectations suggests that variation lowers average log growth relative to arithmetic average return. More precisely, the second-order correction involves $E[R^2] = \text{variance}(R) + E[R]^2$. This approximation needs small returns and does not replace direct compounding, especially with large losses or heavy tails.

Log returns add across time for one wealth path. They do not generally add across assets with portfolio weights. A portfolio's one-period simple return is a weighted sum of constituent simple returns under the appropriate beginning-of-period weights. Taking a weighted average of their log returns is a different operation.

Measure the path as well as the endpoint

Define the running peak H_t as the largest wealth reached at or before t . The drawdown is $D_t = \text{wealth}_t / H_t - 1$. It is zero at a new high and negative below that high. The maximum drawdown magnitude is $-\min(D_t)$.

In the five-period example, the peak is \$110 and the later trough is \$83.16, giving $83.16/110 - 1 = -24.4$ percent. The final value of \$103.95 is still 5.5 percent below that peak even though it is 3.95 percent above the starting value. A terminal gain can therefore conceal a difficult intervening loss and an unrecovered peak.

Drawdown depends on observation frequency and order. Monthly closing values can miss a much larger intramonth drawdown. Reordering the same simple returns leaves their compounded terminal value unchanged but can change maximum drawdown and time underwater. An unrealized recovery beyond the sample should not be recorded as if it already happened.

For accounts with external flows, raw balances can make a withdrawal look like an investment loss and a deposit look like a recovery. Use an appropriately unitized return index when measuring strategy drawdown. Keep account liquidity analysis separate.

Use covariance to understand a portfolio

Suppose one period's asset-return vector is R and beginning-of-period weights are w . For an unlevered fully invested two-asset portfolio, weights sum to one and the one-period return is $w^T R$. Its expected return is $w^T \mu$, where μ is the vector of expected returns. Its variance is:

```
portfolio variance = w^T Sigma w
```

This expands every pairwise product of weighted deviations. For two assets A and B, with weight w in A and $1-w$ in B:

$$\begin{aligned} \text{variance} = & w^2 \sigma_A^2 + (1-w)^2 \sigma_B^2 \\ & + 2 w (1-w) \text{covariance}(A, B) \end{aligned}$$

The cross-term is why diversification depends on how assets move together, rather than merely on how many names are held. Mean-variance portfolio analysis formalizes this relationship, while leaving open whether mean and variance are sufficient for the actual decision. [MIT portfolio theory](#), [Markowitz on portfolio theory](#)

Take synthetic per-period standard deviations of 20 percent for A and 10 percent for B, with correlation 0.5. Their covariance is $0.5 \times 0.2 \times 0.1 = 0.01$. The covariance matrix is $\begin{bmatrix} 0.04, & 0.01 \\ 0.01, & 0.01 \end{bmatrix}$. At equal weights, variance is 0.0175 and standard deviation is about 13.229 percent. Averaging the individual standard deviations would give 15 percent and miss the covariance effect.

The companion also estimates a sample covariance from four paired return observations, keeping periods in rows and assets in columns. It checks the identity between $w^T S w$ and the direct sample variance of the portfolio-return column. This is a powerful implementation test: two independent calculation paths should agree when weights and observations match.

```
import numpy as np
from gis_finance_examples import portfolio_variance
R = np.array([[.02,.01], [-.01,.00], [.03,.02], [-.02,-.01]])
w = np.array([.5,.5])
S = np.cov(R, rowvar=False, ddof=1)
assert np.isclose(portfolio_variance(w, S), np.var(R @ w, ddof=1))
```

Estimated covariance is not a permanent property of the assets. Missing observations, asynchronous timestamps, regime shifts, and small samples affect it. Pairwise deletion can even produce a matrix that is not positive semidefinite, implying impossible negative variances for some combinations. The companion checks symmetry and positive semidefiniteness within numerical tolerance. Passing those checks establishes algebraic consistency, not forecast quality.

An optimizer can magnify estimation error by concentrating on apparently attractive means or covariance directions. Constraints, shrinkage, and simpler baselines can improve robustness, but they add assumptions to be tested out of sample. A portfolio that looks diversified across names can still share one common exposure. Stress its drivers and liquidity, not only its historical covariance.

Weights also change with performance. Buying two assets once at 50/50 does not maintain 50/50 indefinitely. Rebalancing every period and buying-and-holding are different strategies with different turnover and costs. For each interval, use the weights actually held at its start, not the desired weights or the final portfolio composition.

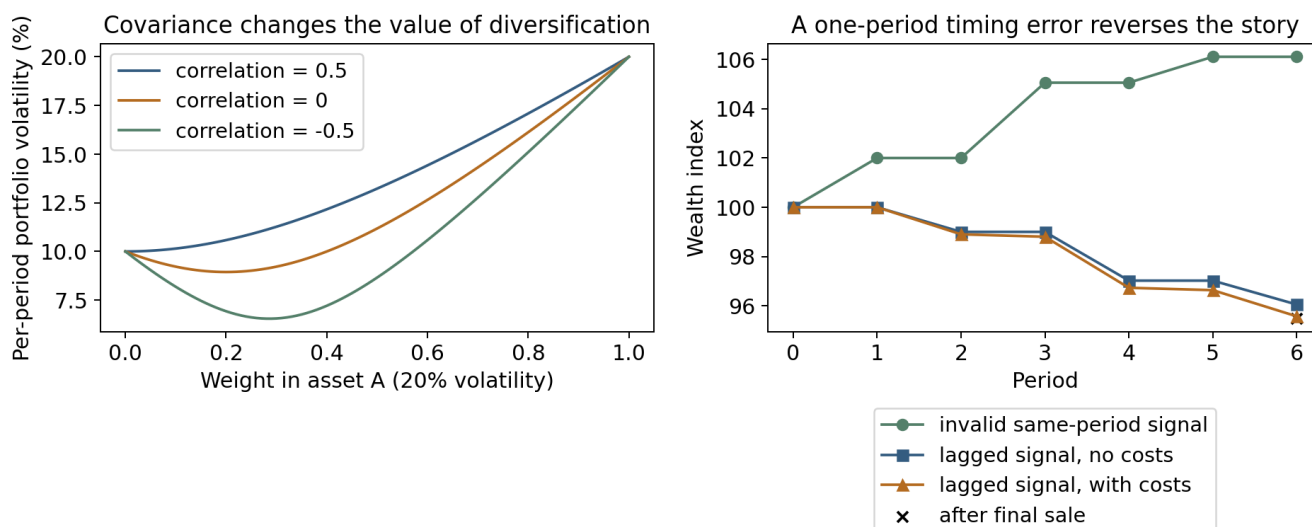


Figure 14. Left: synthetic portfolio volatility under three correlations, holding individual volatilities fixed. Right: six invented returns produce an apparent gain when the signal improperly sees its own period's return, and a loss when the signal is correctly delayed. Transaction costs reduce the delayed strategy further; the final sale is included in its reported terminal wealth.

Treat annualization as a model assumption

If period returns are independent with equal variance σ^2 , the variance of their sum is $m \sigma^2$ over m periods. That produces the familiar square-root-of- m volatility scaling for an additive-return approximation. Log returns provide an additive representation, but independence and stable variance are still assumptions. The exact distribution of compounded simple returns is a different calculation.

For a stationary additive series with lag- k autocovariance γ_k :

$$\begin{aligned} \text{variance of } m\text{-period sum} &= m \gamma_0 \\ &+ 2 \sum_{k=1}^{m-1} (m-k) \gamma_k \end{aligned}$$

To derive it, expand the variance of a sum and count the $m-k$ pairs separated by lag k in each direction. Positive serial covariance can make square-root scaling understate long-horizon variation. Negative covariance can have the opposite effect. Irregular intervals require additional care; 252 is not a universal number of observations per year.

The Sharpe ratio divides average excess return by its standard deviation over a specified period. Excess return subtracts a compatible benchmark cash return, not an arbitrary annual number from each daily row. The ratio is estimated with uncertainty, and converting it across horizons requires assumptions about serial dependence. Lo's original analysis explains why automatic annualization can fail. [Lo on the statistics of Sharpe ratios](#)

A high estimate from a short, selected sample is weak evidence. If risk is dominated by rare losses, standard deviation alone may miss what matters. Show the return horizon, sample length, benchmark, uncertainty method, and path-level losses instead of presenting a ratio without context.

Inspect tail losses and the sample supporting them

Define loss as a positive amount when the portfolio loses money; a gain can therefore be a negative loss. Value at risk at level α is a specified α quantile of that loss distribution. It marks a cutoff, not a maximum possible loss. Expected shortfall averages the worst $1-\alpha$ probability mass under the model.

With discrete observations, the exact quantile and tail convention matter. A definition that remains precise with ties is the integral of the loss quantile function from α to 1, divided by $1-\alpha$. It includes only the fraction of probability mass needed at the cutoff. Acerbi and Tasche examine why definitions that agree for continuous distributions can differ for distributions with point masses. [Acerbi and Tasche on expected shortfall](#)

Take ten equally weighted synthetic losses in thousands of dollars: [0, 0, 0, 0, 1, 1, 2, 3, 4, 10]. Define the empirical quantile as the smallest loss whose cumulative probability reaches α . At $\alpha = 0.80$, VaR is 3. The worst 20 percent consists of losses 4 and 10, so expected shortfall is 7. Averaging every observation greater than or equal to 3 would include 30 percent of the sample and answer a different question.

At $\alpha = 0.85$, VaR is 4. The worst 15 percent consists of all the mass at 10 and half the mass at 4. Its expected shortfall is $(0.10 \times 10 + 0.05 \times 4) / 0.15 = 8$. The companion implements this probability-mass calculation directly and tests the fractional case.

Ten observations cannot support a reliable estimate of an extreme real-world tail. At a 99 percent level, even 1,000 independent observations supply only about ten observations in the nominal upper tail. Dependence and changing conditions can reduce the usefulness of that evidence further. A Gaussian model supplies unobserved tail shape through assumptions; a historical empirical distribution cannot include an event absent from its sample.

Stress scenarios answer a complementary question: what if a specified adverse event occurs? They do not provide its probability by themselves. For the engineering project, stress lower demand, delayed benefits, and a correlated maintenance overrun. For a return process, stress loss concentration, market closure, funding pressure, or a change in transaction costs. Label scenario probabilities as assumptions when they are assumptions.

Put the signal on the correct side of time

A backtest reconstructs what a rule could have done using information available at each decision time. This is a data-availability problem before it is an optimization problem. A timestamp can refer to when an event occurred, when a vendor published it, when the system received it, or when it was revised. Features must respect the relevant availability time.

Use six synthetic interval returns: [2%, -1%, 3%, -2%, 1%, -1%]. At the end of interval t , form a signal that is one if that interval's return was positive and zero otherwise. If you multiply this signal by the same interval's return, the rule earns only the positive outcomes. Starting at 100, it ends at 106.1106. That is an invalid look-ahead result because the signal was not known at the beginning of the interval it supposedly traded.

The earliest eligible held position in this teaching model is the previous interval's signal. Starting in cash gives positions [0, 1, 0, 1, 0, 1]. The resulting gross wealth is $100 \times 0.99 \times 0.98 \times 0.99 = 96.0498$. Delaying the signal reverses the conclusion.

```
import numpy as np
from gis_finance_examples import causal_backtest
r = np.array([.02, -.01, .03, -.02, .01, -.01])
signal = (r > 0).astype(float)
result = causal_backtest(r, signal, cost_rate=.001, liquidate=True)
assert np.allclose(result["position"], [0,1,0,1,0,1])
assert np.isclose(result["terminal_wealth"], 95.47494002744416)
```

The model charges a one-way fee k on the wealth moved between cash and the asset, before the next holding interval. For binary positions, turnover is the absolute change in position. The period growth factor is $(1 - k \times \text{turnover}_t) \times (1 + \text{position}_t \times R_t)$. With $k = 0.001$, five position changes during the sample and the final sale give six one-way trades. Terminal wealth after liquidation is about 95.47494.

Subtracting $k \times \text{turnover}$ from a return is a common first-order cost approximation. The companion uses the multiplicative convention so that fee timing is explicit. It assumes zero cash yield, no taxes, no borrowing, a binary position, and an idealized boundary execution price. A real close-based signal requires an executable quote after information arrives; it cannot simply assume a fill at an already observed closing price. Spreads, slippage, market impact, latency, and failed fills may matter more than the explicit fee. The SEC's educational bulletin explains the cumulative effect of fees on wealth. [SEC on fees and expenses](#)

A useful causality test changes future data and reruns the pipeline. Positions already chosen should remain unchanged. If replacing the last observation changes a trade near the start, a feature, scaler, threshold, or data-cleaning decision is seeing the future. This test catches more than an obvious missing shift.

Evaluate the entire research process

Split time in the order the strategy would encounter it. Within each training window, fit preprocessing, estimate parameters, choose thresholds, and construct any feature selection using only data available there. Use a later validation period for model choices and an untouched final evaluation period for the frozen procedure. Repeatedly viewing that final period turns it into another validation set.

If a target measures a future multi-period return, training labels near a split boundary may reach into the test period. Separate or remove overlapping label windows as the task requires. A walk-forward schedule should state the training length, refit frequency, forecast horizon, information lag, execution rule, and gap or purge rule. Randomly mixing dates can give a model information about the regime it is supposedly forecasting.

Selecting the best of many strategies introduces another bias even when each individual backtest has no timing bug. If many noisy candidates are tried, the reported winner is likely to benefit from luck. Preserve every attempted configuration and the metric used to choose it, including manual experiments. Bailey and colleagues analyze backtest overfitting as a property of this selection process. [The probability of backtest overfitting](#)

Use a universe that could have been known at each historical date. Today's surviving firms or funds are not the same set that was available then. Include relevant closures, delistings, mergers, and contemporaneous eligibility rules. Missing terminal outcomes can bias performance even in a dataset that retains an identifier. Shumway's original study documents a historical delisting-return problem; it is evidence for auditing such fields, not a claim that every current vendor has the same defect. [Shumway on delisting bias](#)

When estimating uncertainty, resample in a way that respects the question. Independent resampling of daily rows destroys serial dependence. A block bootstrap resamples consecutive stretches, preserving some local dependence while introducing its own stationarity and block-length assumptions. The companion's twelve-point synthetic series has a 95 percent percentile range for the resampled mean of about $[-1.167\%, 1.083\%]$ with single-row blocks and $[-1.75\%, 1.75\%]$ with length-three blocks. Those are conditional bootstrap summaries of an invented sequence, not a forecast interval or a reliable inference from twelve market observations.

Try several justified block lengths and inspect the sensitivity. Resample synchronized rows when several assets share market shocks. If the aim is uncertainty in a strategy chosen by training, repeating only the final return summary misses selection uncertainty: the resampling or validation procedure must include the relevant fitting and selection steps. No resampling method creates evidence for a regime absent from the source data.

Practice and a stopping rule

Run `python companion/gis_finance_examples.py` and `python companion/check_gis_finance_examples.py`. For chapter snippets, place the companion directory on Python's import path. Inspect `finance_backtest.csv`: each row exposes the observed return, newly formed signal, held position, turnover, and net wealth before the final sale. Reconcile the last row with the additional liquidation cost.

Change the project savings until NPV crosses zero and compare the numerical answer with the analytic break-even amount. Move a benefit one year later and predict the direction of change before rerunning. Check both roots of the IRR counterexample. Then reorder the five return observations: verify unchanged terminal wealth but potentially changed drawdown.

For the backtest, replace future returns and verify that earlier positions do not change. Compare zero costs, stated costs, and a stressed cost assumption. Explain why the invalid same-period strategy is a software counterexample rather than an investment opportunity. Finally, replace one covariance off-diagonal entry without its symmetric counterpart and confirm that validation rejects the matrix.

Stop when the accounting reconciles, the timeline is causal, the decision is robust enough to its important assumptions, and the remaining uncertainty is visible to the decision maker. For the engineering proposal, the next useful step might be measuring actual net savings against the \$3,672 annual break-even level. The purpose of financial analysis is to make that next decision clearer, not to turn a fragile forecast into a confident-looking number.

18 Optimize under uncertainty

A better score is only useful when it answers the decision

Chapter 6 showed a scheduler choosing the wrong action for the completion-time objective even after a missing candidate was restored. Optimization could perfectly maximize that score and still produce the undesirable schedule. Before improving the optimizer, check the objective, the feasible actions, and the model connecting actions to outcomes.

This chapter builds a smaller problem you can solve by hand, enumerate in code, and analyze under uncertainty. The project is choosing how many workers to run for a fixed batch. We use a deliberately synthetic timing model:

$$T(n) = a/n + b + c(n - 1)$$

T is duration in seconds. The worker count n is a positive integer. The term a/n represents perfectly divisible work, b represents fixed overhead, and $c(n - 1)$ represents an increasing coordination penalty. For the nominal example, $a = 100$, $b = 2$, and $c = 0.2$. These coefficients were invented for instruction. They were not fitted to the scheduler, an agent system, or measured hardware.

At one worker, T is 102 seconds. At ten, it is 13.8 seconds. Extra workers reduce the divisible work but increase coordination. The model makes that tradeoff visible. It omits memory contention, discrete tasks, network saturation, startup delays, and many other effects a deployed system might require.

Define the feasible problem

An optimization problem contains decision variables, an objective, and constraints. Here the decision is n , the objective is to minimize $T(n)$, and the basic constraints are integer n between 1 and 32. A memory budget might restrict n to at most 12. A cost budget or a failure-rate limit would create a different feasible set.

If the goal is cost, measure cost. If the goal is a latency deadline, minimizing mean duration may be insufficient. A weighted sum of dollars, seconds, and failure probability requires conversion weights that express a real tradeoff. A hard permission or safety requirement generally belongs in a constraint or gate, not in a penalty small enough to be offset by speed.

Write a baseline and a stopping condition before tuning. A baseline could be the current worker count or a simple enumeration of all feasible counts. A stopping condition could be finding a configuration that meets the deadline with a stated uncertainty margin at acceptable cost. “The optimizer converged” describes a numerical event, not the user's outcome.

A derivative tells you which direction helps locally

Treat n temporarily as a continuous positive number. The derivative of T with respect to n is $-a/n^2 + c$. It describes the local change in seconds for a small change in worker count. Setting this derivative to zero gives $n = \text{square root of } (a/c)$.

For the nominal coefficients, this is square root of 500, approximately 22.36068. The second derivative is $2a/n^3$, which is positive when a and n are positive. The curve bends upward everywhere in the positive domain, so this stationary point is the continuous global minimum. The fixed term b changes the height of the curve without changing the optimum.

Workers are integers. Evaluate the neighboring feasible values rather than rounding without checking. $T(22) = 10.7454545$ seconds and $T(23) = 10.7478261$ seconds. Enumeration over 1 through 32 confirms that 22 is best for the nominal model. The difference between 22 and 23 is only about 0.00237 seconds. Such a small model difference would require correspondingly strong evidence before it justified a real operational choice.

With the additional constraint n at most 12, the optimum is 12 and the modeled duration is 12.53333 seconds. A solution that ignores the constraint is not a candidate you can deploy.

```
import numpy as np
workers = np.arange(1, 33)
seconds = 100 / workers + 2 + 0.2 * (workers - 1)
best = workers[np.argmin(seconds)]
feasible = workers <= 12
budget_best = workers[feasible][np.argmin(seconds[feasible])]
```

For a larger differentiable problem, the gradient contains one partial derivative per variable. Moving against it can reduce an objective locally, but the step size, constraints, scaling, and curvature matter. Convexity gives useful guarantees under the appropriate conditions; general nonlinear problems can contain several local minima. Solver choice follows the problem's structure. [Boyd and Vandenberghe on convex optimization](#), [SciPy optimization guide](#)

Check numerical behavior against a simple answer

A finite difference estimates a derivative by changing an input slightly. The centered estimate is $[f(x+h) - f(x-h)] / (2h)$. A large h averages over curvature. A very small h can amplify floating-point cancellation or simulation noise. Try several scales and compare with an analytic derivative when one is available.

An optimizer's tolerance should relate to useful precision. Requesting many decimal places does not make uncertain measurements more accurate. Check the objective value, constraint residuals, termination status, and sensitivity to starting points. For small discrete problems, exhaustive enumeration gives a transparent reference against which to test a more elaborate search.

Poorly scaled parameters can also cause trouble. If one coordinate is measured in millions and another in thousandths, a common numerical step can be inappropriate. Scale variables deliberately and translate the answer back to physical units. Do not mistake a parameter rescaling for a new physical model.

When the objective is noisy, repeated evaluations at the same decision may disagree. Stochastic gradient methods estimate gradients from samples, often using data batches. Their randomness can be useful computationally, but it does not eliminate the need for held-out

evaluation or a defensible stopping rule. A noisy score that rises after many trials may reflect selection of favorable noise.

Sensitivity is a question about what could change the answer

At a fixed worker count, the duration's sensitivities to the three coefficients are simple: the derivative with respect to a is $1/n$, with respect to b is 1, and with respect to c is $n - 1$. At $n = 22$, these are approximately 0.04545, 1, and 21.

The largest derivative does not automatically identify the most important uncertainty source. The coefficients have different scales and uncertainty. Multiply each sensitivity by a plausible input standard uncertainty before comparing its effect on duration. A parameter with a large derivative but known very precisely may contribute little uncertainty.

A local sensitivity asks about a small change near specified values. A global sensitivity study explores a defined input range or distribution and can reveal interactions, thresholds, or changing parameter importance. Varying one input at a time is easy to interpret but can miss combinations where two inputs change together. Keep the range and dependence assumptions visible.

For this teaching problem, suppose a is uniformly distributed between 80 and 120, b between 1 and 3, and c between 0.1 and 0.3, independently. These are explicit assumptions about unknown coefficients, not estimated real-world distributions. A uniform interval of width w has variance $w^2/12$, so the three variances are 133.33333, 0.33333, and 0.00333333.

Propagate uncertainty with the model

For a scalar output $y = f(x)$, collect the local sensitivity coefficients into a vector g and the input covariance into a matrix S . A first-order approximation to output variance is:

$$\text{variance}(y) \approx g^T S g$$

If inputs are independent, off-diagonal covariances are zero and this becomes a sum of squared sensitivities times input variances. With dependent inputs, the covariance terms can increase or decrease uncertainty. The approximation follows a local linearization; strong curvature, discontinuities, or wide input ranges can make it inadequate. [NIST law of propagation of uncertainty](#)

In our model, duration at a fixed n is exactly linear in a , b , and c . Consequently its variance formula is exact under the stated finite-variance input assumptions:

$$\text{Var}[T(n)] = \text{Var}(a)/n^2 + \text{Var}(b) + (n - 1)^2 \text{Var}(c)$$

At 22 workers, the three contributions are approximately 0.27548, 0.33333, and 1.47000 seconds². They sum to about 2.07882, whose square root is 1.44181 seconds. The coordination coefficient contributes about 71 percent of this modeled variance. Measuring coordination more accurately could therefore be more useful than further refining the divisible-work coefficient, if these uncertainty assumptions were supported by real evidence.

This is an uncertainty budget: a record of inputs, their uncertainties, their influence, and their contribution to the result. It gives a measurement plan rather than merely a wider error bar.

NIST uncertainty budgets

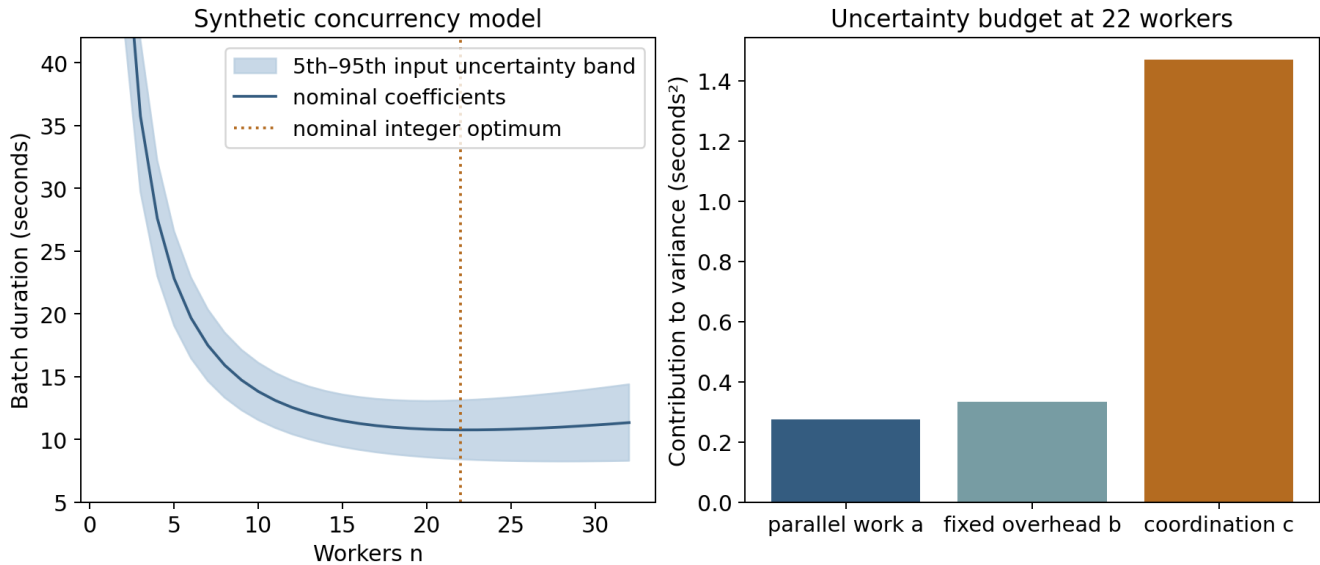


Figure 15. Left: the synthetic timing model with the fifth through ninety-fifth percentiles induced by the assumed input distributions. The vertical range emphasizes the useful worker-count region; the one-worker nominal value is 102 seconds, above the panel. Right: the analytic variance contributions at 22 workers. The shaded band describes input uncertainty under this model, not a confidence band fitted from scheduler data.

Monte Carlo sends uncertainty through the whole calculation

Monte Carlo propagation samples plausible inputs, evaluates the model for each draw, and summarizes the resulting outputs. It is especially useful when a nonlinear transformation makes a simple approximation inaccurate. Its interpretation depends on the distributions and dependence structure supplied to it. The method propagates assumptions; it does not validate them. [JCGM guidance on Monte Carlo propagation](#)

The companion draws 100,000 independent triples using a fixed random seed. It reuses each triple across all 32 worker counts. This makes each comparison refer to the same hypothetical workload coefficients, an example of common random numbers.

```
rng = np.random.default_rng(1801)
N = 100_000
a = rng.uniform(80, 120, N)
b = rng.uniform(1, 3, N)
c = rng.uniform(0.1, 0.3, N)
t22 = a / 22 + b + 21 * c
```

The simulated mean duration is 10.74927 seconds and its sample standard deviation is 1.44266 seconds. The analytic mean is 10.74545 and the analytic standard deviation is 1.44181. Their agreement is a useful code and arithmetic check. It does not test whether the input distributions describe a real system.

The simulated chance that duration exceeds 12 seconds is 0.21400. With independent Monte Carlo draws, its simulation standard error is approximately square root of $[p(1-p)/N]$, here 0.00130. This is uncertainty from finite numerical sampling conditional on the chosen input model. It is distinct from uncertainty about the input distributions, model structure, or deployed workload.

More draws reduce Monte Carlo noise. They do not resolve missing measurements, confounding, a wrong coordination model, or an omitted failure mode. A million draws from a poorly chosen model can produce a stable and misleading answer.

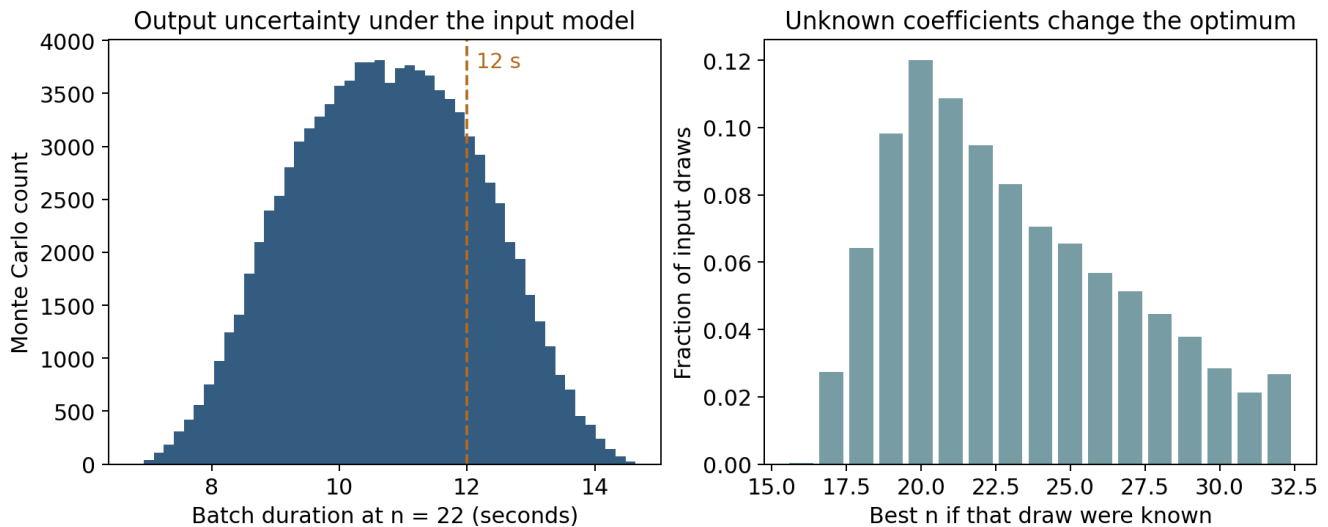


Figure 16. Left: modeled duration at a fixed 22 workers. Right: which worker count would be best if each sampled coefficient triple were known in advance. The right panel describes sensitivity of the optimum; it is not the performance of a deployable policy that can observe unknown coefficients for free.

Nonlinear transformations change the answer

Suppose each batch contains 600 items. Its rate is $600/T$ items per second. The average of those per-batch rates is not the rate obtained by dividing all processed items by total elapsed batch time.

In the simulation, the mean of $600/T$ at 22 workers is about 56.86854 items per second. Dividing 600 by the mean duration gives about 55.81774. For sequential equal-sized batches with negligible gaps, total items divided by total time corresponds to the second calculation. For a question about the distribution of individual batch rates, the first may be relevant. Write the denominator before naming either number throughput.

A first-order approximation to the rate's standard deviation is $[600 / \text{mean}(T)^2]$ times the duration standard deviation. Using the nominal mean gives about 7.49220 items per second, whereas direct Monte Carlo gives about 7.93353. The transformation's curvature matters at this uncertainty scale. Inspect the full distribution when decisions depend on tails or thresholds.

The same issue appears in finance when averaging compounded returns, in image measurement when squaring a scale factor, and in GIS when transforming coordinates before

computing area. Transforming an average generally differs from averaging transformed observations.

Choose a decision rule under uncertainty

Several legitimate objectives give different worker choices. Minimizing nominal duration chooses 22. In this simulation, minimizing the estimated ninety-fifth percentile over the 32 counts chooses 20. Minimizing the worst duration over the assumed coefficient intervals also chooses 20: the worst-case curve is $120/n + 3 + 0.3(n - 1)$, whose continuous minimum is exactly 20.

These are answers to different questions. A worst-case rule protects against all combinations inside the specified box, including combinations that may be extremely unlikely or impossible if inputs are dependent. A percentile rule depends on the input distribution and tail estimation. A mean rule can tolerate rare slow outcomes. Pick the rule from the consequence of failure, then test it on independent evidence.

The difference in nominal duration between 20 and 22 workers is only about 0.05455 seconds in this model. That may be negligible relative to uncertainty or cost. A broad region of acceptable choices can be more useful than one apparently exact optimum. Record the range of decisions meeting your requirements and identify which missing measurement could change the choice.

For stochastic engineering simulations, pair candidate policies on the same generated workload when possible. Compare differences within independent replications; do not treat events within one run as independent runs. A fixed seed helps reproduce a calculation. It does not supply an uncertainty estimate by itself. If policies consume randomness differently, arrange random inputs by stable event or workload identifiers so the pairing means what you intend.

Close the loop with observations

Before applying this model to a real system, measure durations across several worker counts and workload families, inspect residuals, and hold back fresh runs. Test whether the assumed coordination term is adequate and whether the feasible region includes memory, reliability, and cost limits. Use the model to identify informative experiments, not to replace them.

Exercise: suppose uncertainty in c is cut in half while its mean remains 0.2. Its variance contribution becomes one quarter as large, so the duration variance at 22 becomes approximately 0.97632 seconds² and the standard deviation about 0.98809 seconds. Explain why this improves knowledge of the result without changing the nominal optimum. Then change the mean of c and check why the optimum does move.

The useful deliverable is a decision with explicit assumptions, an uncertainty budget, and a test that could overturn it. The mathematics helps you make that deliverable precise.

Mathematical reference and practice

Read a formula as a calculation

A scalar is one number. A vector is an ordered list of numbers; its order and units matter. A matrix is a rectangular array. In this book, a data matrix usually has observations in rows and features in columns, but every important use states the convention.

The symbol n often means a count of observations or workers. A subscript selects an entry: x_i means the i th value of x . A summation asks you to add terms over specified indices. A bar over a quantity commonly means its sample mean. A hat commonly marks an estimate or prediction. These are conventions, not substitutes for definitions.

A transpose, written T as a superscript or `.T` in NumPy, swaps rows and columns. A dot product multiplies matching entries and adds the products. The squared Euclidean norm adds squared entries of a vector. A matrix inverse reverses a square transformation when that inverse exists; many useful problems should be solved without explicitly forming it.

An eigenvector retains its direction when a matrix acts on it. Its eigenvalue is the corresponding scale factor. Singular values describe the strengths of orthogonal input and output directions of a matrix. PCA uses directions that retain variation after specified centering and scaling. A rank- k approximation keeps k such directions and discards the rest.

A derivative describes local change in output per change in input. A partial derivative varies one input while holding the others fixed. A gradient collects partial derivatives. Its entries can have different units, so an unscaled comparison of their magnitudes may be misleading.

A standard deviation describes spread and has the same units as the quantity. A variance is its square and has squared units. Covariance records paired variation and carries the product of two units. Correlation normalizes covariance into a unitless quantity. Standard uncertainty expresses uncertainty as an estimated standard deviation under a stated measurement model; it is not automatically the standard error of a sample mean.

A confidence interval describes a procedure's long-run coverage of a fixed target under repeated sampling and its assumptions. An input-uncertainty interval in a simulation summarizes outcomes induced by a chosen distribution over inputs. A sensitivity range reports what happened when selected inputs or settings changed. Label the kind of interval before interpreting its endpoints.

Spatial and financial terms

A coordinate reference system, or CRS, defines how coordinates relate to locations. Declaring a CRS assigns meaning to existing numbers; transforming coordinates computes numbers in another CRS. A spatial join associates records through a geometric relationship and can produce several matches per input. A raster stores values on cells whose size, origin, orientation, and interpretation matter. NoData identifies unknown or unavailable values rather than physical zero.

Spatial autocorrelation describes association between values at locations considered neighbors under a stated weight matrix. Moran's I is one such summary. Its randomization test requires an exchangeability assumption and a defined tail. Inverse distance weighting, or IDW, estimates a value from distance-weighted observations; its surface is not automatically an uncertainty map.

Net present value, or NPV, sums dated cash flows after discounting them to a common date. A discount rate must match the cash-flow time unit and nominal or real convention. A simple return measures fractional gain over a period after the specified handling of external flows. A log return is the logarithm of one plus simple return and adds across periods on a compatible wealth path.

Drawdown measures a wealth path's decline from its running peak. Volatility is a standard deviation of returns under a stated period and model; it does not describe every form of risk. Value at risk, or VaR, is a specified loss quantile. Expected shortfall, or ES, averages the upper quantile tail with fractional probability mass at a discrete cutoff when necessary. A backtest evaluates a decision rule using historical or simulated records while respecting when information and execution were available.

Four worked checks to keep nearby

For projection, $[3, 1]$ projected onto the line through $[1, 1]$ becomes $[2, 2]$. Its residual $[1, -1]$ has zero dot product with that direction. If your result fails this perpendicularity check, inspect the calculation or the intended error metric.

For the Chapter 14 PCA example, the first component keeps 80 percent of total sample variance. The four reconstructed points are $[1.5, 1.5]$, $[1.5, 1.5]$, $[3.5, 3.5]$, and $[3.5, 3.5]$. Their total squared reconstruction error is 2. Flipping the signs of a component and all its scores changes neither reconstruction nor error.

For image area, 576 square pixels with 0.2-mm sides cover 23.04 mm². If a scale standard uncertainty of 0.002 mm is the only uncertain input, the first-order area uncertainty is 0.4608 mm². Repeated pixels share that scale uncertainty.

For the synthetic worker model, the nominal best integer count from 1 through 32 is 22. Constraining the count to at most 12 makes 12 best. Reducing uncertainty about a coefficient can narrow the output distribution without changing its nominal optimum. Changing the coefficient's mean can move the optimum.

A lab that joins the chapters

Choose one measurable engineering outcome, such as accepted tasks per hour, image-based area error, or research coverage of a region. Write the unit of observation, the outcome's denominator, and the action you are considering. Then list the inputs that could change your conclusion.

Build the smallest table or array that represents those inputs faithfully. Retain identifiers, units, provenance, and missing-value meaning. Make one plot that exposes a mechanism or

comparison. If you compress the data, check what disappears. If you fit a model, keep an independent evaluation split that matches the way the model will be used.

Calculate one answer by hand or by a second method. For a matrix operation, check dimensions and a reconstruction identity. For a spatial result, check the coordinate system and a known distance or area. For a financial series, reconcile beginning value, flows, gains, costs, and ending value. For a simulation, compare a simplified case with an analytic answer.

Finally, alter an assumption that could reverse the decision. Report what stays true, what changes, and what new observation would resolve the important uncertainty. This is a more useful capstone than reproducing a chart without understanding its limits.

Sources and evidence

These references support the definitions, methods, and further reading in the book. All worked datasets and figures are original or drawn from the pinned scheduler simulation. Generated teaching examples are labeled synthetic. Source pages were checked on 2 October 2026; official documentation may evolve independently of the companion runtime.

S01. OpenTelemetry traces

Used in chapter 2. Trace, span, and attribute concepts for instrumentation.

S02. Prometheus histograms and summaries

Used in chapter 2. Aggregation limits of precomputed quantiles and histograms.

S03. Wickham, Tidy Data

Used in chapter 3. One variable per column, one observation per row, and one table per observational unit.

S04. pandas merge documentation

Used in chapter 3. Join cardinality validation and pandas null-key behavior.

S05. pandas scaling guidance

Used in chapter 3. In-memory scaling, reducing loaded data, and chunking.

S06. NIST exploratory data analysis

Used in chapter 4. Exploratory graphics and assumption checking.

S07. NIST intervals for paired differences

Used in chapter 5. Paired mean-difference confidence intervals.

S08. SciPy bootstrap

Used in chapter 5. Bootstrap methods and paired resampling.

S09. ASA statement on p values

Used in chapter 5. Interpretation limits of p values and threshold-only decisions.

S10. NIST multiple comparisons

Used in chapter 5. Bonferroni control for preselected multiple comparisons.

S11. Causal Inference What If

Used in chapter 7, 13. Counterfactual causal framework and further reading.

S12. NIST design selection

Used in chapter 7. Comparative, screening, and response-surface experiment objectives.

S13. Columbia notes on Little's law

Used in chapter 8. Little's law, consistent system boundaries, and time averages.

S14. NIST process monitoring

Used in chapter 8. Control limits versus specification limits.

S15. Forecasting Principles and Practice on time-series cross-validation

Used in chapter 8. Time-ordered rolling-origin evaluation.

S16. Google SRE monitoring

Used in chapter 8. Latency, traffic, errors, saturation, and symptoms versus causes.

S17. Demystifying evals for AI agents

Used in chapter 9. Agent task, trial, grader, transcript, and outcome distinctions.

S18. Zheng and colleagues on LLM judges

Used in chapter 9. Documented judge position, verbosity, and self-enhancement biases; no universal accuracy claim.

S19. scikit-learn probability calibration

Used in chapter 9. Calibration versus discrimination and reliability diagrams.

S20. scikit-learn common pitfalls

Used in chapter 10. Training-only preprocessing and leakage prevention.

S21. scikit-learn cross-validation

Used in chapter 10. Grouped and time-aware cross-validation.

S22. Kapoor and Narayanan on leakage

Used in chapter 10. Leakage risks in machine-learning-based science.

S23. scikit-learn model-evaluation metrics

Used in chapter 10. Classification and regression metric definitions.

S24. Model Cards for Model Reporting

Used in chapter 10. Structured reporting of model use, performance, and limitations.

S25. W3C PROV primer

Used in chapter 11. Entities, activities, and agents in provenance.

S26. Rubin on inference with missing data

Used in chapter 11. Missing-data mechanisms and the conditions behind ignoring missingness.

S27. GeoPandas projections

Used in chapter 11. Coordinate reference systems and assigning versus transforming coordinates.

S28. Gebru and colleagues on datasheets

Used in chapter 11. Dataset motivation, composition, collection, use, and documentation.

S29. Python for Data Analysis

Used in chapter 13. Legally available author online book and practical reading path; no text reproduced.

S30. Fundamentals of Data Visualization

Used in chapter 13. Author manuscript for visualization reading; no figures or text reproduced.

S31. OpenIntro Statistics

Used in chapter 13. Foundational statistics reading recommendation and official availability.

S32. NIST engineering statistics

Used in chapter 13. Engineering-statistics reference reading recommendation.

S33. An Introduction to Statistical Learning

Used in chapter 13. Authors' statistical-learning book and Python edition reading recommendation.

S34. Forecasting Principles and Practice

Used in chapter 13. Forecasting book reading recommendation.

M01. MIT projections and least squares

Used in chapter 14. Geometric connection between projection and least-squares residuals.

M02. NIST mean vector and covariance matrix

Used in chapter 14. Sample covariance definition and observation/feature orientation.

M03. NumPy covariance

Used in chapter 14. rowvar orientation and covariance API.

M04. MIT singular value decomposition

Used in chapter 14. Matrix factorization and singular directions; original example derived separately.

M05. NumPy singular value decomposition

Used in chapter 14. Returned U, singular values, Vh and reconstruction conventions.

M06. scikit-learn clustering guide

Used in chapter 14. Clustering objectives and contrasting method assumptions.

M07. scikit-image image data types

Used in chapter 15. Image dtype and range conventions.

M08. SciPy multidimensional convolution

Used in chapter 15. Convolution and boundary-mode concepts; companion implements its own explicit NumPy reflection convention.

M09. NumPy discrete Fourier transform conventions

Used in chapter 15. Frequency ordering, transform normalization, real-valued transform representation.

M10. scikit-image morphology

Used in chapter 15. Erosion, dilation, opening, closing and footprints.

M11. scikit-image region measurements

Used in chapter 15. Connected-region measurement, spacing, area and coordinates.

M12. Boyd and Vandenberghe Convex Optimization

Used in chapter 18. Author-hosted legal book availability and further study of convex problems.

M13. SciPy optimization guide

Used in chapter 18. Distinguishing solver families and problem structures.

M14. NIST law of propagation of uncertainty

Used in chapter 18. First-order sensitivities, variances, and covariance terms.

M15. NIST uncertainty budgets and sensitivity coefficients

Used in chapter 18. Relating input uncertainty to output uncertainty contributions.

M16. JCGM 101 Monte Carlo propagation

Used in chapter 18. Propagation of distributions, conditions and separation from validity of the input model.

G01. Projections

Used in chapter 16. CRS metadata, declaring versus transforming coordinates, and GeoPandas longitude/latitude order. Does not establish the accuracy of any particular local projection.

G02. Geodesic calculations

Used in chapter 16. Distinguishing ellipsoidal geodesics from the chapter's explicitly spherical approximation. No claim that the synthetic sphere is survey grade.

G03. Merging data

Used in chapter 16. Attribute versus spatial joins, geometric predicates, one-to-many matches and nearest-join options. Rectangle half-open assignment is an original teaching convention.

G04. Geotransform tutorial

Used in chapter 16. Six affine coefficients, top-left corner coordinates, north-up negative pixel height and half-cell center offset. Zonal overlap example independently derived.

G05. GDAL Grid tutorial

Used in chapter 16. Inverse distance to a power interpolation and neighborhood parameters. Chapter fixture uses all points, $p=2$, zero smoothing; measurement-only variance is separately derived.

G06. How Kriging works

Used in chapter 16. Kriging relies on a modeled spatial relationship; prediction error is conditional on model assumptions. No kriging implementation or empirical calibration is claimed.

G07. Cross-validation strategies for data with temporal spatial hierarchical or phylogenetic structure

Used in chapter 16. Structured validation must reflect interpolation versus transfer to new space; dependence can make random validation misleading. Not a universal endorsement of every blocking design.

G08. Global Spatial Autocorrelation with Moran's I

Used in chapter 16. Moran statistic, spatial lag and random-label permutation reference. Chapter explicitly fixes locations and weights and defines its own right tail; it does not assert generic point-process CSR or reproduce software default p-value semantics.

G09. Local Spatial Autocorrelation 1

Used in chapter 16. Local quadrant membership differs from significance; local tests raise multiplicity and permutation-resolution issues. Chapter implements descriptive local values only.

G10. Choropleth Map Design for Cancer Incidence Part 2

Used in chapter 16. Warnings about ecological inference, small denominators and geographic pattern interpretation. Chapter has no disease analysis and its four-person numerical counterexample is original.

F01. Present Value Relations Slides 1-36

Used in chapter 17. Present value, compounding and real/nominal consistency. All project cash flows, break-even numbers and IRR counterexample are original calculations; no recommended discount rate.

F02. Portfolio Theory Slides 1-46

Used in chapter 17. Weighted portfolio returns and the role of covariance in mean-variance analysis. Matrices and return observations are synthetic, not estimates for named assets.

F03. Foundations of Portfolio Theory

Used in chapter 17. Historical foundation and conditional role of mean-variance portfolio criteria. Does not supply the chapter's numerical portfolio or current market claims.

F04. The Statistics of Sharpe Ratios

Used in chapter 17. Sharpe estimates have sampling error; simple annualization is not generally valid under serial correlation. The additive-sum variance identity is derived independently in the chapter.

F05. On the coherence of Expected Shortfall

Used in chapter 17. Expected shortfall definitions need care for discontinuous distributions; precise tail probability mass. Chapter uses upper-tail loss confidence alpha, which must not be confused with an author's lower-tail probability convention.

F06. How Fees and Expenses Affect Your Investment Portfolio

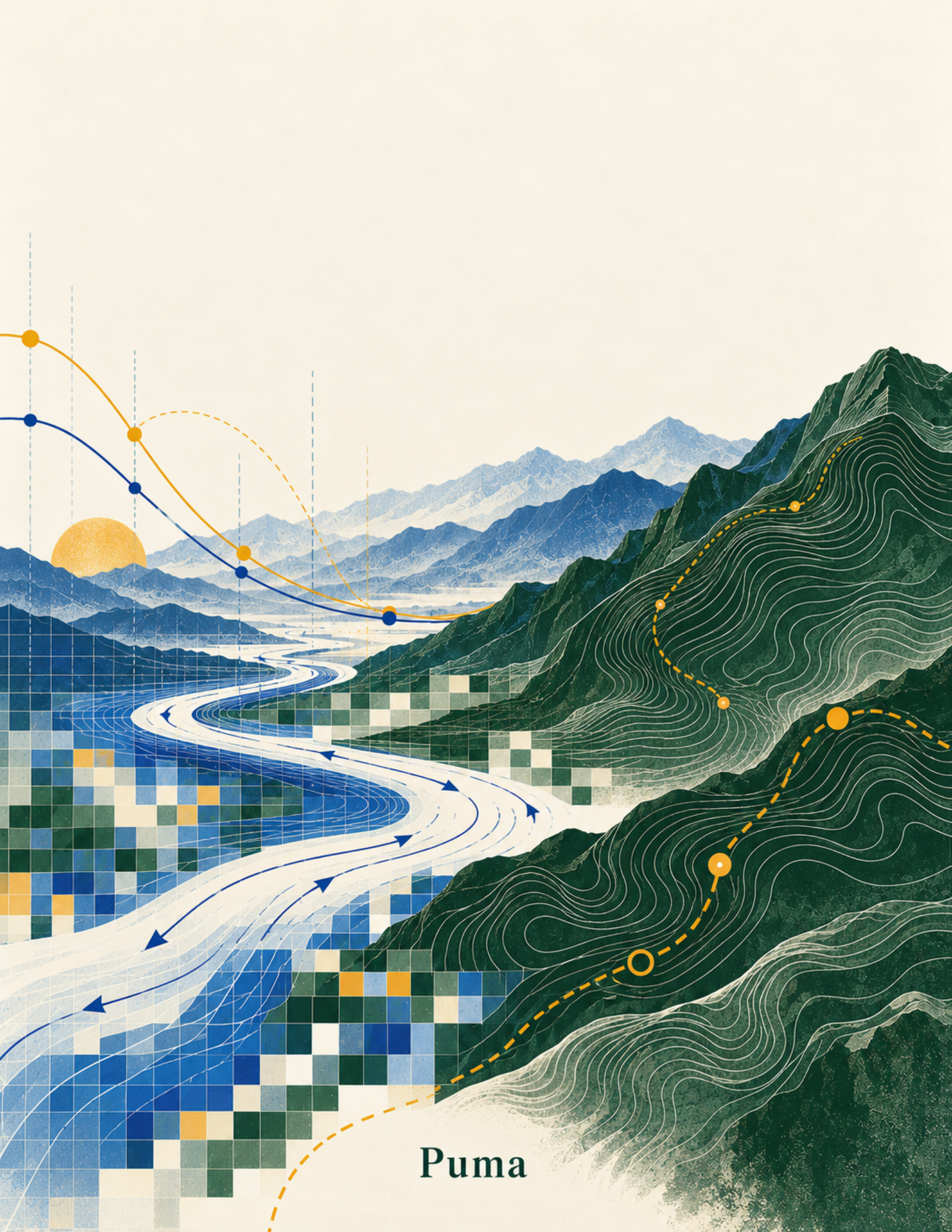
Used in chapter 17. Fees reduce returns and the capital left to compound. Does not justify the toy 0.001 transaction cost as realistic or current.

F07. The Probability of Backtest Overfitting

Used in chapter 17. Trying many alternatives creates selection risk even without a direct look-ahead bug; retain experiment history. Chapter does not implement CSCV or estimate a PBO.

F08. The Delisting Bias in CRSP Data

Used in chapter 17. Historical research example of missing delisting outcomes. Supports auditing terminal observations, not a claim that current CRSP data still has the reported defect.



Puma